



COURSE SLIDES

Wireshark Network Analysis Masterclass

Course Code: C1123

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v5.1 · 5 October 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

PhD — specialises in networking, cyber security, cloud and AI.

DELIVERS

Courses on network analysis, cyber security, cloud and AI for Tertiary Infotech Academy.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

1 Your name and organisation / role.

2 Your experience with Wireshark or network troubleshooting (if any).

3 A slow or broken network problem you would like to be able to explain with packets.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

5

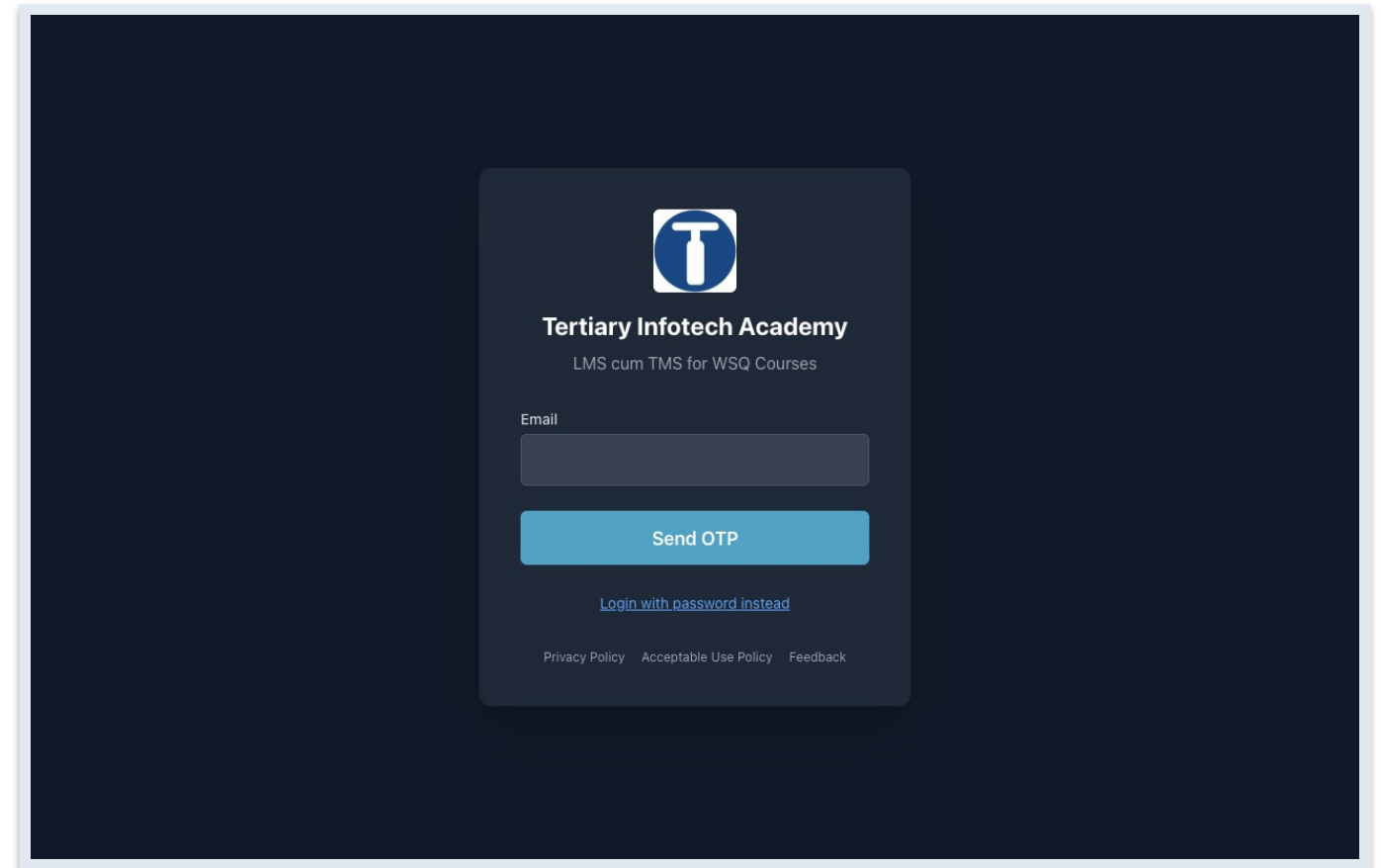
Be punctual; return from breaks on time.

6

Learn by doing — try every hands-on lab.

Download Course Material

- Go to lms-tms.tertiaryinfotech.com.
- Enter your registered email and click Send OTP.
- Key in the one-time password from your inbox to log in.
- Open this course to download the slides, Learner Guide and Lesson Plan.
- Keep them open during class — refer to them as you work through each lab.



<https://lms-tms.tertiaryinfotech.com>

SCHEDULE

Lesson Plan — 4 Days, 8 hours/day (Days 1–2)

Day 1

- Day 1 — Capture and analyst workflow
 - Topic 1: Introduction to Network Analysis and Wireshark (Lab 1)
 - Topic 2: Capture Methods and Capture Filters (Lab 2)
 - Topic 3: Global Preferences and Troubleshooting Profiles (Lab 3)
 - Topic 4: Navigation and Coloring Techniques (Lab 4)

Day 2

- Day 2 — Time, statistics, filters and DNS
 - Topic 5: Time Values and Delay Types (Lab 5)
 - Topic 6: Trace Statistics and VoIP Overview (Lab 6)
 - Topic 7: Display Filters (Lab 7)
 - Topic 8: TCP/IP Communications and Resolution (Lab 8)
 - Topic 9: DNS Traffic Analysis (Lab 9)

SCHEDULE

Lesson Plan — 4 Days, 8 hours/day (Days 3–4)

Day 3

- Day 3 — IP and transport diagnostics
 - Topic 10: ARP Traffic Analysis (Lab 10)
 - Topic 11: IPv4 Traffic Analysis (Lab 11)
 - Topic 12: ICMP Traffic Analysis (Lab 12)
 - Topic 13: UDP Traffic Analysis (Lab 13)
 - Topic 14: TCP Protocol Analysis (Lab 14)

Day 4

- Day 4 — Graphs, applications and reporting
 - Topic 15: Traffic Graphs (Lab 15)
 - Topic 16: HTTP and HTTP/2 Analysis (Lab 16)
 - Topic 17: TLS-Encrypted Traffic Analysis (Lab 17)
 - Topic 18: Ten Troubleshooting Steps and Reporting (Lab 18)
- Daily timing
 - 9:30am–6:30pm · 1-hour lunch · tea breaks within training time

Learning Outcomes

1

Capture & profile

Plan scoped captures and configure a reproducible analyst profile.

2

Navigate & time

Navigate, filter, summarize and time network exchanges.

3

Core protocols

Interpret DNS, ARP, IPv4, ICMP and UDP evidence.

4

TCP & applications

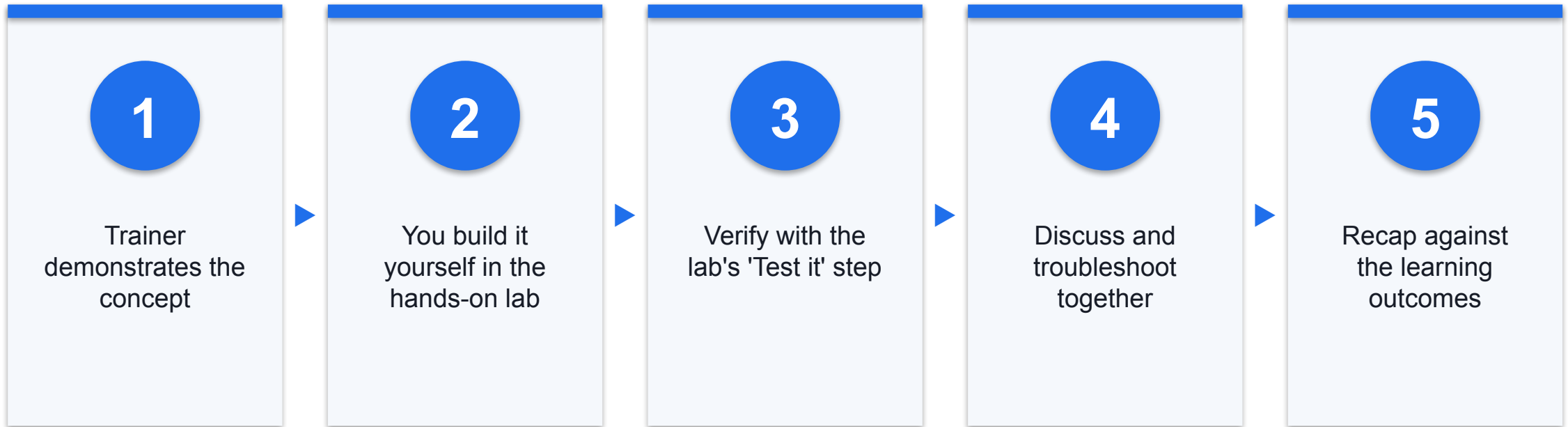
Diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

5

Evidence reporting

Report observed facts, hypotheses and next actions with defensible evidence.

How You'll Learn





CORE CONCEPTS

Network Analysis Fundamentals

What is Wireshark?

1

Packet analyzer

Captures packets and presents every field it can decode — a measuring device for the network.

2

Free and open source

Cross-platform (Windows, macOS, Linux) with TShark as its command-line twin.

3

3,000+ protocols

Dissectors decode Ethernet to HTTP/2, DNS, TLS, SIP/RTP and more.

4

Evidence, not opinion

Every claim cites a frame number, a filter and a measurement.

Who Uses Wireshark — and Why

1

Network administrators

Troubleshoot slow and failing connections.

2

Security analysts

Examine suspicious conversations and indicators.

3

QA and developers

Verify and debug protocol implementations.

4

Support engineers

Prove where a delay or failure actually occurs.

5

Learners

See how network protocols really behave on the wire.

6

Incident responders

Preserve packet evidence for a report.

THE ANALYST MINDSET

Capture the evidence. Explain the uncertainty.

A trace is a measurement at one observation point — this course teaches you to say exactly what it proves and what it does not.

The Four Days at a Glance

Capture & navigate

- Place the capture point correctly
- Build a reproducible analyst profile
- Colour, time and filter efficiently

Protocols & transport

- DNS, ARP, IPv4, ICMP, UDP evidence
- TCP handshake, loss, retransmission
- Zero window and recovery

Applications & reporting

- I/O, RTT and Stevens graphs
- HTTP outcomes, HTTP/2 and TLS
- An evidence-led incident report

How Every Lab Progresses

1

Open the lab's own synthetic capture — no live network traffic is needed.

2

Apply the lab filter and locate the frames that answer the question.

3

Record frame numbers, measurements and one limitation in outputs/findings.md.

4

Run scripts/verify.py — it confirms the capture facts your findings rely on.

TOPIC 01

Introduction to Network Analysis and Wireshark

Establish a trace baseline

KEY CONCEPTS

Key Concepts — Introduction to Network Analysis and Wireshark

1

A capture is a measurement at one observation point, not a complete network history.

2

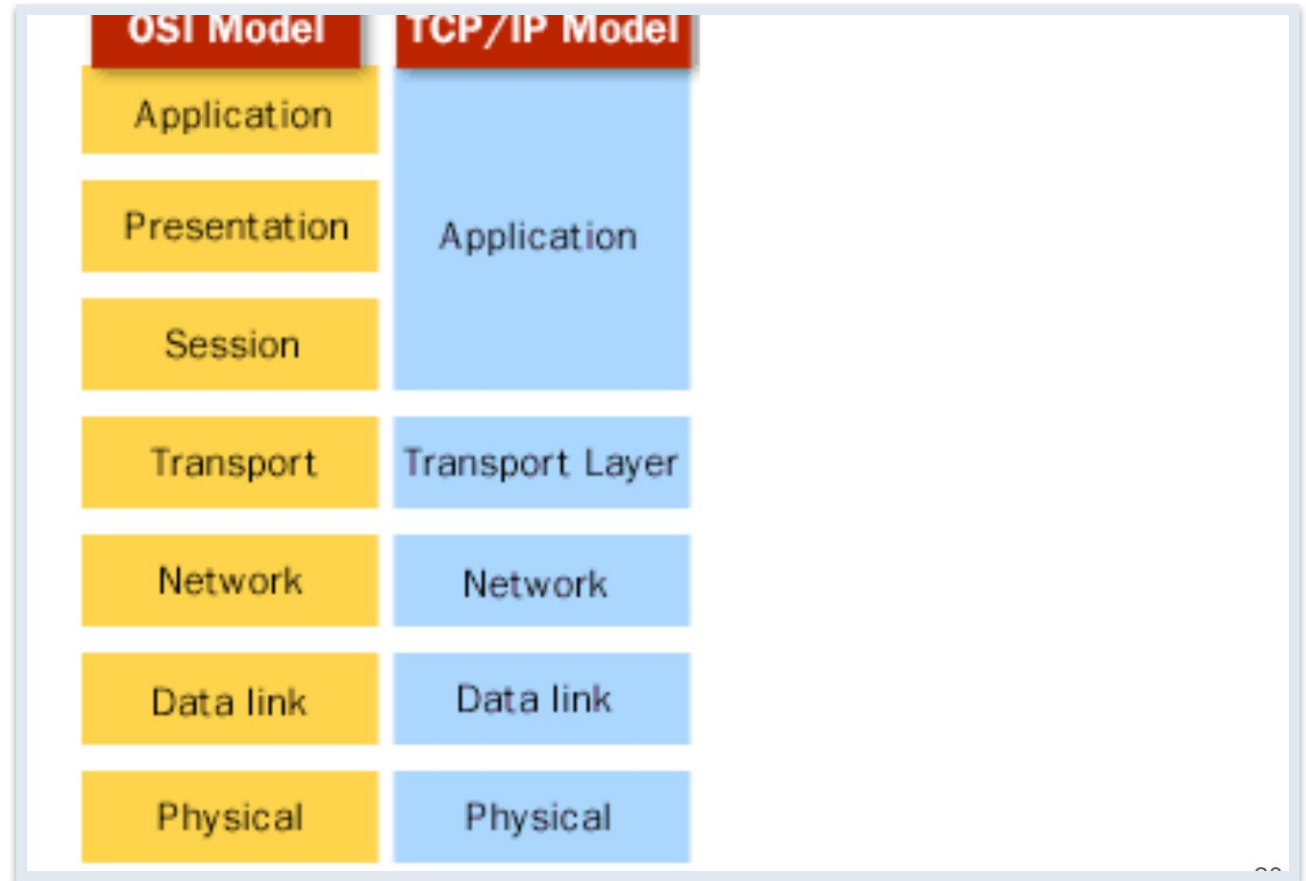
Packet list, protocol details and bytes connect a summary to the underlying evidence.

3

Record interface, timestamp precision, dropped packets and capture scope before interpreting a trace.

The TCP/IP and OSI Models

- The OSI model has seven layers; TCP/IP folds them into four.
- Wireshark's Packet Details pane is ordered the same way — frame, Ethernet, IP, TCP/UDP, application.
- Locate each fault at a layer before you choose a filter.



OSI layers mapped to the TCP/IP model

Anatomy of a Packet

- Every frame is nested headers: Ethernet II (14 B) › IPv4 (20 B) › TCP (20 B) › application payload.
- Packet Details lists them in the same top-to-bottom order.
- Clicking a field highlights its exact bytes in the Packet Bytes pane.

Frame 21 in branch-office.pcap — 124 bytes on the wire



Layer 2 · MAC addresses change per hop

Layer 3 · IP addresses end-to-end

Layer 4 · ports identify the conversation

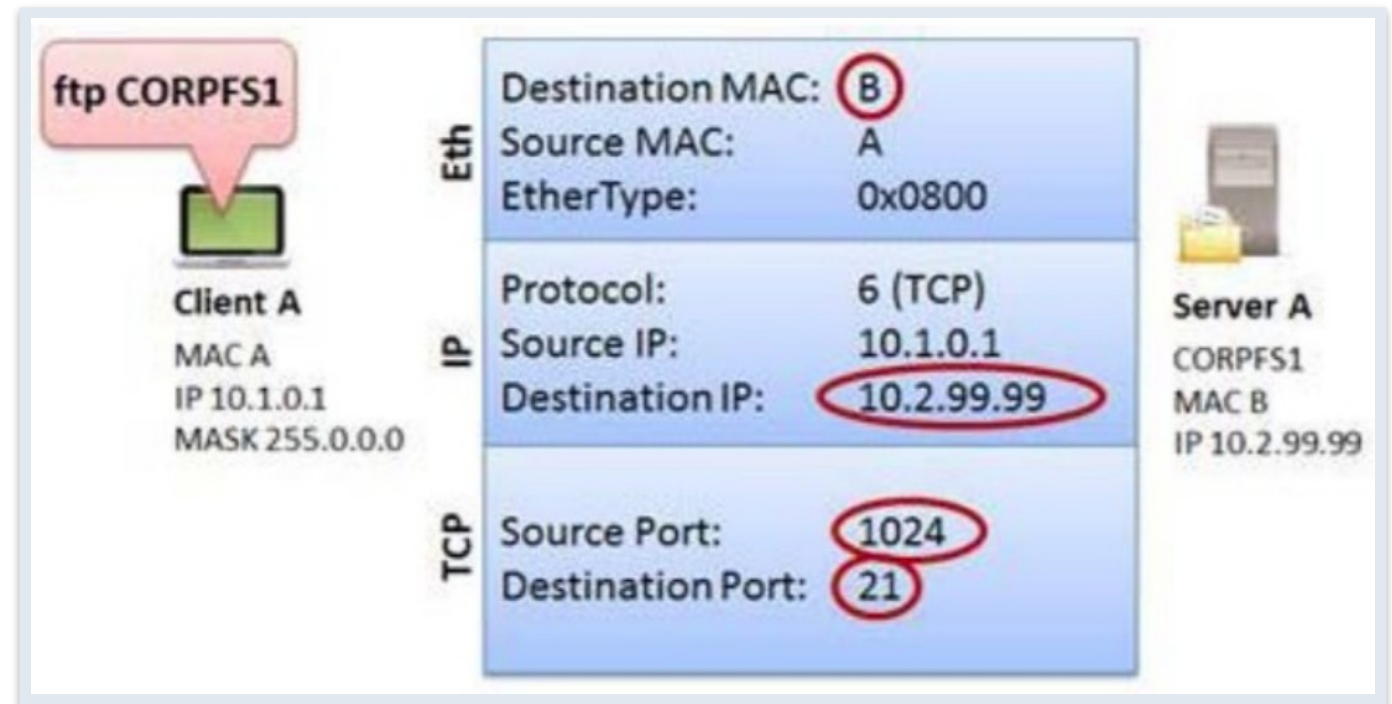
Layer 7 · the application message

Packet Details shows the same order top-to-bottom: Frame › Ethernet II › Internet Protocol › TCP › HTTP

Frame 21 of the lab capture: GET /health, 124 bytes

Building a Packet

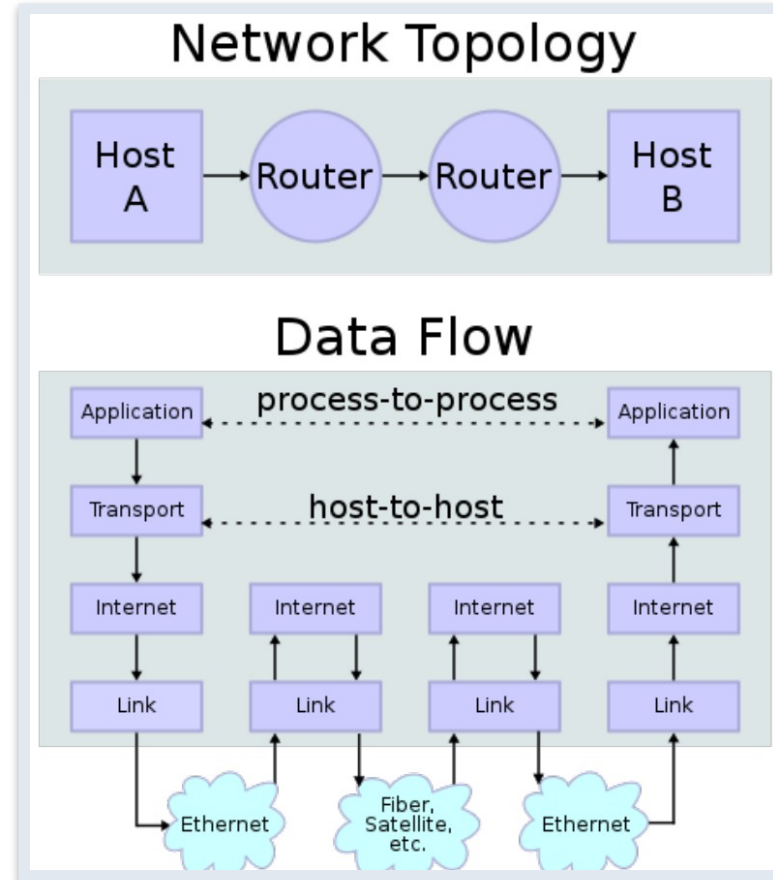
- Each layer wraps the one above: an application payload gains TCP, IP and Ethernet headers.
- MAC addresses change at every hop; IP addresses stay end-to-end (unless NAT rewrites them).
- Ports identify the conversation on each host.



Encapsulation of an FTP request from client A to server A

Data Flow Through the Network

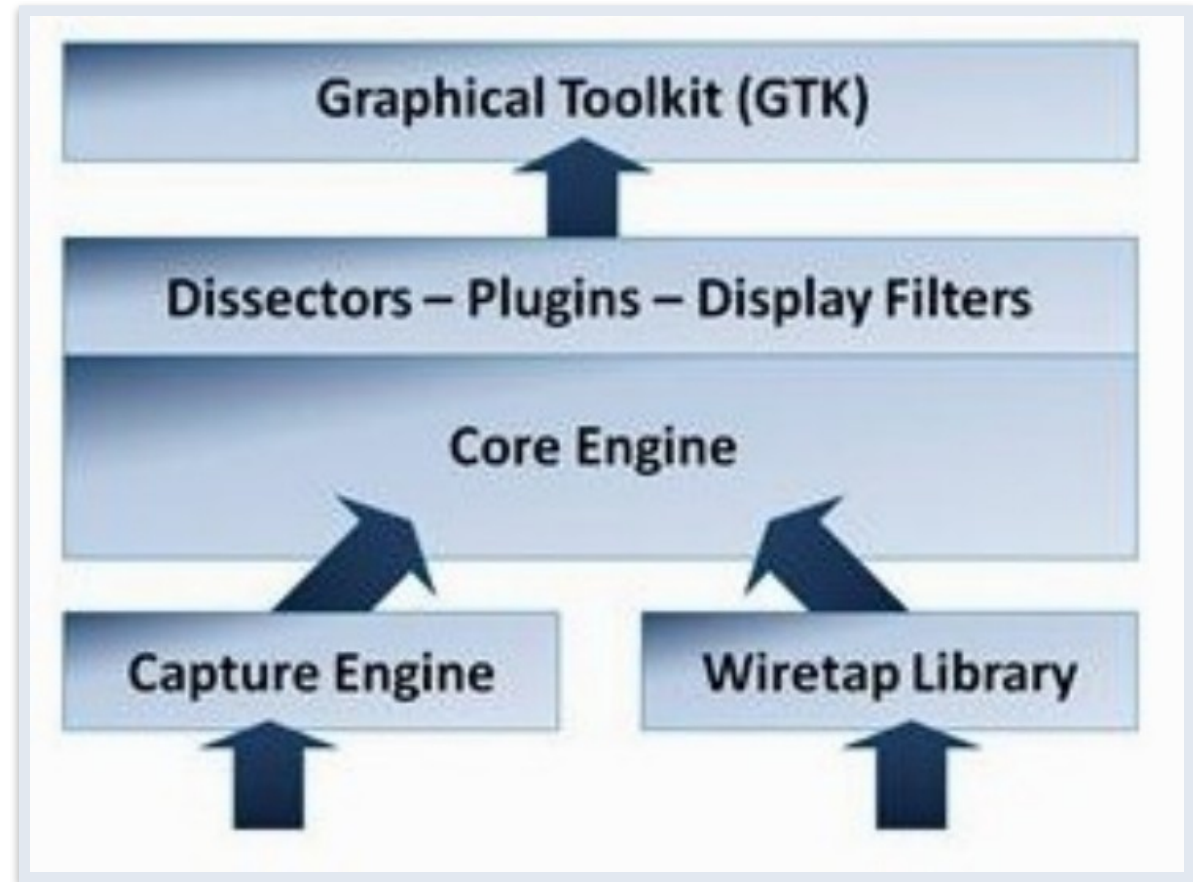
- Data passes down the sender's stack, across switches and routers, and up the receiver's stack.
- Switches forward on MAC; routers forward on IP.
- Where you capture decides which hops you can see.



Process-to-process, host-to-host and hop-by-hop delivery

Inside Wireshark

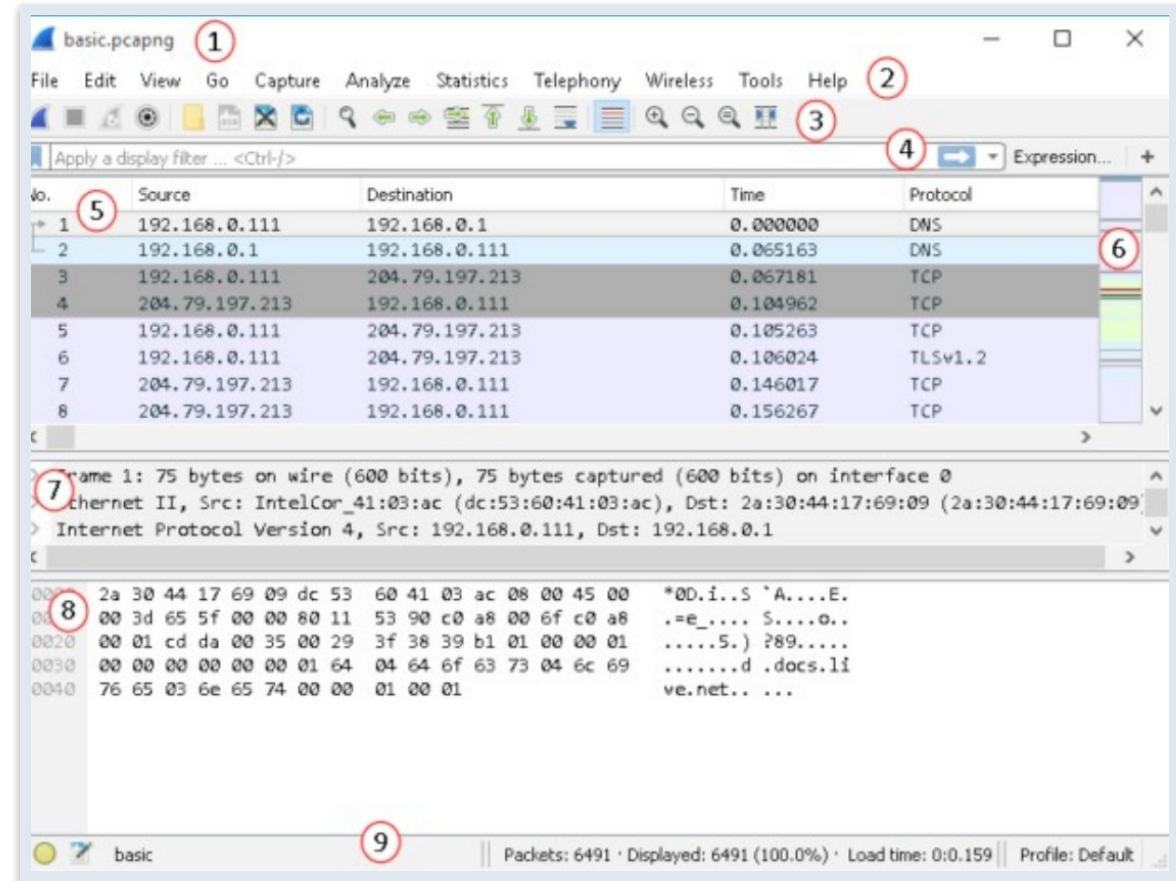
- The capture engine collects packets through libpcap / Npcap.
- Wiretap reads and writes trace formats: pcap, pcapng, snoop and more.
- Dissectors decode each protocol; display filters operate on the dissected fields.



Packet processing elements (the GUI is Qt-based in current releases)

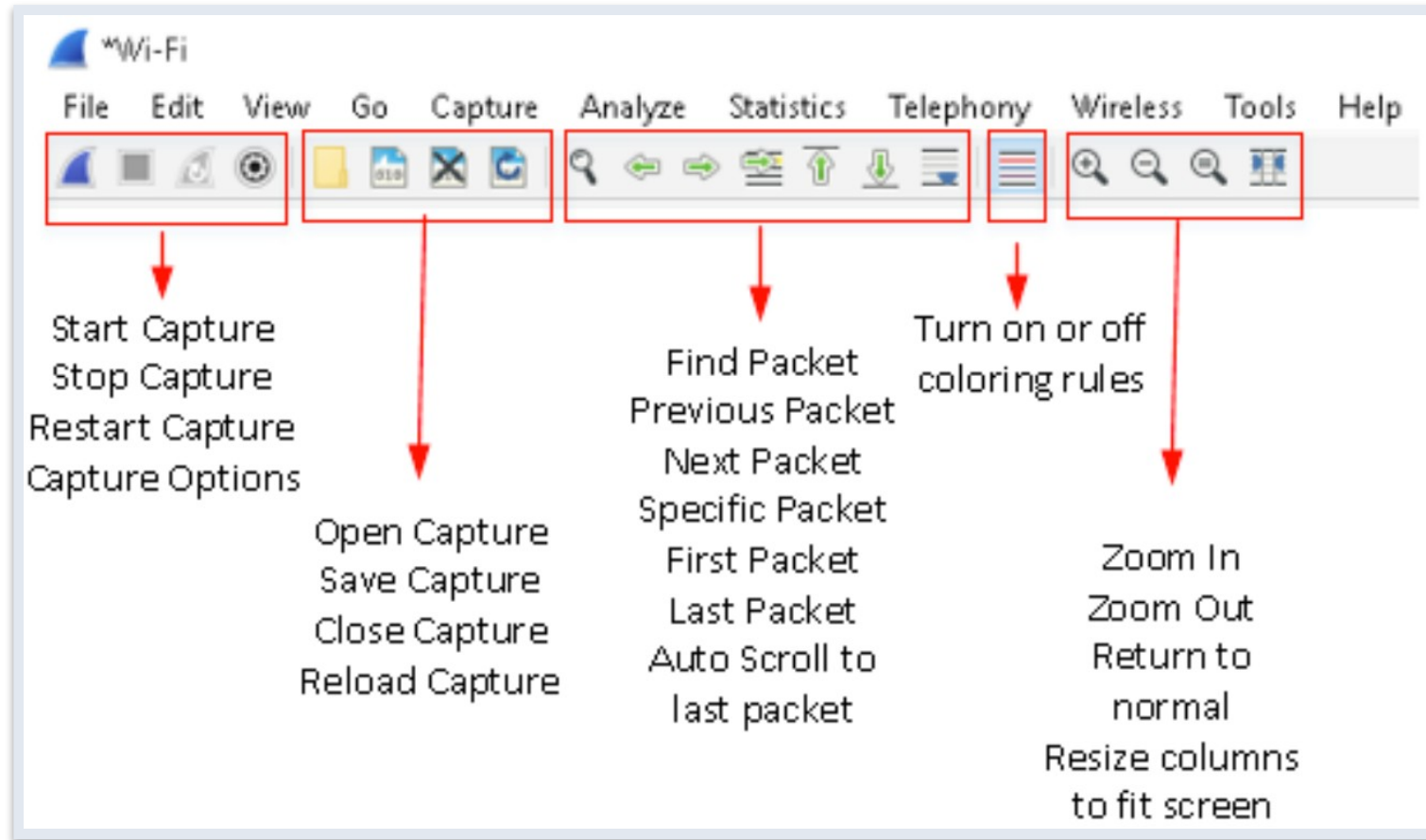
The Wireshark Window

- Packet List — one summary row per frame.
- Packet Details — the decoded protocol tree.
- Packet Bytes — the raw hex for the selected field.
- Status bar — packet counts, profile and file information.



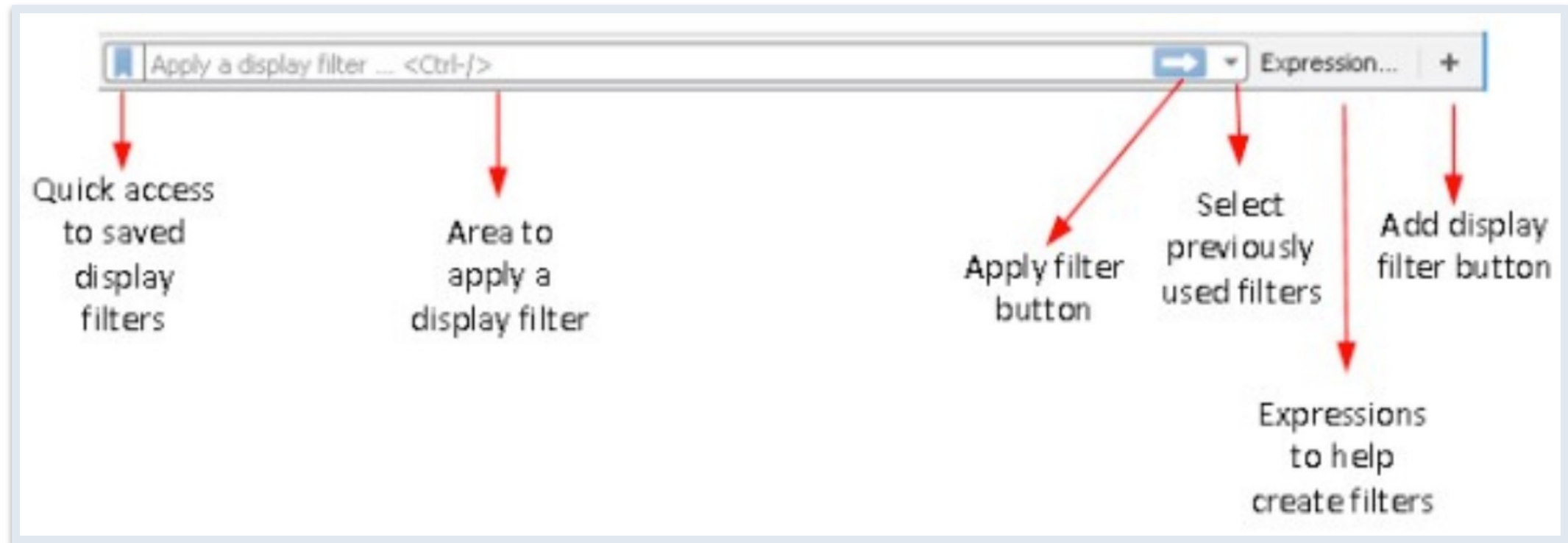
The three main panes plus toolbars and status bar

The Main Toolbar



Start/stop capture, open/save, find and go-to packet, colourise and zoom controls

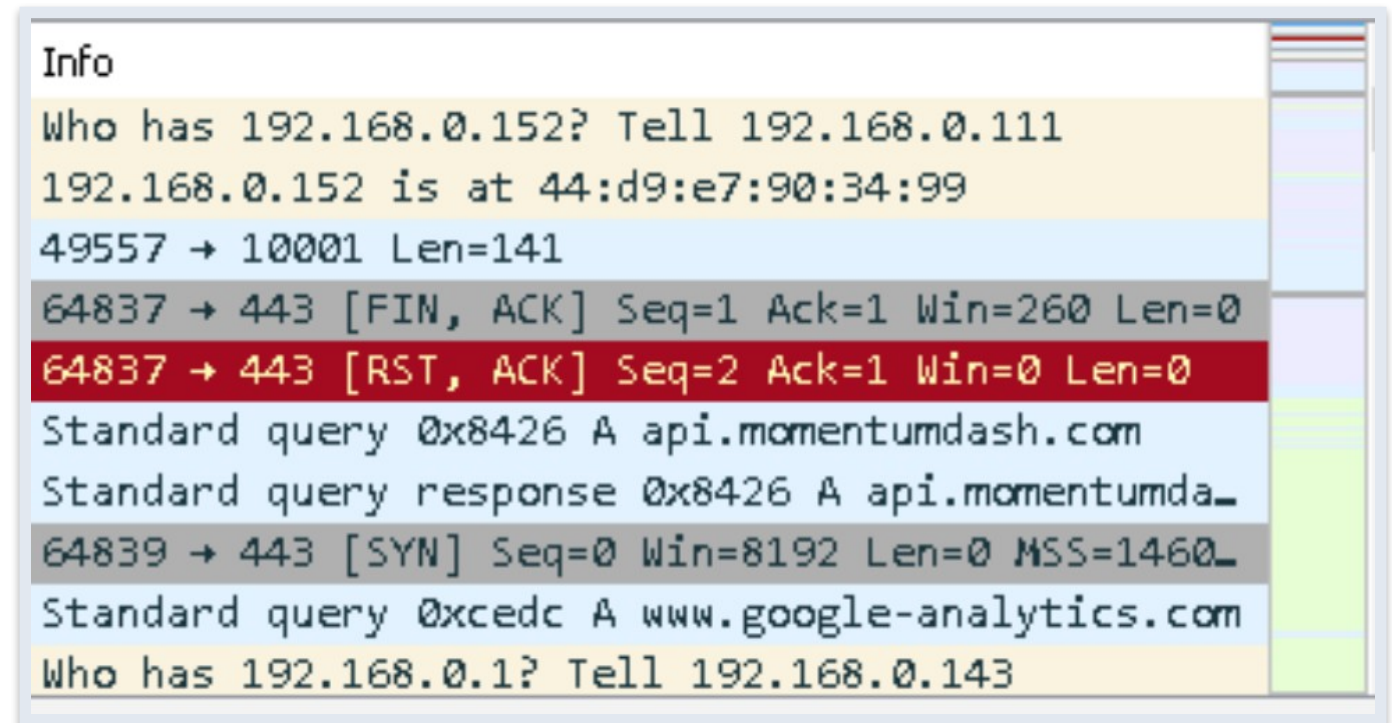
The Display Filter Toolbar



Green = valid expression, red = invalid; bookmark saved filters and add filter buttons

Status Bar and Intelligent Scrollbar

- The status bar shows packets captured vs. displayed and the active profile.
- The intelligent scrollbar mirrors packet colouring — red/black bands reveal resets, aborts or low-TTL packets at a glance.



Coloured scrollbar bands mark problem packets

Capture Your Own Baseline

1

Pick the busy interface — the sparkline beside each interface shows live traffic.

2

Capture 30–60 s of normal activity on a network you are authorised to monitor, then stop.

3

Save as pcapng so comments and interface details are kept.

4

Record interface, start time, duration and dropped packets (Statistics > Capture File Properties).

TShark — Wireshark on the Command Line

1

tshark -D

List capture interfaces

2

**tshark -i 1 -a duration:60 -w
base.pcapng**

Timed capture to a file

3

tshark -r base.pcapng -Y "dns"

Read a file with a display filter

4

-T fields -e ip.src -e dns.qry.name

Print chosen fields only

5

-q -z io,phs

Protocol hierarchy statistics

6

-q -z conv,tcp

TCP conversation table

Establish a trace baseline

LAB 1

The branch help desk has received a capture from the client PC (192.0.2.10), but nobody has checked what it contains. Before anyone diagnoses a fault, you establish a trusted baseline of the trace.

You'll build

A baseline record of the capture and the ARP exchange

Tools: Wireshark, TShark, Python

Lab 1 — What You'll Do

1

Verify the lab fixture with
verify.py

2

Record packet count
and capture duration

3

Trace a field from
Packet Details to bytes

4

Pair the ARP request
and reply

Full step-by-step instructions: [Learner Guide · labs/lab-01-establish-a-trace-baseline/LAB-01-Instructions.pdf](#)

Expected Evidence — Lab 1

arp					
No.	Time	Source	Destination	Protocol	Info
1	0.000000	02:00:00:00:00:10	ff:ff:ff:ff:ff:ff	ARP	Who has 192.0.2.20? Tell 192.0.2.10
2	0.010000	02:00:00:00:00:20	02:00:00:00:00:10	ARP	192.0.2.20 is at 02:00:00:00:00:20
branch-office.pcap Displayed: 2					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Establish a trace baseline

Test it

The supplied arp expression matches 2 frame(s) in the specified capture. Run scripts/verify.py and compare the listed frame numbers. Keep your findings and exported table in outputs/. Explain the observed result rather than only copying a count.

TOPIC RECAP

Recap — Introduction to Network Analysis and Wireshark

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

A capture is a measurement at one observation point, not a complete network history.

3

Packet list, protocol details and bytes connect a summary to the underlying evidence.

4

Record interface, timestamp precision, dropped packets and capture scope before interpreting a trace.

Tea Break

15 minutes

TOPIC 02

Capture Methods and Capture Filters

Plan capture placement

KEY CONCEPTS

Key Concepts — Capture Methods and Capture Filters

1

A switched access port normally observes its own traffic and broadcasts; SPAN or TAP placement changes visibility.

2

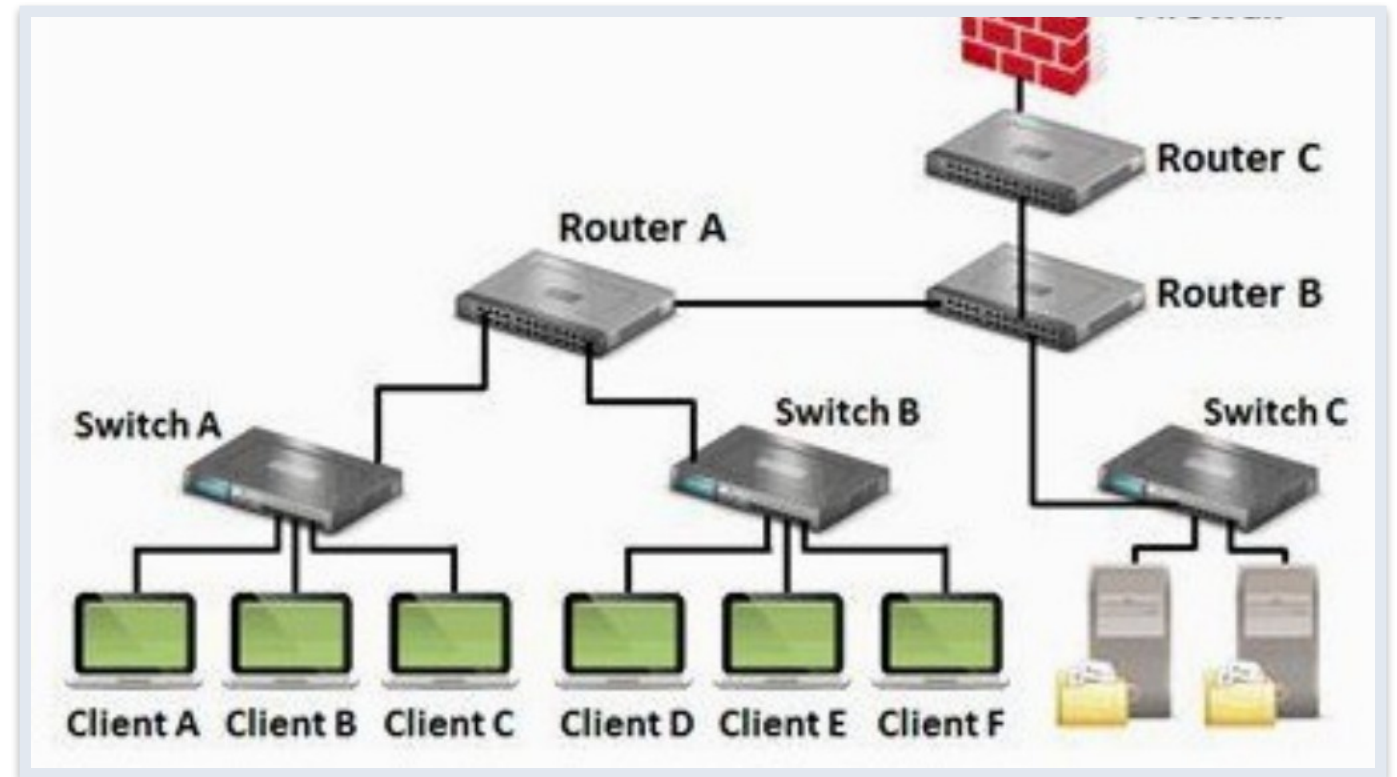
Capture filters use libpcap syntax before packets are stored; display filters hide or show stored packets.

3

Bound file size and duration; check capture drops before attributing missing packets to network loss.

Where to Tap Into the Network

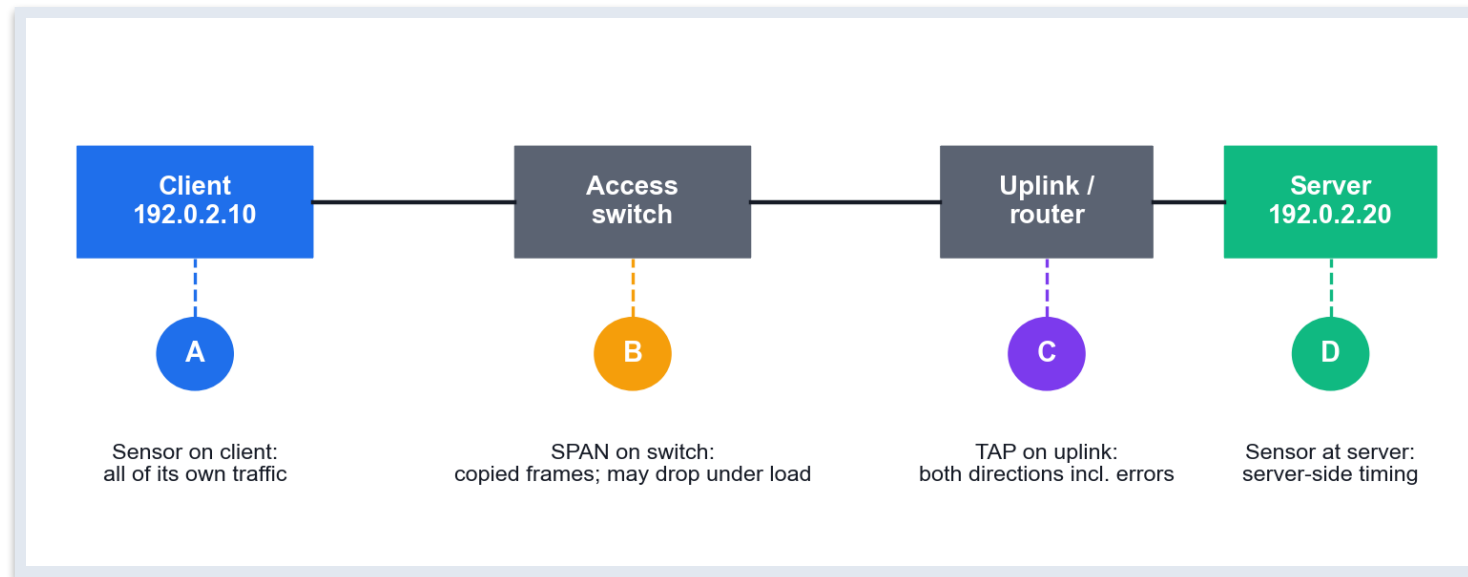
- Client A complains about reaching the servers — where do you put the analyzer?
- Capture as close as possible to the complaining host first, then move toward the servers.
- Each point sees only the traffic that physically passes it.



Choose the observation point before you capture

Choosing the Observation Point

- A — on the client: sees everything the client sends and receives.
- B — SPAN on the switch: copies frames, may drop under load and hides errored frames.
- C — TAP on the uplink: both directions, including physical-layer errors.
- D — at the server: shows server-side timing.



The same problem looks different from each capture point

What a Switch Port Can See

Switch port (default)

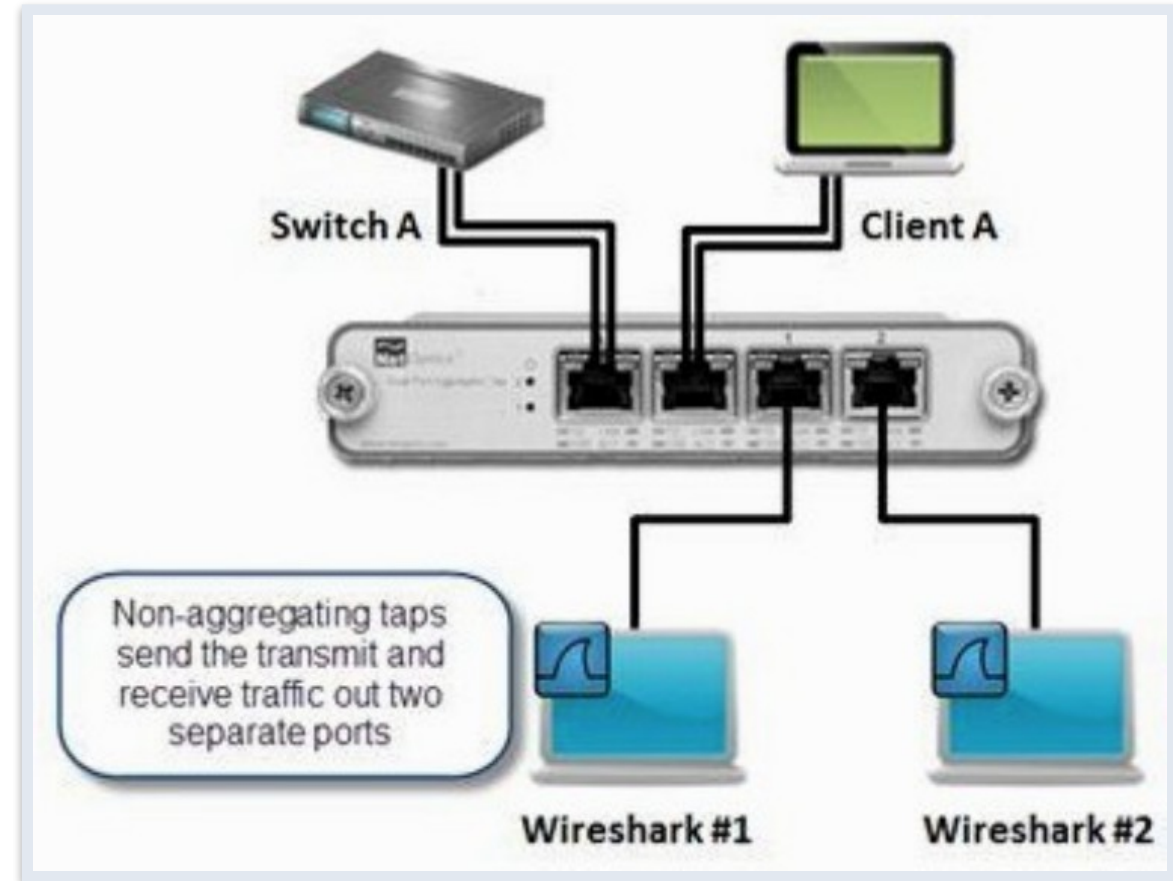
- Broadcast traffic
- Multicast traffic (if forwarded)
- Traffic to and from your own host
- Not other hosts' unicast traffic

Ways to see more

- Hub in half-duplex segment (lab only)
- Network TAP — passive, in-line
- SPAN / port mirroring on the switch
- Capture on the target host itself

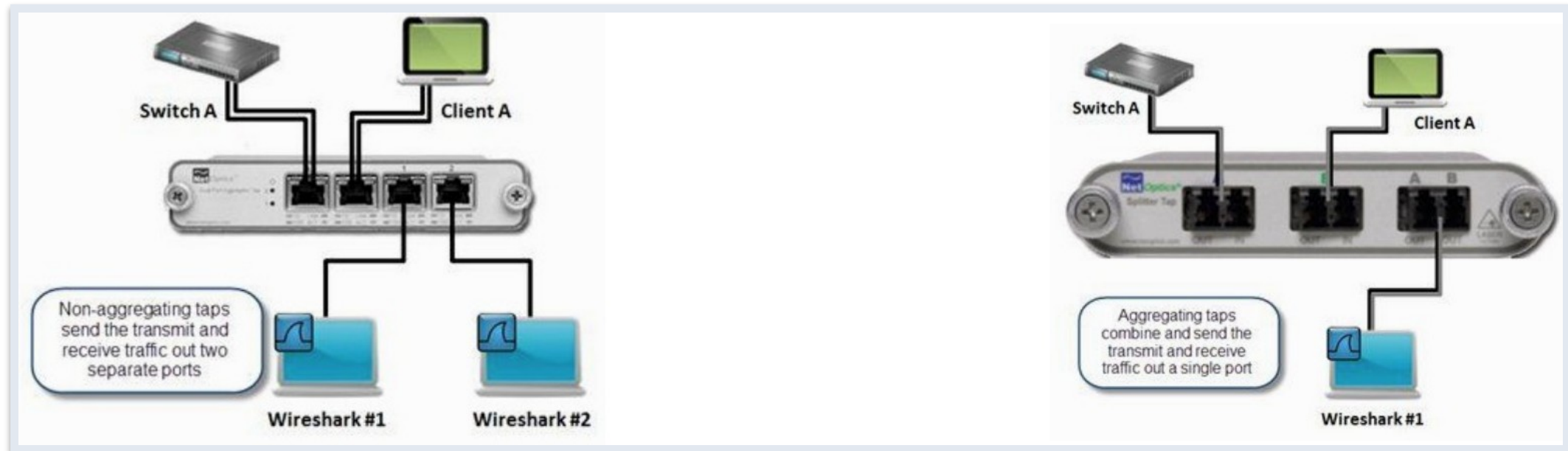
Test Access Points (TAPs)

- A TAP sits in-line and passively copies traffic to the analyzer.
- Full-duplex TAPs forward both directions — and can pass physical-layer errors.
- Aggregating TAPs merge both directions onto one monitor port.



A non-aggregating TAP with two monitor ports

Aggregating vs Non-aggregating TAPs

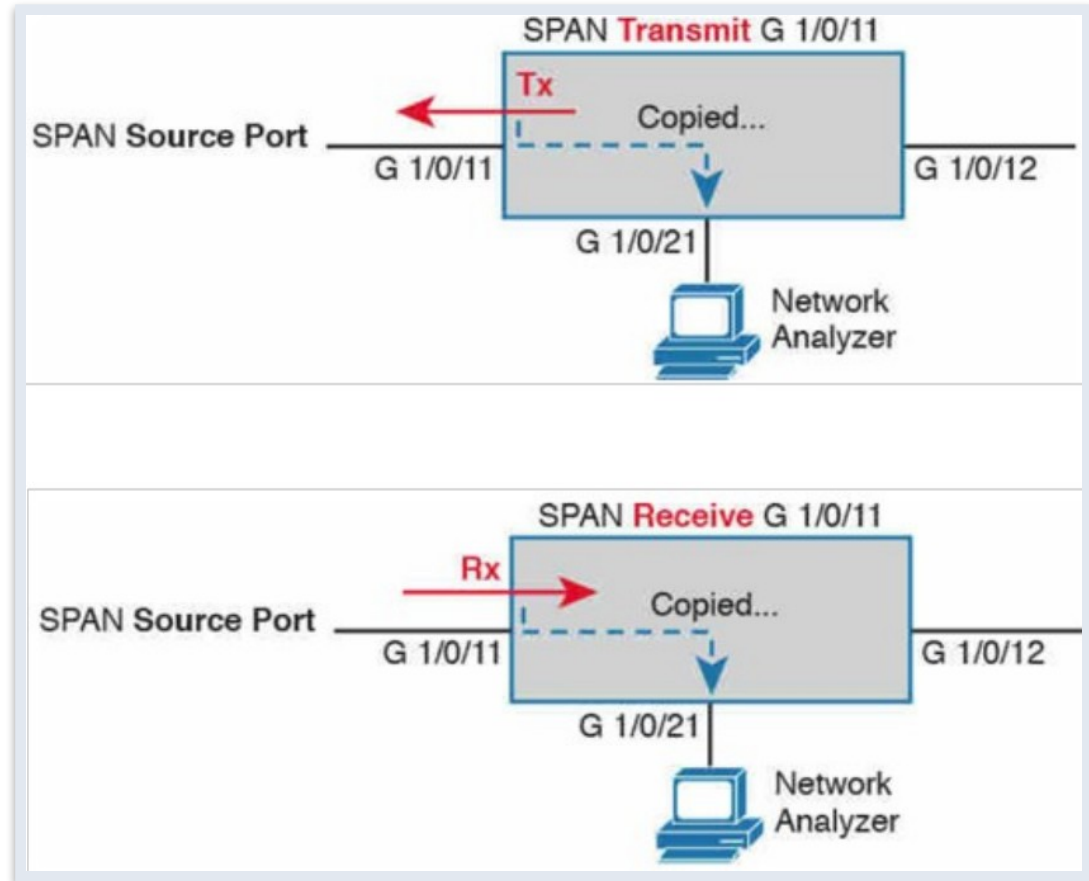


Non-aggregating: one port per direction. Aggregating: both directions combined on one port — can oversubscribe at high load

SPAN

SPAN / Port Mirroring

- The switch copies traffic from source ports or VLANs to a destination port.
- Mirrored traffic can drop under load and normally excludes errored frames.
- Example (Cisco): monitor session 1 source interface g1/0/11 both.



SPAN copies transmitted and received frames to the analyzer port

Wireless Capture

- Promiscuous mode on Wi-Fi usually shows only your own traffic.
- Monitor mode captures 802.11 frames from all stations on a channel — needs a supporting adapter/driver.
- Decrypting WPA2 traffic requires the passphrase and the 4-way handshake.



Wireless capture depends on the adapter mode

Capture Filters vs Display Filters

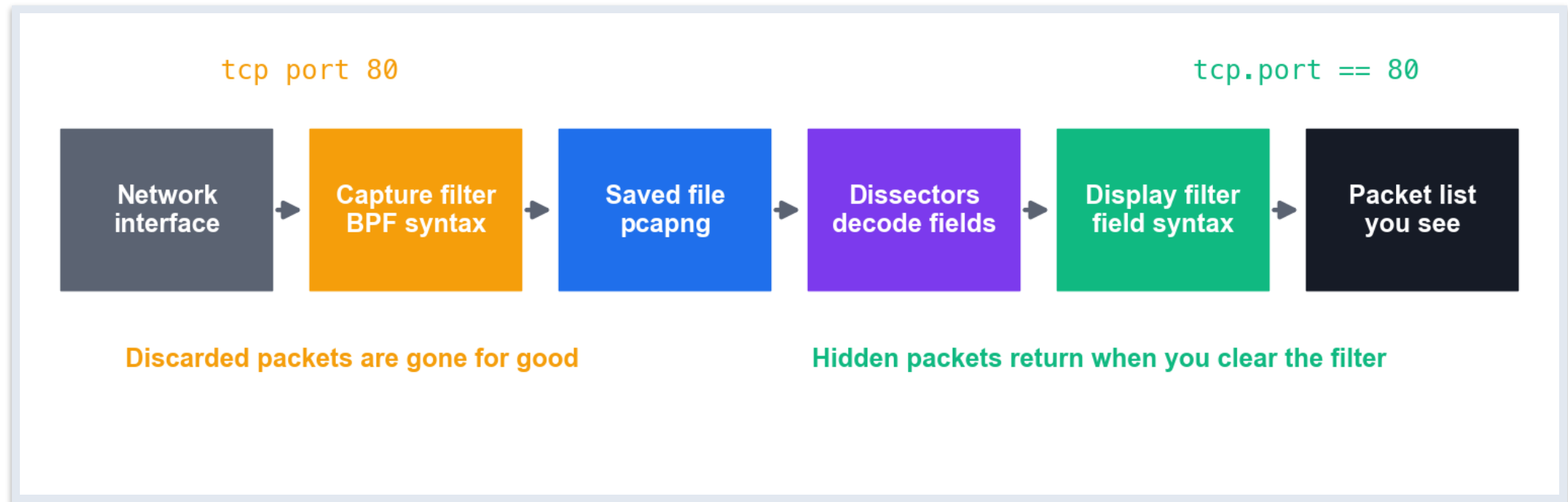
Capture filter (BPF)

- Applied before packets are stored
- Syntax: host 10.1.1.5 and tcp port 80
- Discarded packets cannot be recovered
- Use to limit file size on busy links

Display filter

- Applied to packets already captured
- Syntax: ip.addr == 10.1.1.5 && tcp.port == 80
- Clear it to see everything again
- Use for analysis and evidence

The Filtering Pipeline



Capture filters (BPF) decide what is stored; display filters only decide what is shown

Capture Filter (BPF) Cheat Sheet

1

host 192.0.2.10

To or from one host

2

net 192.0.2.0/24

One subnet

3

tcp port 80

One TCP port

4

port 53

DNS over UDP or TCP

5

not arp and not stp

Drop background noise

6

tcp[tcpflags] & tcp-syn != 0

TCP SYN packets only

Capture Options for Long Captures

1

Ring buffer: e.g. 10 files × 100 MB keeps the most recent traffic without filling the disk.

2

Auto-stop after a duration, file size or packet count.

3

Snapshot length can limit stored bytes per packet for privacy and size.

4

Always check “dropped” counts before trusting a gap.

Plan capture placement

LAB 2

A user on the branch LAN reports slow web pages. Your manager asks where a sensor should go — and what it would see — before anyone touches the switch.

You'll build

A completed capture plan with sensor placement and limits

Tools: Wireshark, TShark, Python

Lab 2 — What You'll Do

1

Mark three sensor
points on the topology

2

Check capture-filter
syntax (tcp port 80)

3

Compare capture vs
display filtering

4

Complete the capture
plan and permissions

Full step-by-step instructions: [Learner Guide · labs/lab-02-plan-capture-placement/LAB-02-Instructions.pdf](#)

Expected Evidence — Lab 2

tcp.port == 80					
No.	Time	Source	Destination	Protocol	Info
18	1.040000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 W...
19	1.070000	192.0.2.20	192.0.2.10	TCP	80 → 51001 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0...
20	1.080000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [ACK] Seq=1 Ack=1 Win=1048576 Len=0
21	1.090000	192.0.2.10	192.0.2.20	HTTP	GET /health HTTP/1.1
22	1.100000	192.0.2.20	192.0.2.10	TCP	80 → 51001 [ACK] Seq=1 Ack=71 Win=1048576 Len=0
23	1.140000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
24	1.150000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [ACK] Seq=71 Ack=71 Win=8222720 Len=0
25	1.160000	192.0.2.20	192.0.2.10	TCP	80 → 51001 [FIN, ACK] Seq=71 Ack=71 Win=1048576 L...
26	1.170000	192.0.2.10	192.0.2.20	TCP	51001 → 80 [ACK] Seq=71 Ack=72 Win=1048576 Len=0
27	1.180000	192.0.2.10	192.0.2.20	TCP	51002 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 W...
... 27 more matching packets					
branch-office.pcap Displayed: 37					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Plan capture placement

Test it

The supplied `tcp.port == 80` expression matches 37 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Capture Methods and Capture Filters

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

A switched access port normally observes its own traffic and broadcasts; SPAN or TAP placement changes visibility.

3

Capture filters use libpcap syntax before packets are stored; display filters hide or show stored packets.

4

Bound file size and duration; check capture drops before attributing missing packets to network loss.

Lunch Break

1 hour

TOPIC 03

Global Preferences and Troubleshooting Profiles

Create an analyst profile

KEY CONCEPTS

Key Concepts — Global Preferences and Troubleshooting Profiles

1

A profile bundles reproducible columns, coloring rules and protocol settings.

2

Name resolution can obscure numeric evidence and generate additional traffic; document the setting.

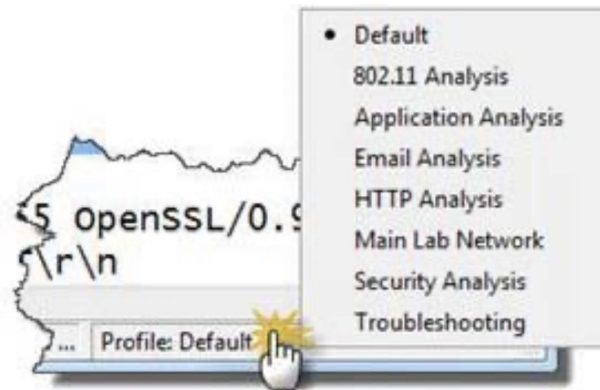
3

Protocol heuristics and TCP analysis preferences change interpretation, not the stored bytes.

Configuration Profiles

- A profile bundles preferences, columns, colouring rules and filter buttons.
- Switch profiles from the status bar or Edit > Configuration Profiles.
- Create task profiles: Troubleshooting, Security, VoIP, Wireless.

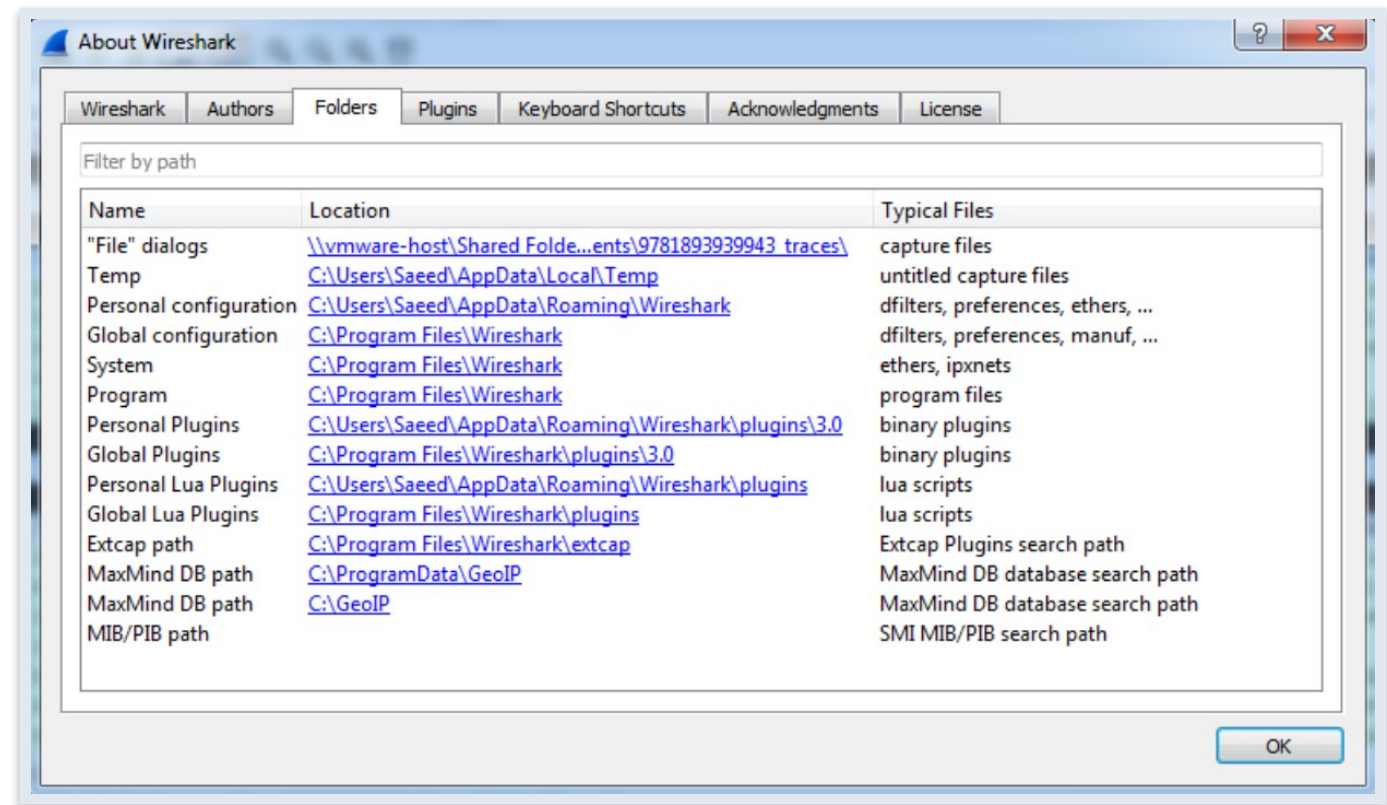
Right click on the Profile area in the Status bar or select **Edit | Configuration Profiles | New** to create a new profile.



Right-click the Profile area in the status bar to switch or create

Where Settings Live

- Help > About Wireshark > Folders shows personal and global configuration paths.
- Copy a profile folder to share it with your team.
- Keep a clean Default profile untouched.



The Folders tab lists personal configuration locations

Preferences That Change Interpretation

Name resolution

- Off by default for evidence
- MAC/transport names are local look-ups
- Network names cause extra DNS traffic

Columns

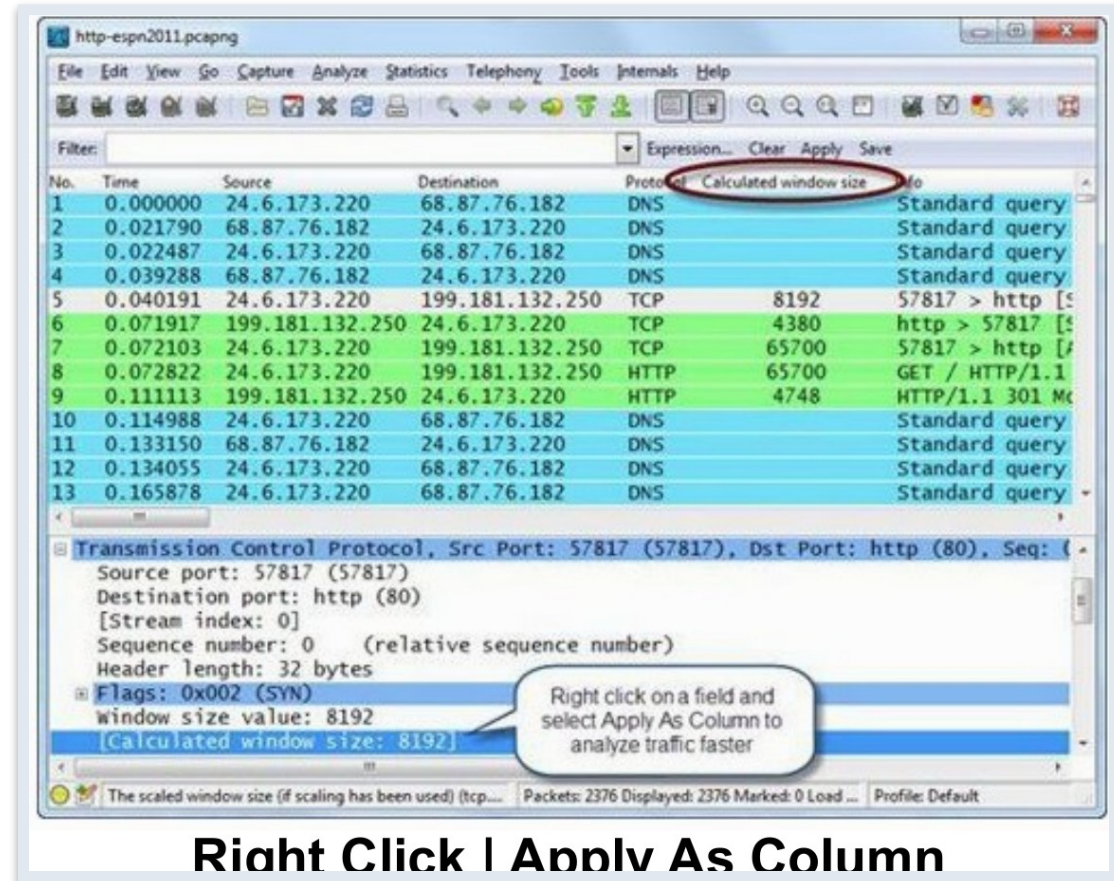
- Add fields as columns (Apply as Column)
- e.g. tcp.time_delta, http.host
- Save them in the profile

Protocol settings

- TCP: calculate conversation timestamps
- TCP: relative sequence numbers
- TLS: key log file per lab

Apply as Column

- Right-click any field in Packet Details > Apply as Column.
- Sort the new column to find extremes — largest delay, largest window, error codes.
- Columns are saved in the active profile.



Right Click | Apply As Column

Right-click | Apply As Column

Recommended Analyst Columns

1

frame.time_delta_displayed

Gap from the previous shown packet

2

tcp.stream

Which conversation

3

tcp.time_delta

Gap within the TCP stream

4

dns.time

DNS response time

5

http.time

HTTP response time

6

tcp.window_size

Receiver window (scaled)

Create an analyst profile

LAB 3

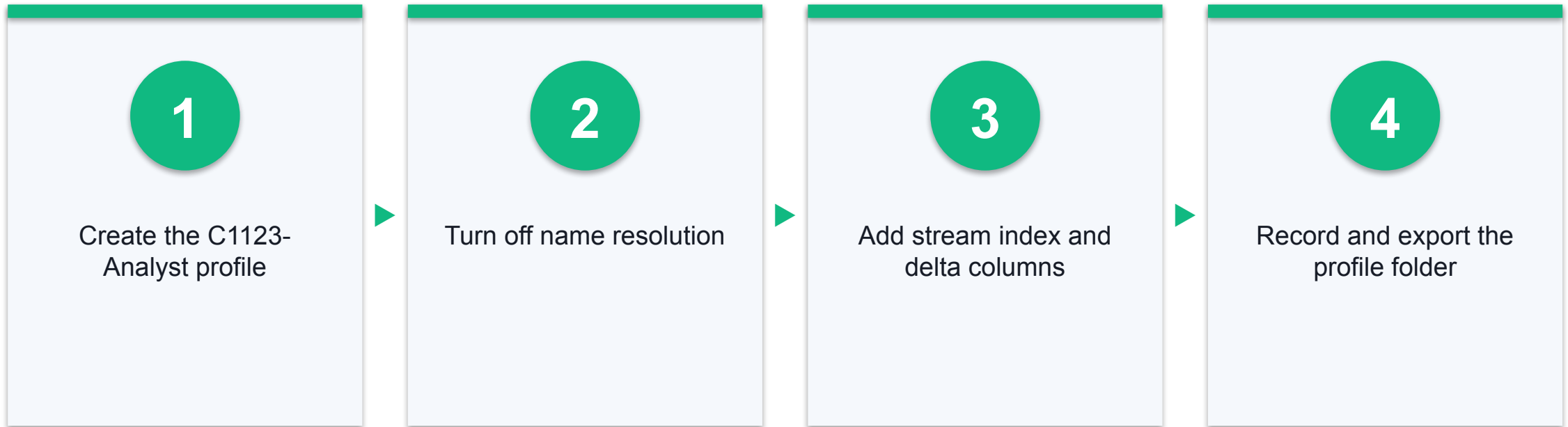
Three analysts will share findings on the same case. To make results reproducible, you build a named analyst profile with numeric addresses and evidence columns.

You'll build

A reusable C1123-Analyst profile and its folder path

Tools: Wireshark, TShark, Python

Lab 3 — What You'll Do



Full step-by-step instructions: [Learner Guide · labs/lab-03-create-an-analyst-profile/LAB-03-Instructions.pdf](#)

Expected Evidence — Lab 3

dns					
No.	Time	Source	Destination	Protocol	Info
3	0.020000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0065 A portal.example.test
4	0.040000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0065 A portal.example.t...
5	0.050000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0066 A missing.example.test
6	0.070000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0066 No such name A mis...
7	0.080000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0067 A slow.example.test
8	0.880000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0067 A slow.example.tes...
branch-office.pcap Displayed: 6					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Create an analyst profile

Test it

The supplied dns expression matches 6 frame(s) in the specified capture. Run scripts/verify.py and compare the listed frame numbers. Keep your findings and exported table in outputs/. Explain the observed result rather than only copying a count.

TOPIC RECAP

Recap — Global Preferences and Troubleshooting Profiles

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

A profile bundles reproducible columns, coloring rules and protocol settings.

3

Name resolution can obscure numeric evidence and generate additional traffic; document the setting.

4

Protocol heuristics and TCP analysis preferences change interpretation, not the stored bytes.

Tea Break

15 minutes

TOPIC 04

Navigation and Coloring Techniques

Color and annotate evidence

Key Concepts — Navigation and Coloring Techniques

1

Color rules are applied in order; the first matching rule wins.

2

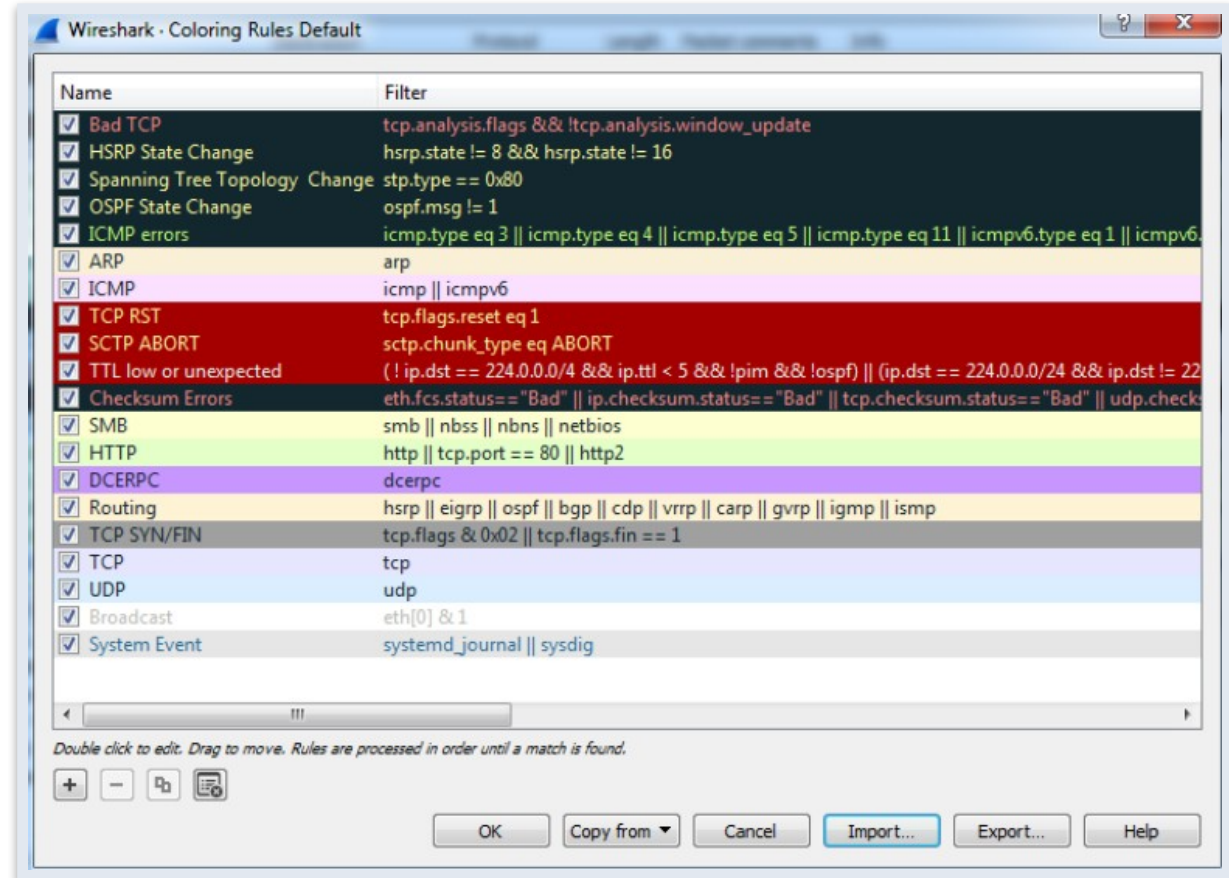
Temporary coloring is useful for a conversation; permanent rules support repeated triage.

3

Bookmarks and comments preserve the analyst path without changing the captured payload.

Colouring Rules

- Rules are evaluated top-down — the first match colours the packet.
- Default rules flag Bad TCP, HTTP, DNS, ARP, ICMP errors and more.
- View > Coloring Rules to edit, import or export a rule set.



The coloring rules list is processed in order

Default Colours — What They Mean

1

Black + red text

Bad TCP: retransmission, zero window, RST

2

Light green

HTTP

3

Light purple

TCP

4

Light blue

UDP (DNS, SIP, RTP ...)

5

Pale yellow

ARP and routing

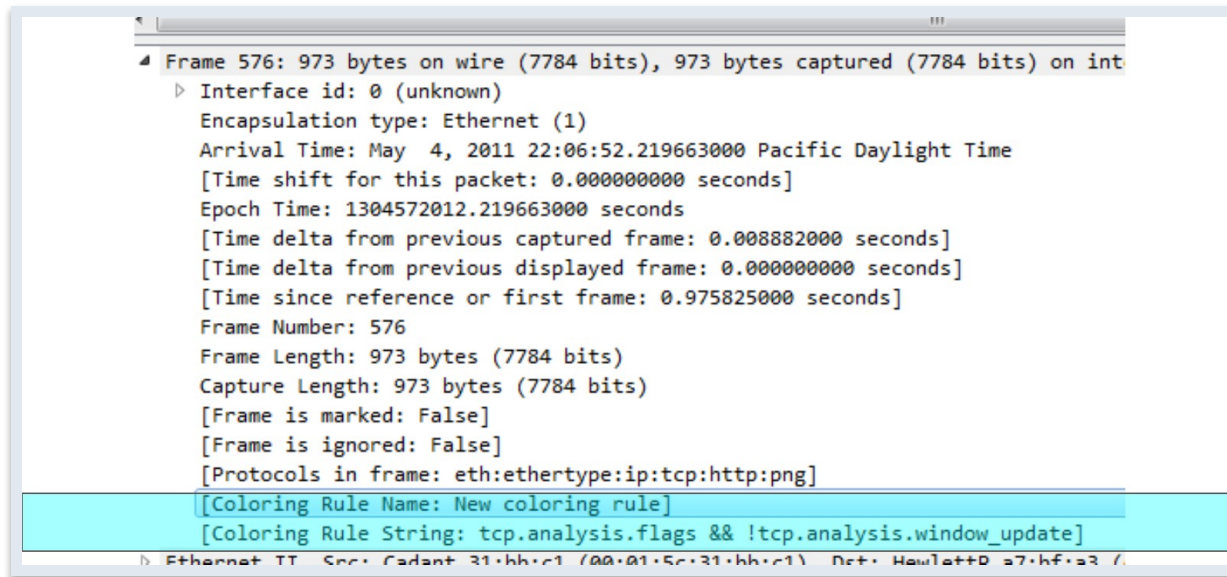
6

Black + green text

ICMP errors

Why Is a Packet This Colour?

- Expand the Frame section of Packet Details.
- Coloring Rule Name and Coloring Rule String show which rule matched.
- Use this to explain colours in a report.

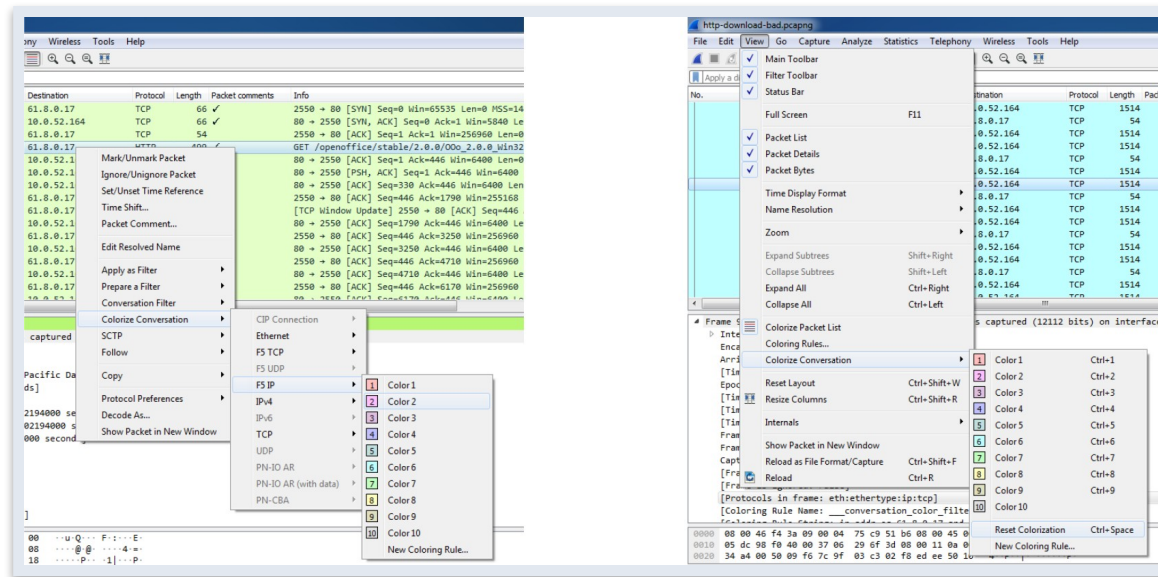


```
Frame 576: 973 bytes on wire (7784 bits), 973 bytes captured (7784 bits) on int
  Interface id: 0 (unknown)
  Encapsulation type: Ethernet (1)
  Arrival Time: May  4, 2011 22:06:52.219663000 Pacific Daylight Time
  [Time shift for this packet: 0.000000000 seconds]
  Epoch Time: 1304572012.219663000 seconds
  [Time delta from previous captured frame: 0.008882000 seconds]
  [Time delta from previous displayed frame: 0.000000000 seconds]
  [Time since reference or first frame: 0.975825000 seconds]
  Frame Number: 576
  Frame Length: 973 bytes (7784 bits)
  Capture Length: 973 bytes (7784 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:tcp:http:png]
  [Coloring Rule Name: New coloring rule]
  [Coloring Rule String: tcp.analysis.flags && !tcp.analysis.window_update]
  Ethernet II, Src: Cadant 31:bb:c1:00:01:5c:31:bb:c1, Dst: HewlettP a7:bf:a3 (/
```

Frame > Coloring Rule Name / String

Colourise a Conversation

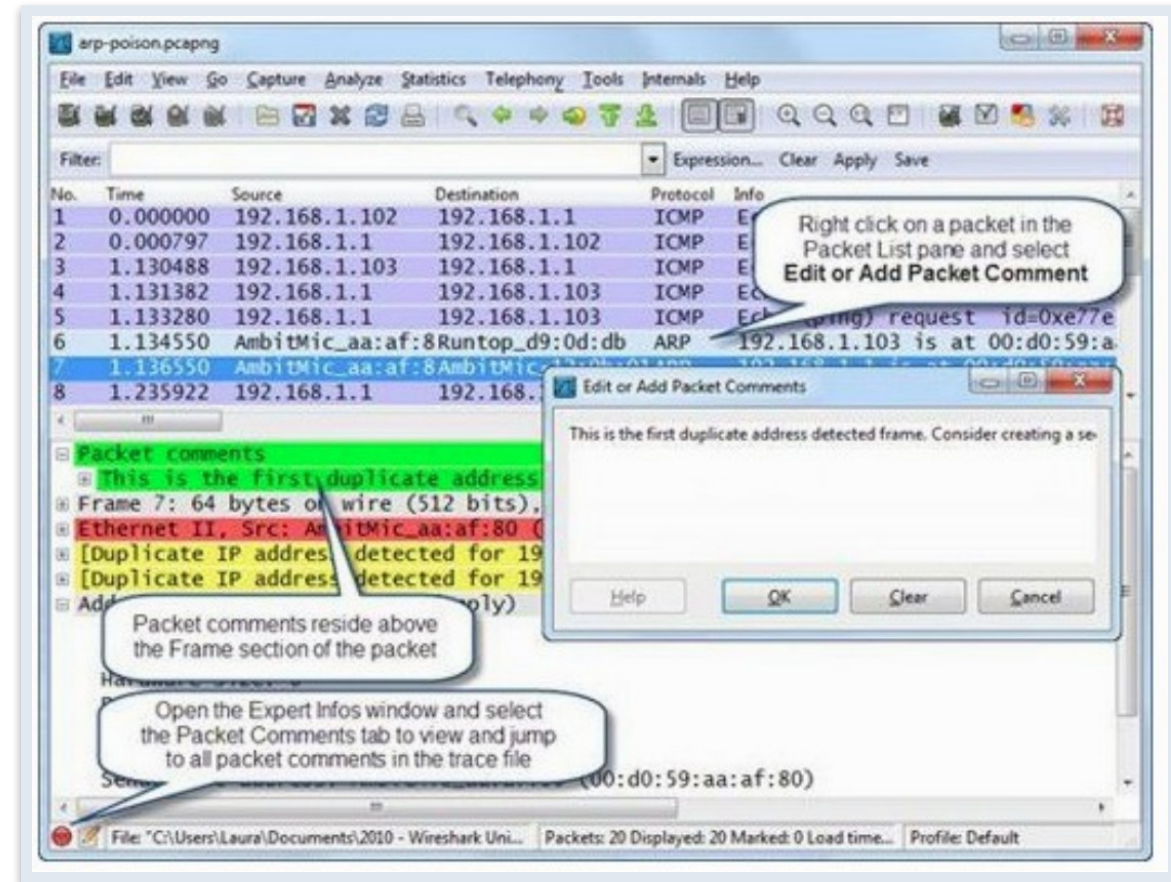
- Right-click a packet > Colorize Conversation to tag a TCP/UDP stream temporarily.
- Ten temporary colours make parallel conversations easy to tell apart.
- View > Colorize Conversation > Reset to clear.



Temporary conversation colouring

Marking and Annotating Packets

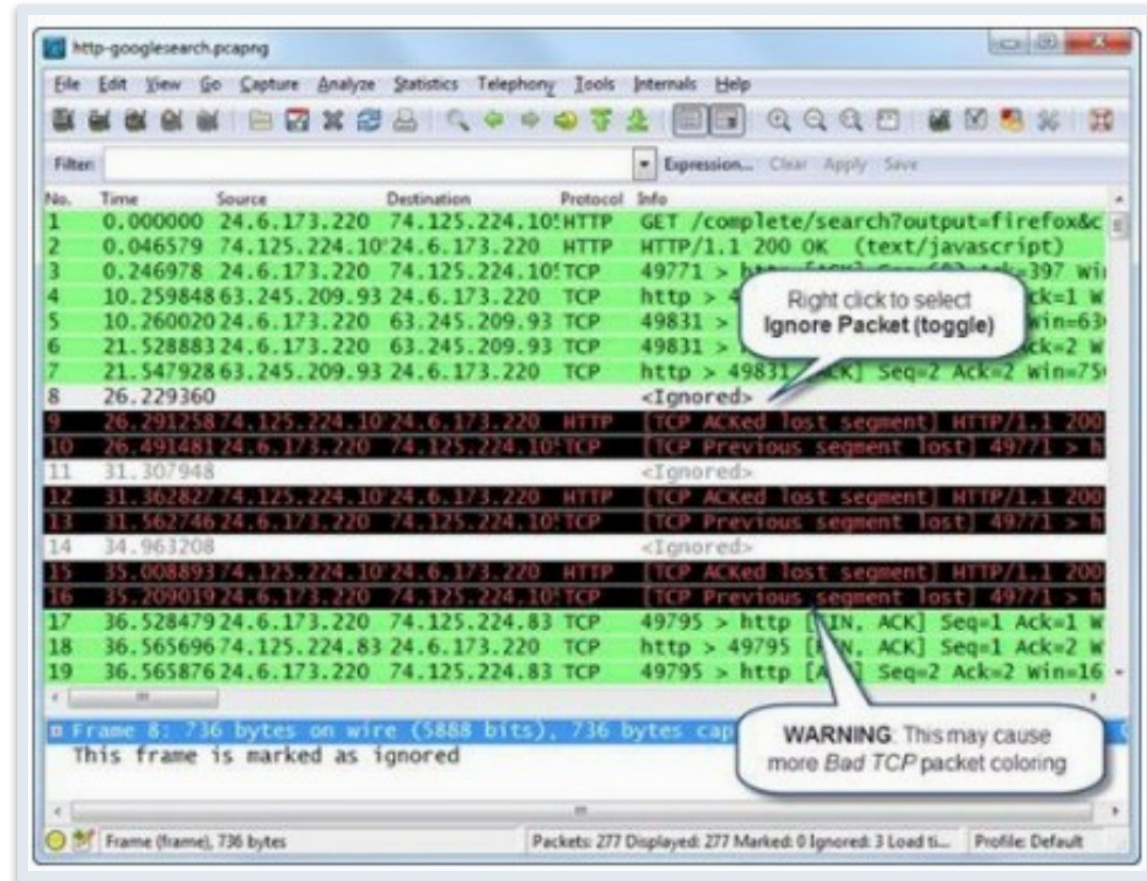
- Mark (Ctrl+M) for quick navigation — marks are lost when the file closes.
- Packet comments are saved in pcapng and travel with the evidence.
- Time Shift corrects clock offsets between captures.



Right-click | Packet Comment

Navigation Shortcuts

- Ctrl+G — go to packet number.
- Ctrl+F — find by display filter, hex or string.
- Ctrl+. / Ctrl+, — next/previous packet in the conversation.
- Ctrl+Space — collapse/expand the Packet Details tree.



Edit | Mark / Ignore packets

Color and annotate evidence

LAB 4

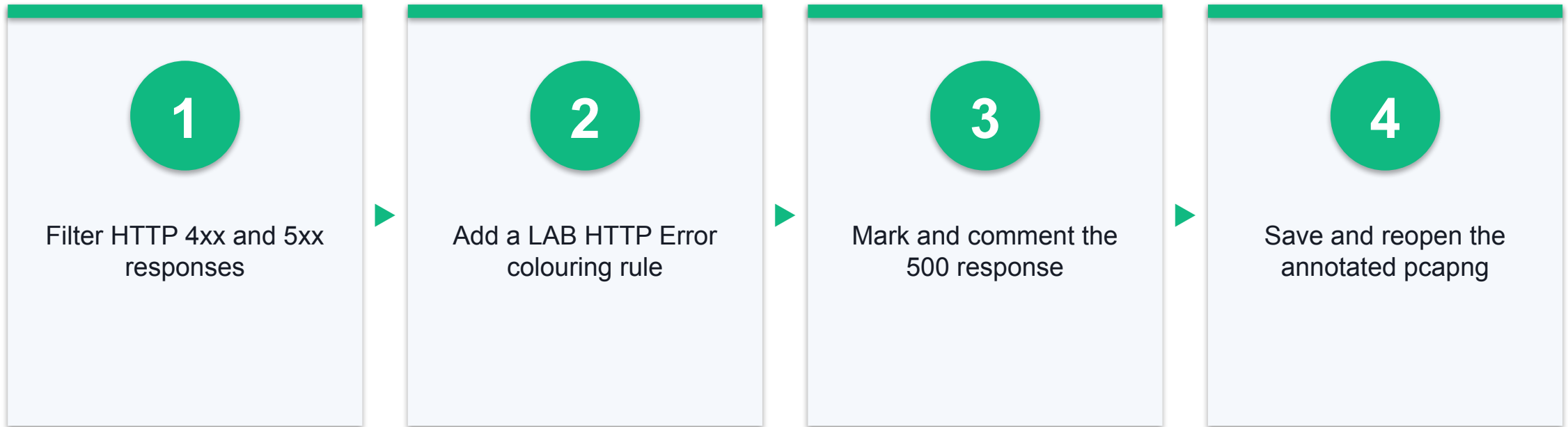
Users see intermittent web errors. You need the failing responses to stand out, and annotations that travel with the evidence.

You'll build

An annotated pcapng with a comment on the failing response

Tools: Wireshark, TShark, Python

Lab 4 — What You'll Do



Full step-by-step instructions: [Learner Guide · labs/lab-04-color-and-annotate-evidence/LAB-04-Instructions.pdf](#)

Expected Evidence — Lab 4

http.response.code >= 400					
No.	Time	Source	Destination	Protocol	Info
41	2.130000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 404 Not Found (text/plain)
50	2.270000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 500 Internal Server Error (text/plain)
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Color and annotate evidence

Test it

The supplied `http.response.code >= 400` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Navigation and Coloring Techniques

1

You can now: plan scoped captures and configure a reproducible analyst profile.

2

Color rules are applied in order; the first matching rule wins.

3

Temporary coloring is useful for a conversation; permanent rules support repeated triage.

4

Bookmarks and comments preserve the analyst path without changing the captured payload.

End of Day 1

See you tomorrow

DAY 2

Time, statistics, filters and DNS

Recap of the previous day, then today's topics and labs

TOPIC 05

Time Values and Delay Types

Separate path and server delay

Key Concepts — Time Values and Delay Types

1

Displayed delta depends on the current filter; capture delta does not.

2

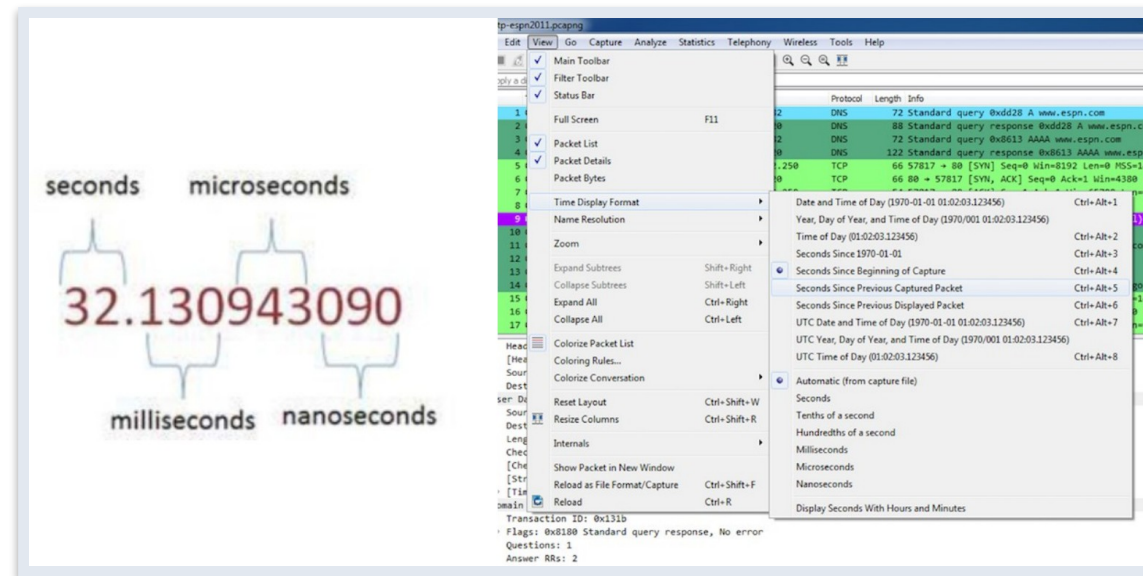
SYN to SYN/ACK gives an initial path-related sample; request to response includes application processing.

3

One-sided timestamps cannot identify exactly which intermediate device delayed or dropped a packet.

Time Display Formats

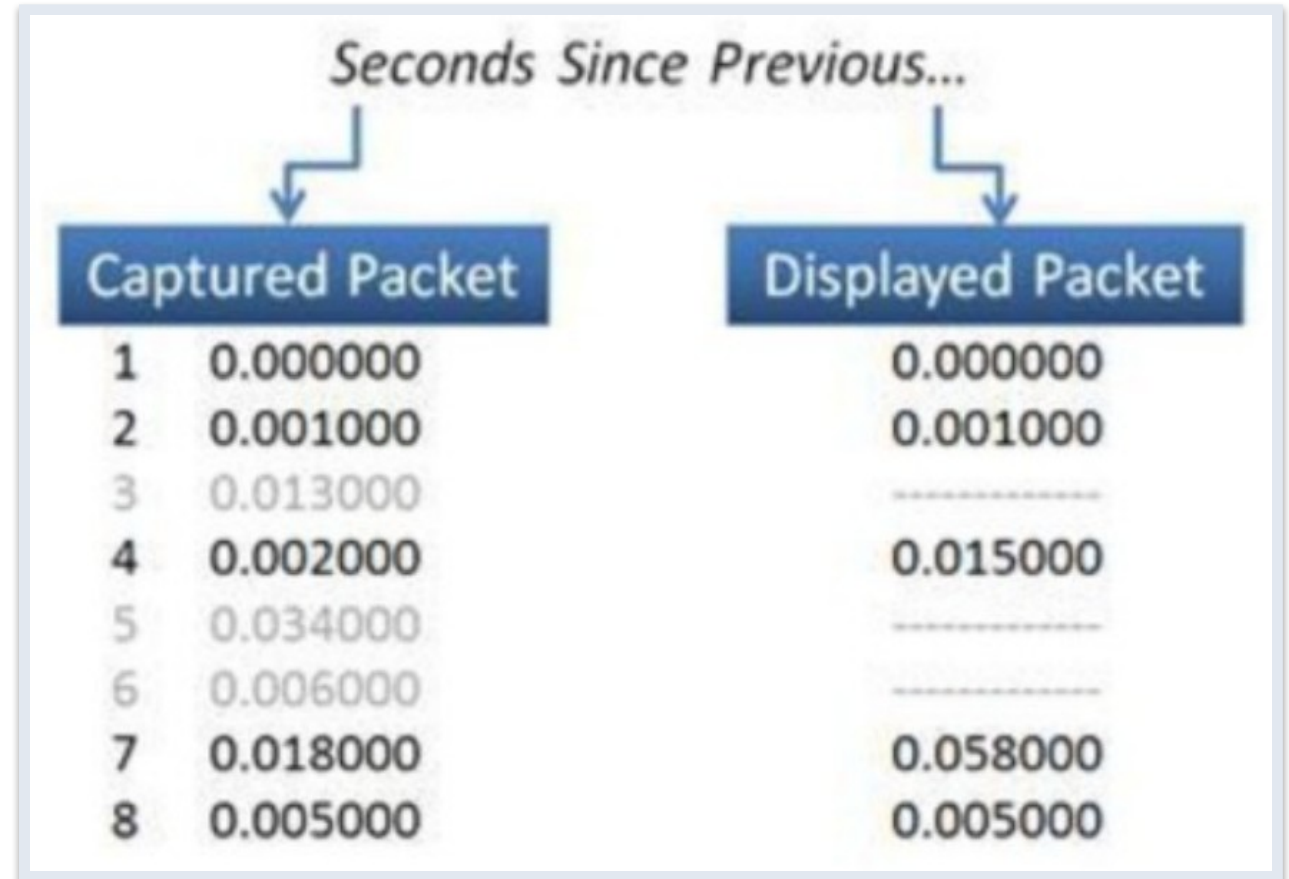
- View > Time Display Format chooses absolute, relative or delta time.
- Seconds Since Beginning of Capture — default overview.
- Seconds Since Previous Displayed Packet — finds gaps in a filtered view.



Precision depends on the capture hardware and file format

Captured vs Displayed Delta

- Delta since previous CAPTURED packet ignores the filter.
- Delta since previous DISPLAYED packet changes when you change the filter.
- State which delta you used in every finding.



<i>Seconds Since Previous...</i>	
Captured Packet	
1	0.000000
2	0.001000
3	0.013000
4	0.002000
5	0.034000
6	0.006000
7	0.018000
8	0.005000
Displayed Packet	
	0.000000
	0.001000

	0.015000

	0.058000

	0.005000

Same packets — different deltas

End-to-End Path Delay

- SYN → SYN/ACK gives the initial round-trip time (iRTT) near the client.
- Request → response adds server processing time.
- Large gaps after an ACK point to the server, not the path.

http-download-bad.pcapng

File Edit View Packets Bytes Time Filter: Expression... Clear Apply Save

TCP packets captured near the client indicate the roundtrip path latency time is ~167ms

After the handshake we see the roundtrip response time is 155ms

The Time column is set to Seconds Since Previous Displayed Packet

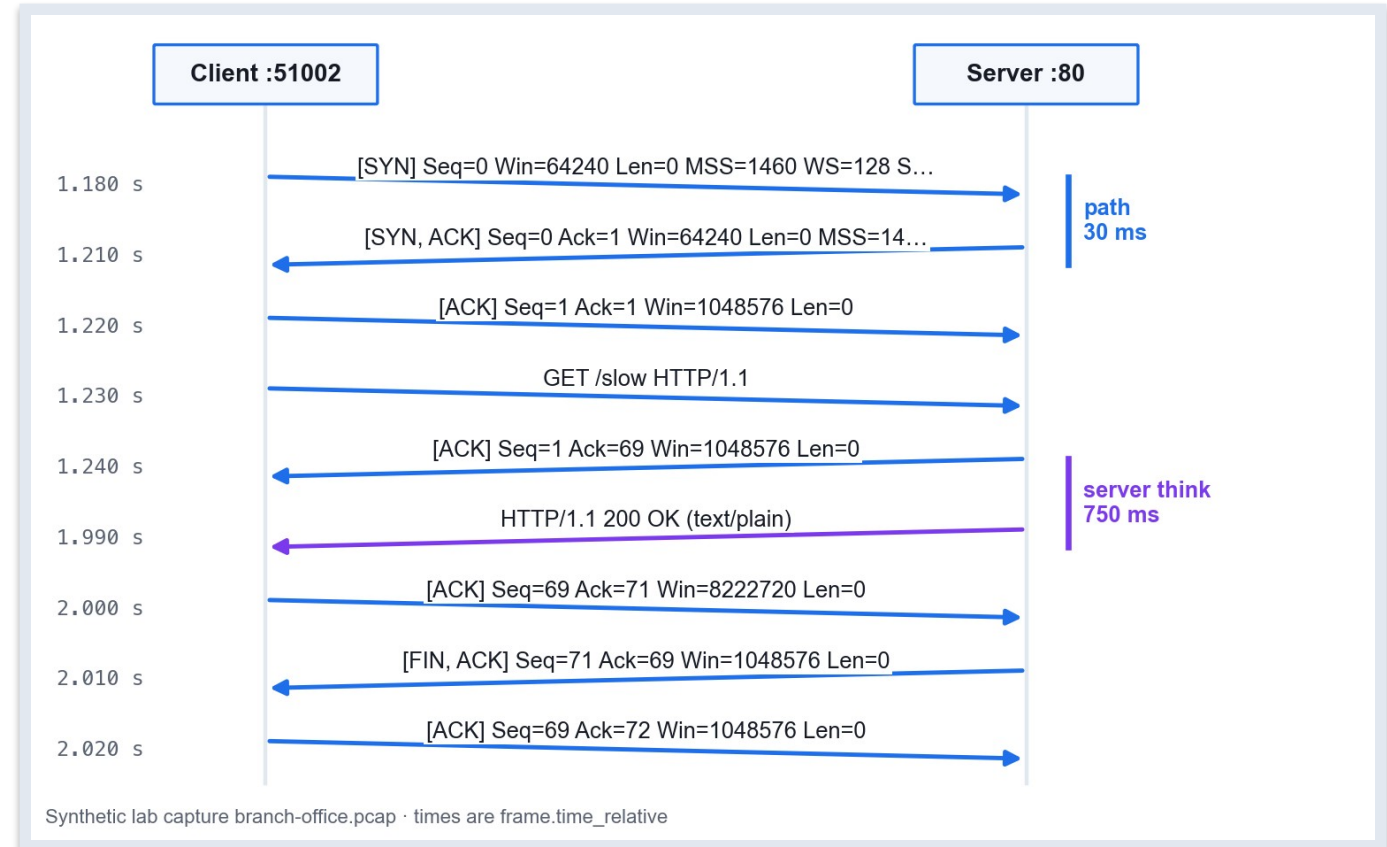
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.52.164	61.8.0.17	TCP	66	ads > http [SYN] Seq=0 win=6553
2	0.167521	61.8.0.17	10.0.52.164	TCP	66	http > ads [SYN, ACK] Seq=0 Ack=
3	0.000035	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=1 Ack=1 Wi
4	0.002194	10.0.52.164	61.8.0.17	HTTP	499	GET /openoffice/stable/2.0.0/00
5	0.155654	61.8.0.17	10.0.52.164	TCP	60	http > ads [ACK] Seq=1 Ack=446
6	0.001938	10.0.52.164	61.8.0.17	HTTP	383	HTTP/1.1 200 OK
7	0.007844	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
8	0.000306	61.8.0.17	10.0.52.164	TCP	54	ads > http [ACK] Seq=446 Ack=17
9	0.000115	10.0.52.164	61.8.0.17	TCP	54	[TCP Window Update] ads > http
10	0.157278	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
11	0.000289	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=32
12	0.005443	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
13	0.000174	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=47
14	0.006313	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
15	0.000148	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=61
16	0.170887	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
17	0.000280	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=76
18	0.005818	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
19	0.000068	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=90
20	0.011148	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
21	0.000043	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=10
22	0.005949	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi
23	0.000047	10.0.52.164	61.8.0.17	TCP	54	ads > http [ACK] Seq=446 Ack=112
24	0.000100	10.0.52.164	61.8.0.17	HTTP	1514	Continuation or non-HTTP traffi

File: "C:\traces-pcapngset\http-download-bad..." Packets: 7195 Displayed: 7195 Marked: 0 Load time: 0:00:1... Profile: Default

Sort a delta-time column to find the largest gaps

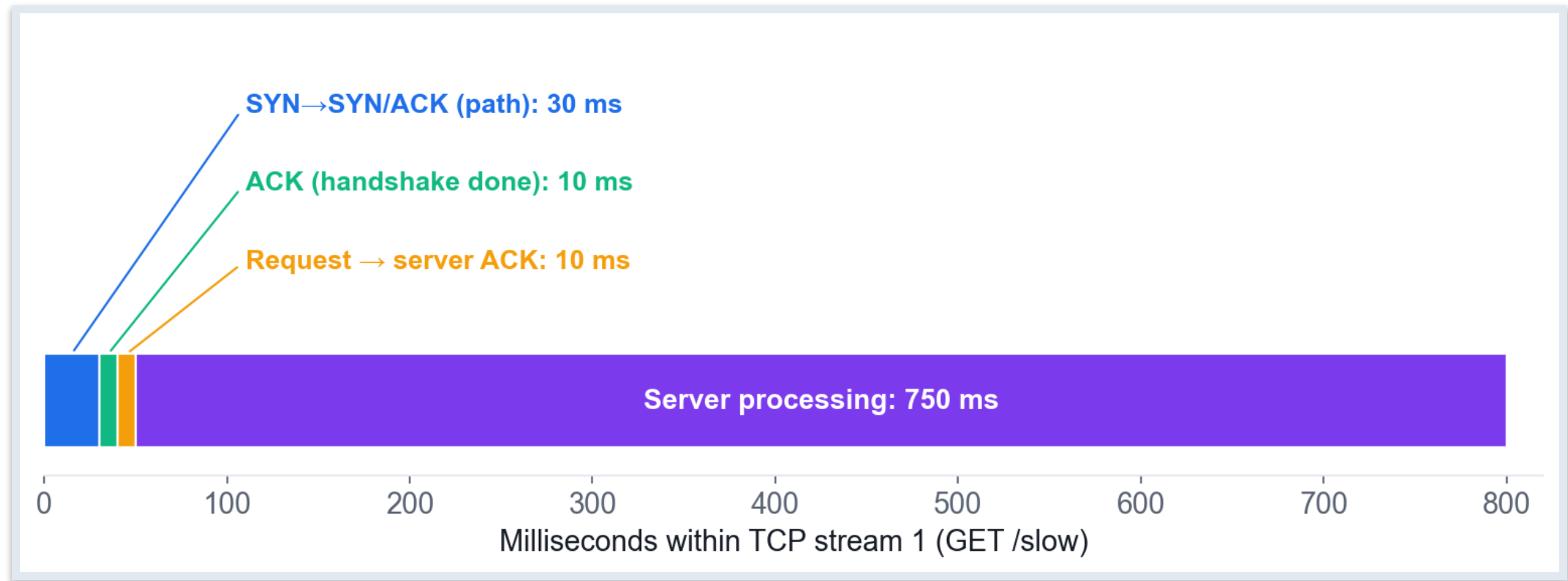
Timing the Slow Request

- Stream 1 handshake: SYN → SYN/ACK 30 ms, complete in 40 ms.
- The server ACKs GET /slow in 10 ms ...
- ... then needs 750 ms before the 200 OK.
- The path is fast; the delay is server processing.



TCP stream 1 from the lab capture

Where the 0.8 Seconds Went



Measured intervals inside TCP stream 1 — path delay is small; server processing dominates

Three Kinds of Delay

Path delay

- SYN → SYN/ACK (iRTT)
- `tcp.analysis.ack_rtt`
- Affects every exchange

Server delay

- ACK → first response byte
- `http.time` for HTTP
- Grows with server load

Client delay

- Response → next request
- Think time or app processing
- Often not a network problem

Time Reference and Timestamps

1

Ctrl+T sets a time reference — times are then measured from that packet.

2

Use a reference per conversation to time one transaction in isolation.

3

Timestamp precision is set by the capture system; you cannot add precision later.

4

Clock offsets between capture points need Time Shift before comparing.

Separate path and server delay

LAB 5

The /slow page takes almost a second to load. The network team blames the server and the server team blames the network — your timing evidence decides.

You'll build

Timing evidence separating path delay from server delay

Tools: Wireshark, TShark, Python

Lab 5 — What You'll Do

1

Measure
SYN→SYN/ACK and
initial RTT

2

Time the /slow request
to its response

3

Compare captured vs
displayed deltas

4

Write a hypothesis and
next capture point

Full step-by-step instructions: [Learner Guide · labs/lab-05-separate-path-and-server-delay/LAB-05-Instructions.pdf](#)

Expected Evidence — Lab 5

http.request.uri == "/slow" http.response.code == 200					
No.	Time	Source	Destination	Protocol	Info
23	1.140000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
30	1.230000	192.0.2.10	192.0.2.20	HTTP	GET /slow HTTP/1.1
32	1.990000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
branch-office.pcap Displayed: 3					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Separate path and server delay

Test it

The supplied `http.request.uri == "/slow" || http.response.code == 200` expression matches 3 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Time Values and Delay Types

1 You can now: navigate, filter, summarize and time network exchanges.

2 Displayed delta depends on the current filter; capture delta does not.

3 SYN to SYN/ACK gives an initial path-related sample; request to response includes application processing.

4 One-sided timestamps cannot identify exactly which intermediate device delayed or dropped a packet.

Tea Break

15 minutes

TOPIC 06

Trace Statistics and VoIP Overview

Summarize traffic and a voice stream

Key Concepts — Trace Statistics and VoIP Overview

1

Protocol Hierarchy shows captured composition; byte share differs from packet share.

2

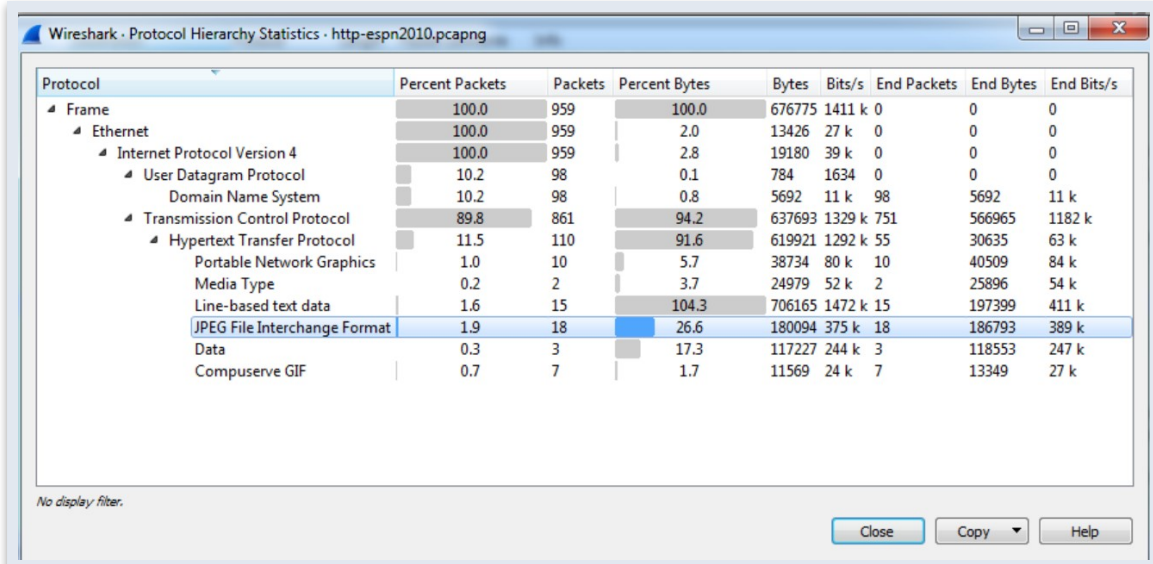
Conversations and Endpoints identify concentration; they do not alone prove malicious activity.

3

SIP signals a call while RTP carries media; jitter, loss and codec interpretation need stream context.

Capture File Properties

- Statistics > Capture File Properties: file, time span, interface and drop counts.
- Check packets dropped before blaming the network for loss.
- Record these facts at the start of every analysis.



Wireshark · Protocol Hierarchy Statistics · http-espn2010.pcapng

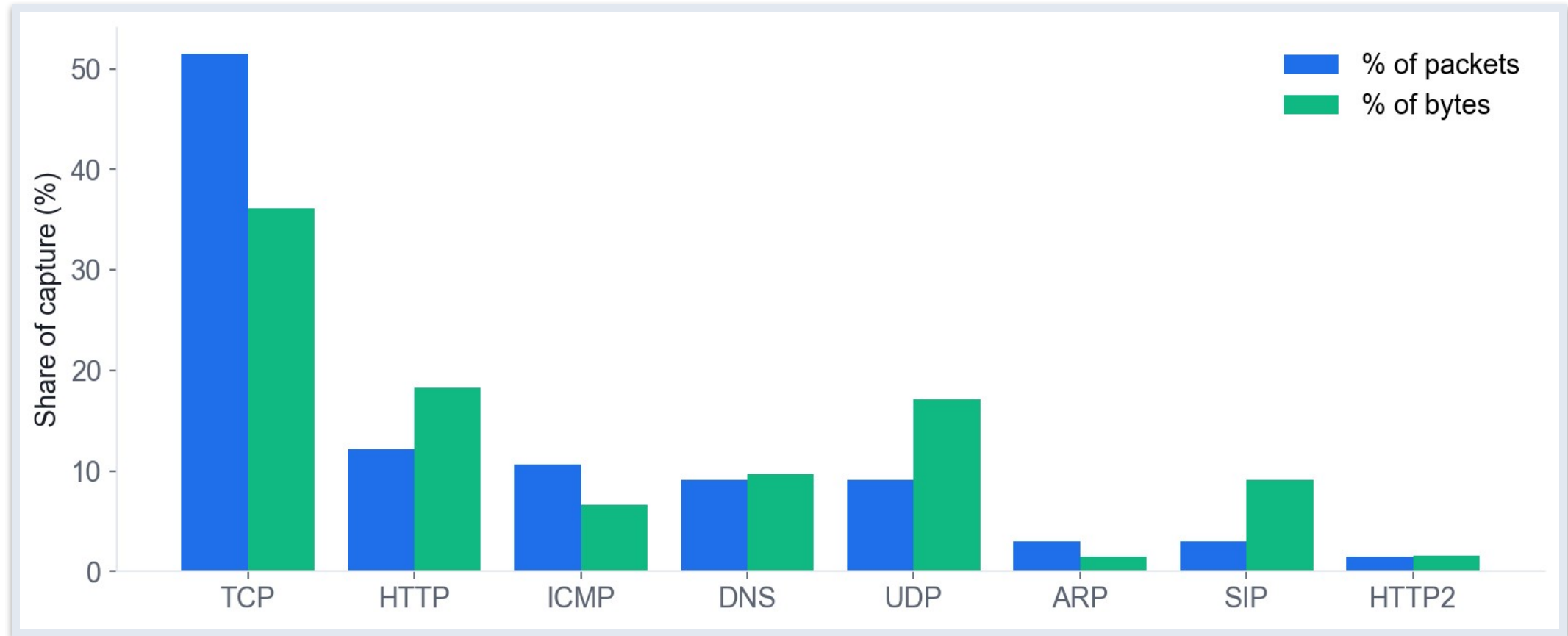
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	959	100.0	676775	1411 k	0	0	0
Ethernet	100.0	959	2.0	13426	27 k	0	0	0
Internet Protocol Version 4	100.0	959	2.8	19180	39 k	0	0	0
User Datagram Protocol	10.2	98	0.1	784	1634	0	0	0
Domain Name System	10.2	98	0.8	5692	11 k	98	5692	11 k
Transmission Control Protocol	89.8	861	94.2	637693	1329 k	751	566965	1182 k
Hypertext Transfer Protocol	11.5	110	91.6	619921	1292 k	55	30635	63 k
Portable Network Graphics	1.0	10	5.7	38734	80 k	10	40509	84 k
Media Type	0.2	2	3.7	24979	52 k	2	25896	54 k
Line-based text data	1.6	15	104.3	706165	1472 k	15	197399	411 k
JPEG File Interchange Format	1.9	18	26.6	180094	375 k	18	186793	389 k
Data	0.3	3	17.3	117227	244 k	3	118553	247 k
Compuserve GIF	0.7	7	1.7	11569	24 k	7	13349	27 k

No display filter.

Close Copy Help

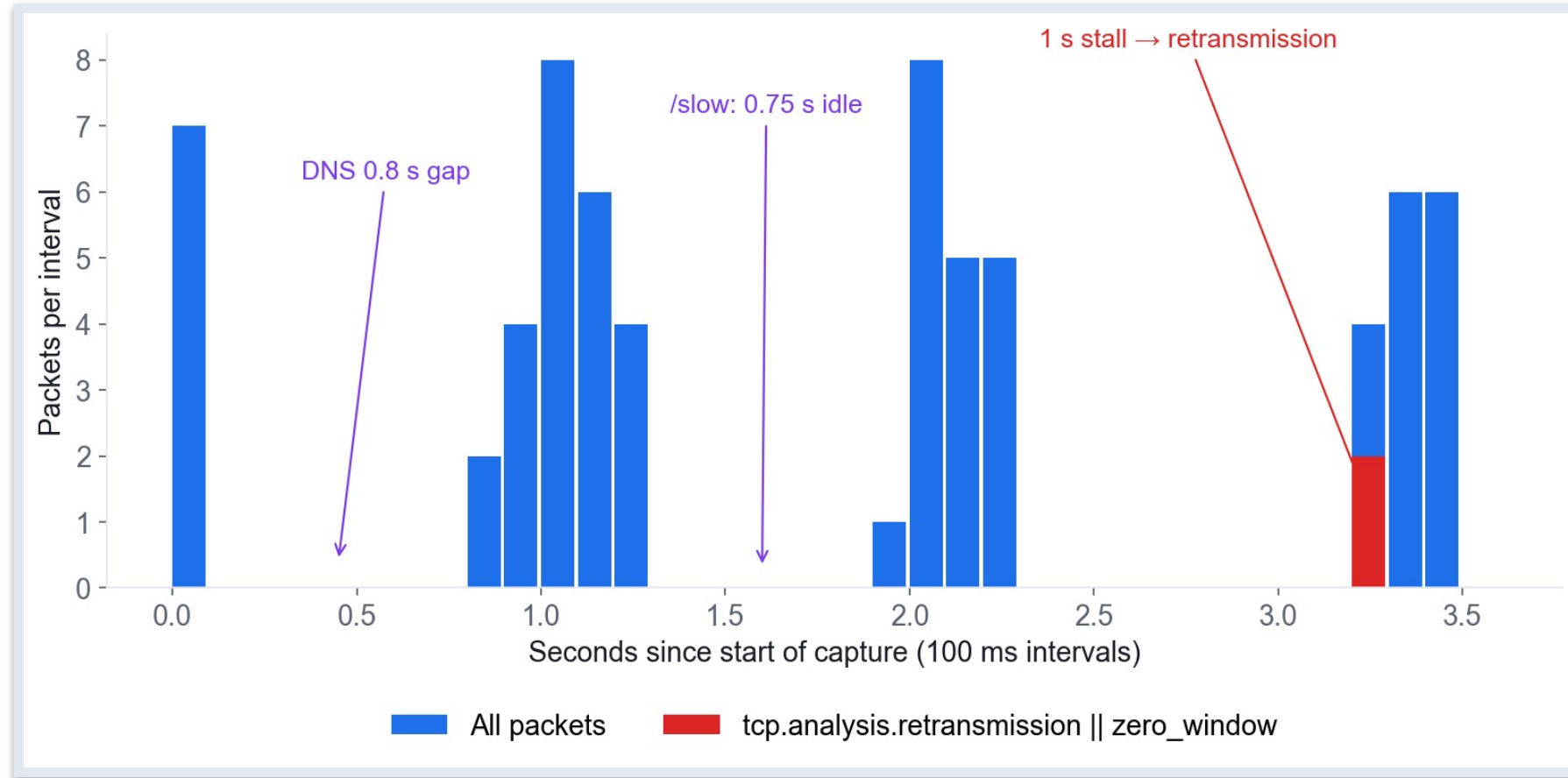
Statistics | Protocol Hierarchy shows the protocol mix

Packets vs Bytes by Protocol



Protocol Hierarchy of the lab capture: TCP has the most packets, but byte share tells a different story

Reading the I/O Graph



Gaps show waiting; the red bar marks the retransmission/zero-window events

Conversations and Endpoints

- Statistics > Conversations lists pairs at Ethernet, IPv4/6, TCP and UDP level.
- Sort by Bytes to find the most active conversation.
- Right-click a row to filter on it.

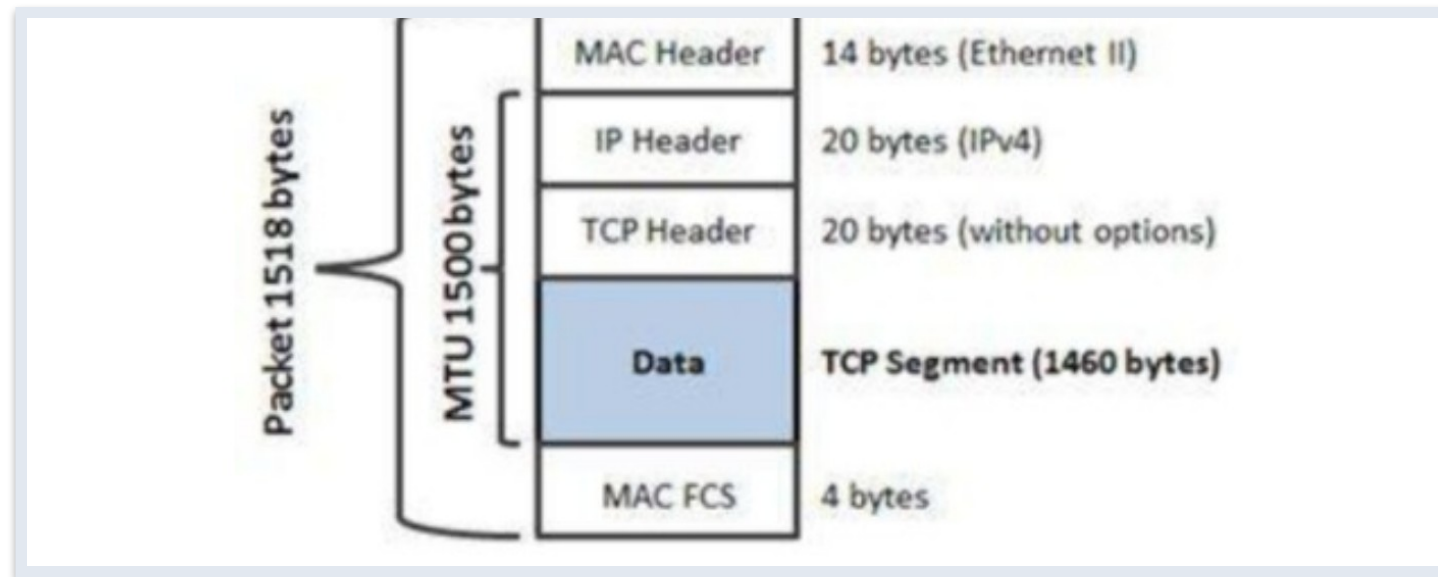
The screenshot shows the 'Conversations' window in Wireshark, filtered for TCP. The table is sorted by 'Bytes' in descending order. The columns are: Address A, Port A, Address B, Port B, Packets, Bytes, Packets A → B, Bytes A → B, Packets B → A, and Bytes B → A. The first row shows a conversation between 69.64.34.124 (Port 6667) and 172.16.1.10 (Port 1049) with 48 packets and 8429 bytes. The second row shows a conversation between 68.164.173.62 (Port 1216) and 172.16.1.10 (Port 135) with 17 packets and 3986 bytes. The third row shows a conversation between 172.16.1.10 (Port 1137) and 216.127.33.119 (Port 80) with 9 packets and 3593 bytes. The fourth row shows a conversation between 172.16.1.10 (Port 1138) and 216.127.33.119 (Port 80) with 9 packets and 3593 bytes. The fifth row shows a conversation between 172.16.1.10 (Port 1144) and 216.127.33.119 (Port 80) with 9 packets and 3593 bytes. The sixth row shows a conversation between 68.164.194.35 (Port 2026) and 172.16.1.10 (Port 135) with 12 packets and 2278 bytes. The seventh row shows a conversation between 68.164.173.62 (Port 4800) and 172.16.1.10 (Port 135) with 16 packets and 1490 bytes. The eighth row shows a conversation between 172.16.1.10 (Port 1092) and 216.127.33.119 (Port 80) with 11 packets and 1380 bytes. The ninth row shows a conversation between 68.164.194.35 (Port 2006) and 172.16.1.10 (Port 135) with 11 packets and 1088 bytes. The tenth row shows a conversation between 172.16.1.10 (Port 1093) and 216.127.33.119 (Port 80) with 9 packets and 893 bytes. The eleventh row shows a conversation between 68.164.173.62 (Port 4731) and 172.16.1.10 (Port 135) with 13 packets and 748 bytes. The twelfth row shows a conversation between 68.164.194.35 (Port 1996) and 172.16.1.10 (Port 135) with 7 packets and 412 bytes. The thirteenth row shows a conversation between 172.16.1.10 (Port 2702) and 216.155.193.156 (Port 119) with 5 packets and 350 bytes. The fourteenth row shows a conversation between 172.16.1.10 (Port 26452) and 68.45.134.187 (Port 4022) with 6 packets and 348 bytes. The fifteenth row shows a conversation between 172.16.1.10 (Port 26452) and 67.38.252.160 (Port 3418) with 6 packets and 348 bytes. The sixteenth row shows a conversation between 172.16.1.10 (Port 26452) and 68.45.134.187 (Port 4068) with 6 packets and 348 bytes. The seventeenth row shows a conversation between 172.16.1.10 (Port 26452) and 67.38.252.160 (Port 3438) with 6 packets and 348 bytes. The eighteenth row shows a conversation between 172.16.1.10 (Port 26452) and 67.38.252.160 (Port 3459) with 6 packets and 348 bytes. The nineteenth row shows a conversation between 172.16.1.10 (Port 26452) and 80.167.183.40 (Port 4215) with 6 packets and 348 bytes. The twentieth row shows a conversation between 172.16.1.10 (Port 26452) and 67.38.252.160 (Port 3481) with 6 packets and 348 bytes. The twenty-first row shows a conversation between 172.16.1.10 (Port 26452) and 68.45.134.187 (Port 4475) with 6 packets and 348 bytes. The twenty-second row shows a conversation between 172.16.1.10 (Port 26452) and 68.45.134.187 (Port 4504) with 6 packets and 348 bytes. The twenty-third row shows a conversation between 172.16.1.10 (Port 4364) and 207.46.196.46 (Port 80) with 6 packets and 324 bytes. The twenty-fourth row shows a conversation between 172.16.1.10 (Port 4392) and 207.46.196.46 (Port 80) with 6 packets and 324 bytes.

Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A
69.64.34.124	6667	172.16.1.10	1049	48	8429	24	5983	24	2446
68.164.173.62	1216	172.16.1.10	135	17	3986	9	3486	8	500
172.16.1.10	1137	216.127.33.119	80	9	3593	5	499	4	3094
172.16.1.10	1138	216.127.33.119	80	9	3593	5	499	4	3094
172.16.1.10	1144	216.127.33.119	80	9	3593	5	499	4	3094
68.164.194.35	2026	172.16.1.10	135	12	2278	7	1940	5	338
68.164.173.62	4800	172.16.1.10	135	16	1490	7	572	9	918
172.16.1.10	1092	216.127.33.119	80	11	1380	6	590	5	790
68.164.194.35	2006	172.16.1.10	135	11	1088	6	446	5	642
172.16.1.10	1093	216.127.33.119	80	9	893	5	407	4	486
68.164.173.62	4731	172.16.1.10	135	13	748	6	362	7	386
68.164.194.35	1996	172.16.1.10	135	7	412	4	242	3	170
172.16.1.10	2702	216.155.193.156	119	5	350	3	230	2	120
172.16.1.10	26452	68.45.134.187	4022	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3418	6	348	3	162	3	186
172.16.1.10	26452	68.45.134.187	4068	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3438	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3459	6	348	3	162	3	186
172.16.1.10	26452	80.167.183.40	4215	6	348	3	162	3	186
172.16.1.10	26452	67.38.252.160	3481	6	348	3	162	3	186
172.16.1.10	26452	68.45.134.187	4475	6	348	3	162	3	186
172.16.1.10	26452	68.45.134.187	4504	6	348	3	162	3	186
172.16.1.10	4364	207.46.196.46	80	6	324	6	324	0	0
172.16.1.10	4392	207.46.196.46	80	6	324	6	324	0	0

Statistics | Conversations — TCP tab, sorted by bytes

Packet Lengths

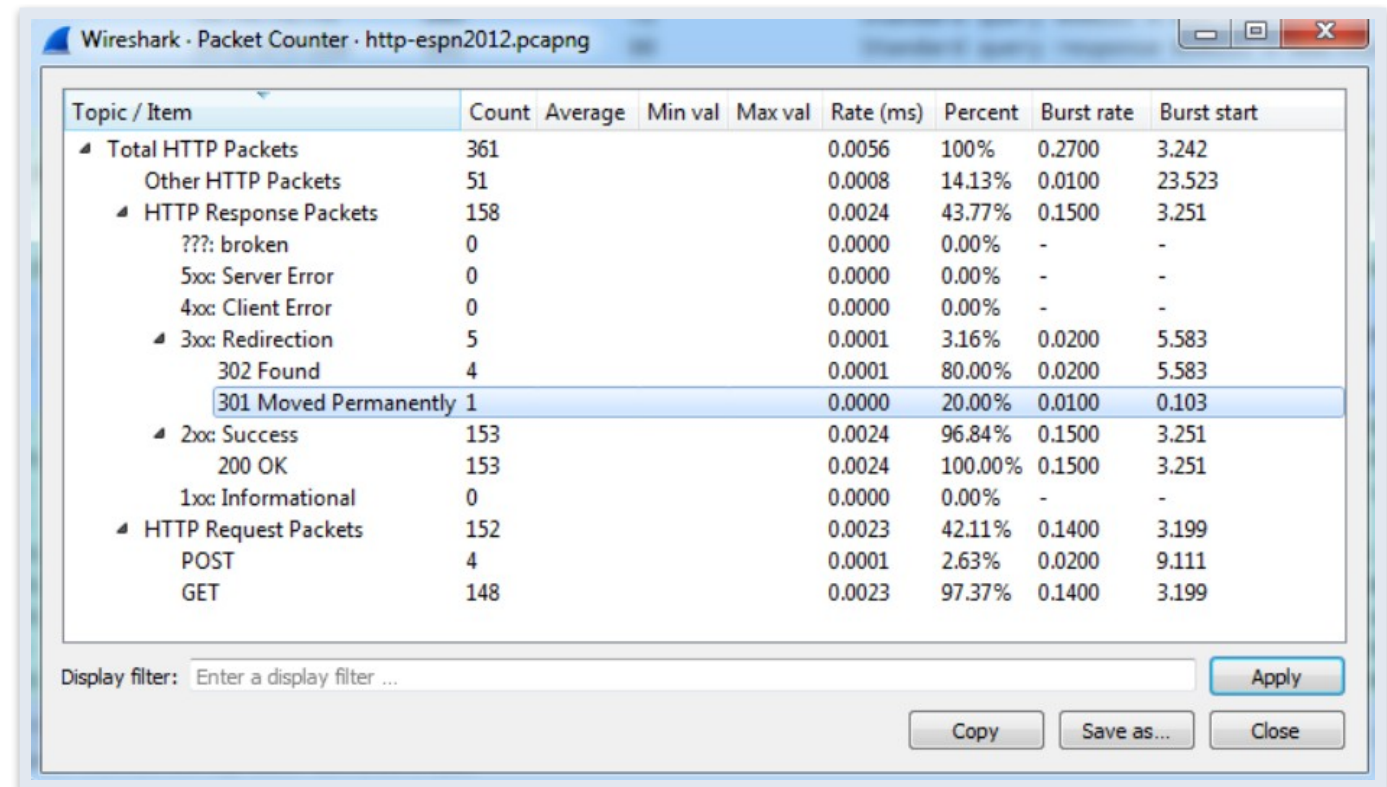
- Statistics > Packet Lengths shows the size distribution.
- Many tiny packets moving bulk data suggest inefficient application behaviour.
- A full-size Ethernet frame is 1518 bytes; TCP payload at MTU 1500 is 1460.



Frame size components

Flow Graph and HTTP Statistics

- Statistics > Flow Graph draws the exchange between hosts as a ladder diagram.
- Statistics > HTTP > Packet Counter summarises requests and response codes.
- Use both to explain sequence and outcome in a report.



The screenshot shows the 'Wireshark - Packet Counter - http-espn2012.pcapng' window. It displays a table of HTTP statistics. The '301 Moved Permanently' entry is highlighted in blue.

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
▲ Total HTTP Packets	361				0.0056	100%	0.2700	3.242
Other HTTP Packets	51				0.0008	14.13%	0.0100	23.523
▲ HTTP Response Packets	158				0.0024	43.77%	0.1500	3.251
??? broken	0				0.0000	0.00%	-	-
5xx Server Error	0				0.0000	0.00%	-	-
4xx Client Error	0				0.0000	0.00%	-	-
▲ 3xx Redirection	5				0.0001	3.16%	0.0200	5.583
302 Found	4				0.0001	80.00%	0.0200	5.583
301 Moved Permanently	1				0.0000	20.00%	0.0100	0.103
▲ 2xx Success	153				0.0024	96.84%	0.1500	3.251
200 OK	153				0.0024	100.00%	0.1500	3.251
1xx Informational	0				0.0000	0.00%	-	-
▲ HTTP Request Packets	152				0.0023	42.11%	0.1400	3.199
POST	4				0.0001	2.63%	0.0200	9.111
GET	148				0.0023	97.37%	0.1400	3.199

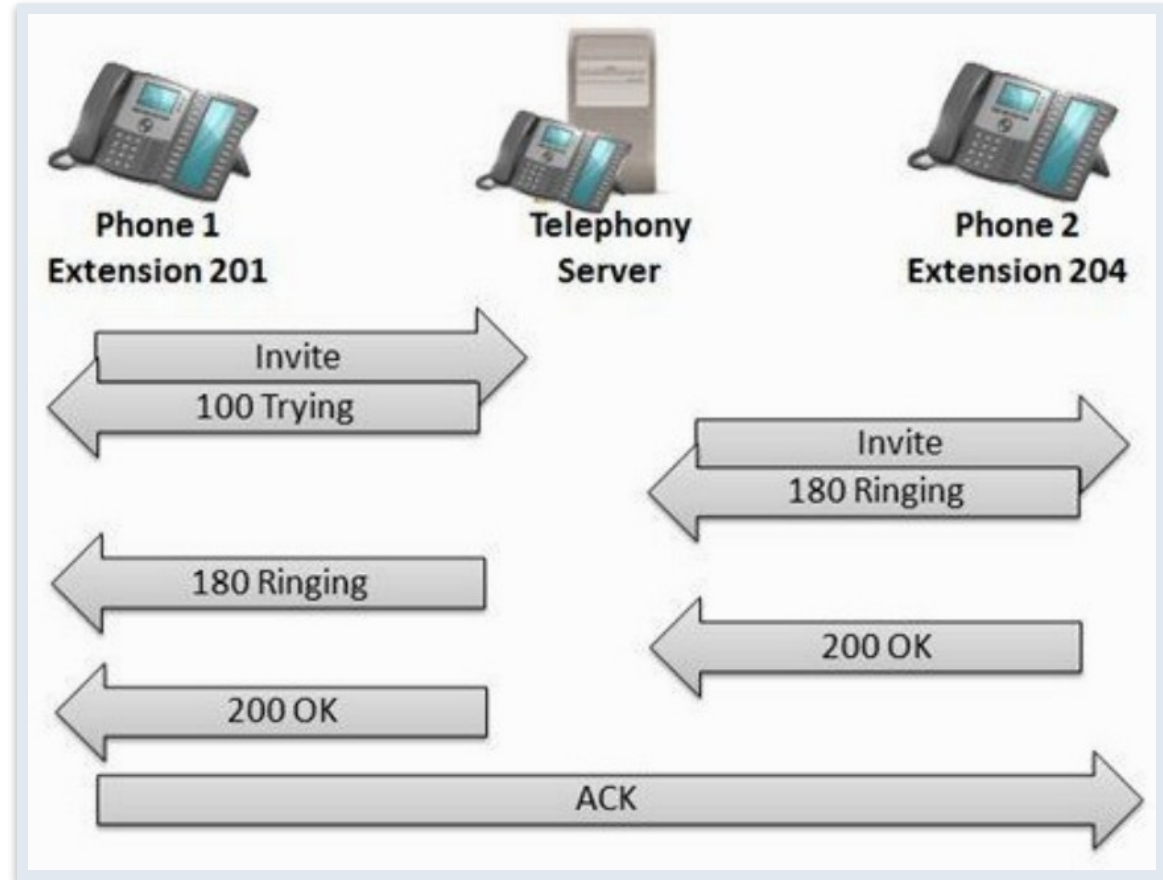
Display filter: Enter a display filter ...

Buttons: Apply, Copy, Save as..., Close

Statistics | HTTP | Packet Counter

Voice over IP (VoIP)

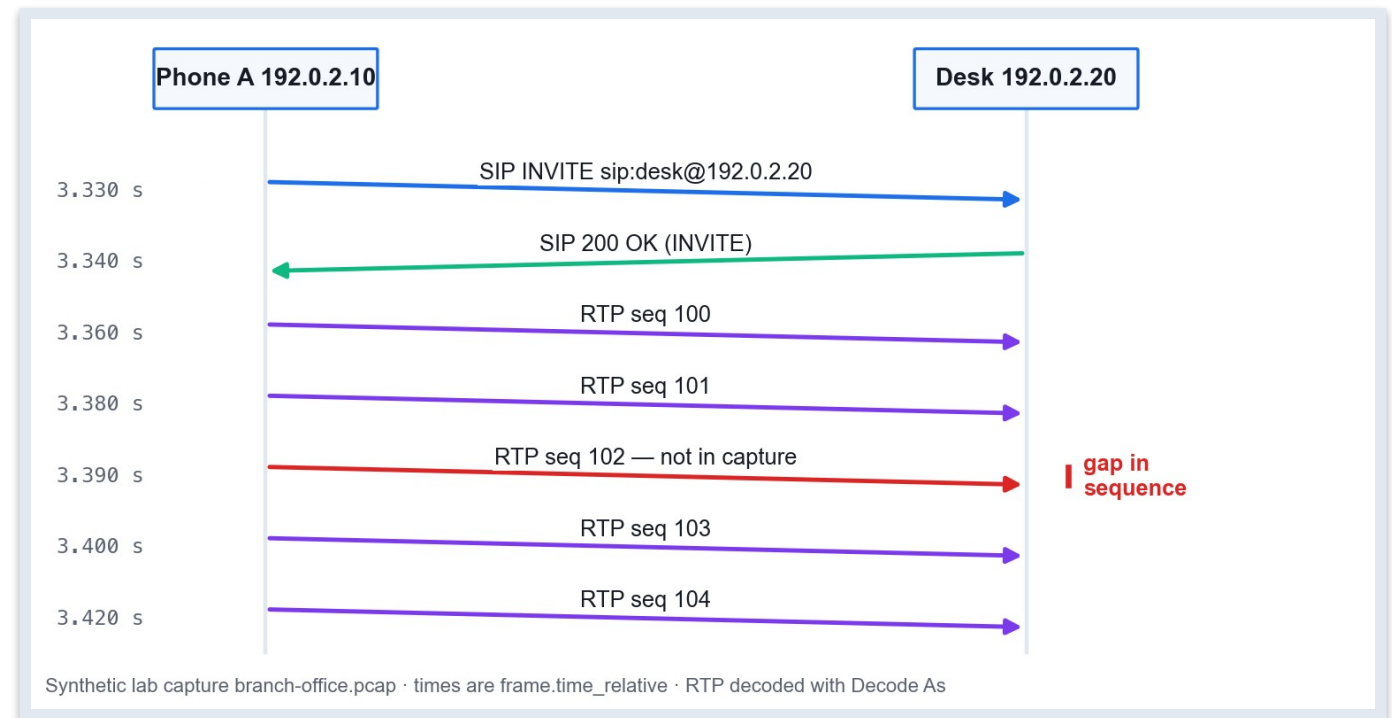
- SIP signals the call: INVITE, 100 Trying, 180 Ringing, 200 OK, ACK.
- RTP carries the media; gaps in RTP sequence numbers indicate loss.
- Telephony > VoIP Calls, SIP Flows and RTP Streams analyse calls.



SIP call set-up between two phones and a telephony server

SIP Signalling and RTP Media

- INVITE / 200 OK set up the call on UDP 5060.
- RTP carries the voice — here on UDP 4002, decoded with Decode As.
- Sequence 100, 101, 103, 104: number 102 never arrived at this capture point.



The lab capture's test call

Summarize traffic and a voice stream

LAB 6

Management wants a one-page summary of what crossed the branch link, including a test voice call that users said sounded choppy.

You'll build

A traffic summary and RTP loss evidence

Tools: Wireshark, TShark, Python

Lab 6 — What You'll Do

1

Review Protocol
Hierarchy and
Conversations

2

Graph all traffic in bits
per second

3

Identify the SIP INVITE
and 200 OK

4

Decode RTP and find
the missing sequence

Full step-by-step instructions: [Learner Guide · labs/lab-06-summarize-traffic-and-a-voice-stream/LAB-06-Instructions.pdf](#)

Expected Evidence — Lab 6

sip					
No.	Time	Source	Destination	Protocol	Info
57	3.330000	192.0.2.10	192.0.2.20	SIP	Request: INVITE sip:desk@192.0.2.20
58	3.340000	192.0.2.20	192.0.2.10	SIP	Status: 200 OK (INVITE)
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Summarize traffic and a voice stream

Test it

The supplied sip expression matches 2 frame(s) in the specified capture. Run scripts/verify.py and compare the listed frame numbers. Keep your findings and exported table in outputs/. Explain the observed result rather than only copying a count.

Recap — Trace Statistics and VoIP Overview

1 You can now: navigate, filter, summarize and time network exchanges.

2 Protocol Hierarchy shows captured composition; byte share differs from packet share.

3 Conversations and Endpoints identify concentration; they do not alone prove malicious activity.

4 SIP signals a call while RTP carries media; jitter, loss and codec interpretation need stream context.

Lunch Break

1 hour

TOPIC 07

Display Filters

Build a filter evidence matrix

Key Concepts — Display Filters

1

Parentheses make mixed and/or expressions explicit.

2

Field existence and Boolean equality differ: `tcp.flags.syn == 1` tests the bit.

3

Since Wireshark 3.6, `!=` uses all-not-equal semantics; historical slides describing any-not-equal are obsolete.

Display Filter Operators

- Comparison: == != > < >= <= (or eq ne gt lt ge le).
- Logical: && (and), || (or), ! (not); parentheses make intent explicit.
- Strings in double quotes; contains and matches search payload text.

Shortcut	C-Like	Description	Example
eq	==	Equal	Ip.src == 10.1.1.5
ne	!=	Not equal	Ip.src != 10.1.1.5
gt	>	Greater than	Frame.len > 64
lt	<	Less Than	Frame.len < 1518
ge	>=	Greater then or equal to	Frame.len ge 0x100
le	<=	Less then or equal to	Frame.len <= 0x20

Comparison operators with C-like and English forms

The != Trap (Wireshark 3.6+)

`ip.addr != 10.2.4.1`

- Means ALL ip.addr fields are not 10.2.4.1
- Excludes every packet to OR from 10.2.4.1
- This is the current behaviour

`ip.addr ~= 10.2.4.1`

- "Any not equal" — the old meaning of !=
- Keeps packets where either address differs
- Prefer `!(ip.addr == 10.2.4.1)` for clarity

Filter Building Techniques

1 Right-click a field > Apply as Filter / Prepare as Filter.

2 Start typing — autocomplete lists valid field names.

3 Save frequent filters as filter buttons in your profile.

4 Test field existence (`tcp.analysis.flags`) vs a value (`tcp.flags.syn == 1`).

Operators and Functions Worth Knowing

1

tcp.port in {80, 443, 8080}

Set membership (comma-separated)

2

http.host contains "example"

Substring match

3

dns.qry.name matches "^miss"

Regular expression

4

eth.src[0:3] == 02:00:00

Byte slice of a field

5

len(http.request.uri) > 50

Length of a field

6

!(arp || dns)

Exclude whole protocols

Filters You Will Use All Week

1

arp || icmp

Local resolution and reachability

2

dns.flags.rcode != 0

DNS errors

3

tcp.flags.syn == 1 && tcp.flags.ack == 0

Connection attempts

4

tcp.analysis.flags

All TCP expert warnings

5

http.response.code >= 400

Client and server errors

6

frame.time_delta_displayed > 1

Gaps over 1 s in the view

Build a filter evidence matrix

LAB 7

The team keeps sharing filters that silently hide evidence. You build a tested filter matrix so everyone answers the same question the same way.

You'll build

A filter evidence matrix with verified frame numbers

Tools: Wireshark, TShark, Python

Lab 7 — What You'll Do

1

Find the NXDOMAIN
response and SYNs

2

Compare two
bracketings of one filter

3

Test `!(ip.addr == x)` vs
`ip.addr != x`

4

Save five verified filters
with frames

Full step-by-step instructions: [Learner Guide · labs/lab-07-build-a-filter-evidence-matrix/LAB-07-Instructions.pdf](#)

Expected Evidence — Lab 7

dns.flags.rcode == 3					
No.	Time	Source	Destination	Protocol	Info
6	0.070000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0066 No such name A mis...
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Build a filter evidence matrix

Test it

The supplied `dns.flags.rcode == 3` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Display Filters

1

You can now: navigate, filter, summarize and time network exchanges.

2

Parentheses make mixed and/or expressions explicit.

3

Field existence and Boolean equality differ: `tcp.flags.syn == 1` tests the bit.

4

Since Wireshark 3.6, `!=` uses all-not-equal semantics; historical slides describing any-not-equal are obsolete.

TOPIC 08

TCP/IP Communications and Resolution

Reconstruct an application dependency chain

Key Concepts — TCP/IP Communications and Resolution

1

ARP resolves a local next-hop MAC; DNS resolves a name to an address.

2

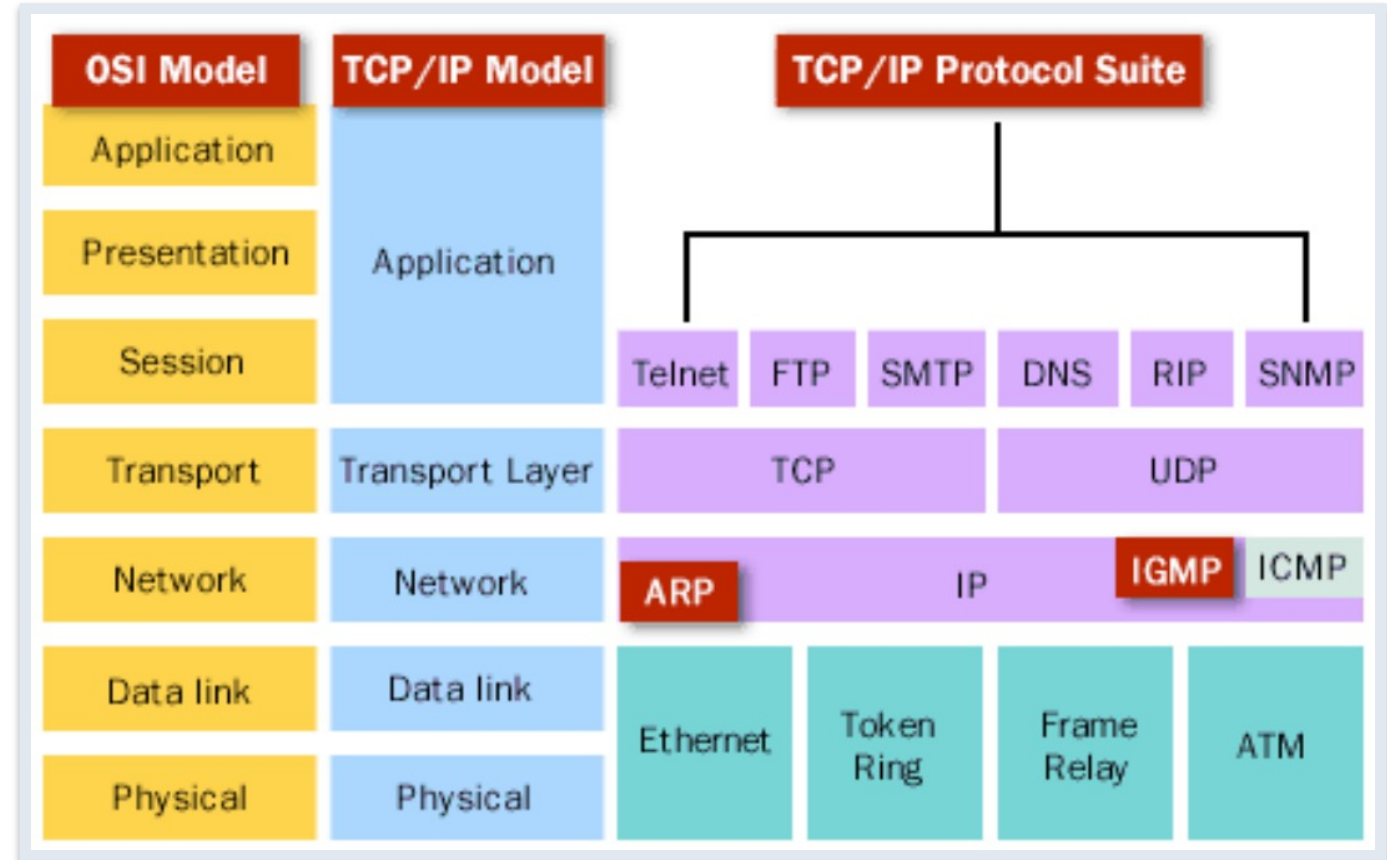
The destination IP stays end-to-end across routing while link-layer addresses change per hop.

3

Resolution, connection establishment and application exchange form a dependency chain.

The TCP/IP Protocol Suite

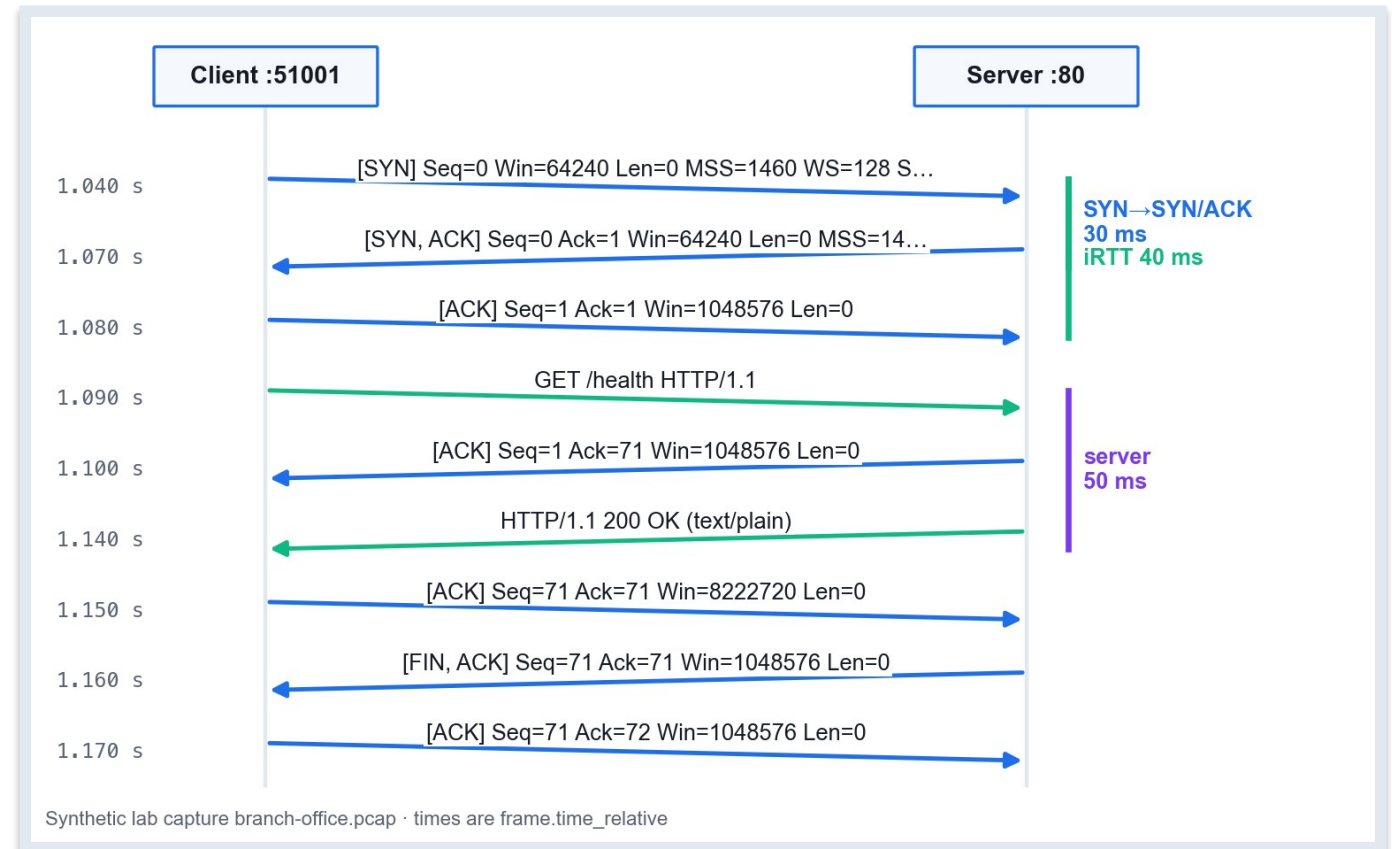
- Application: HTTP, DNS, SMTP, SNMP
- ...
- Transport: TCP (reliable) and UDP (best-effort).
- Network: IP, ICMP, ARP sits between network and link.



Where common protocols sit

One Web Request, Step by Step

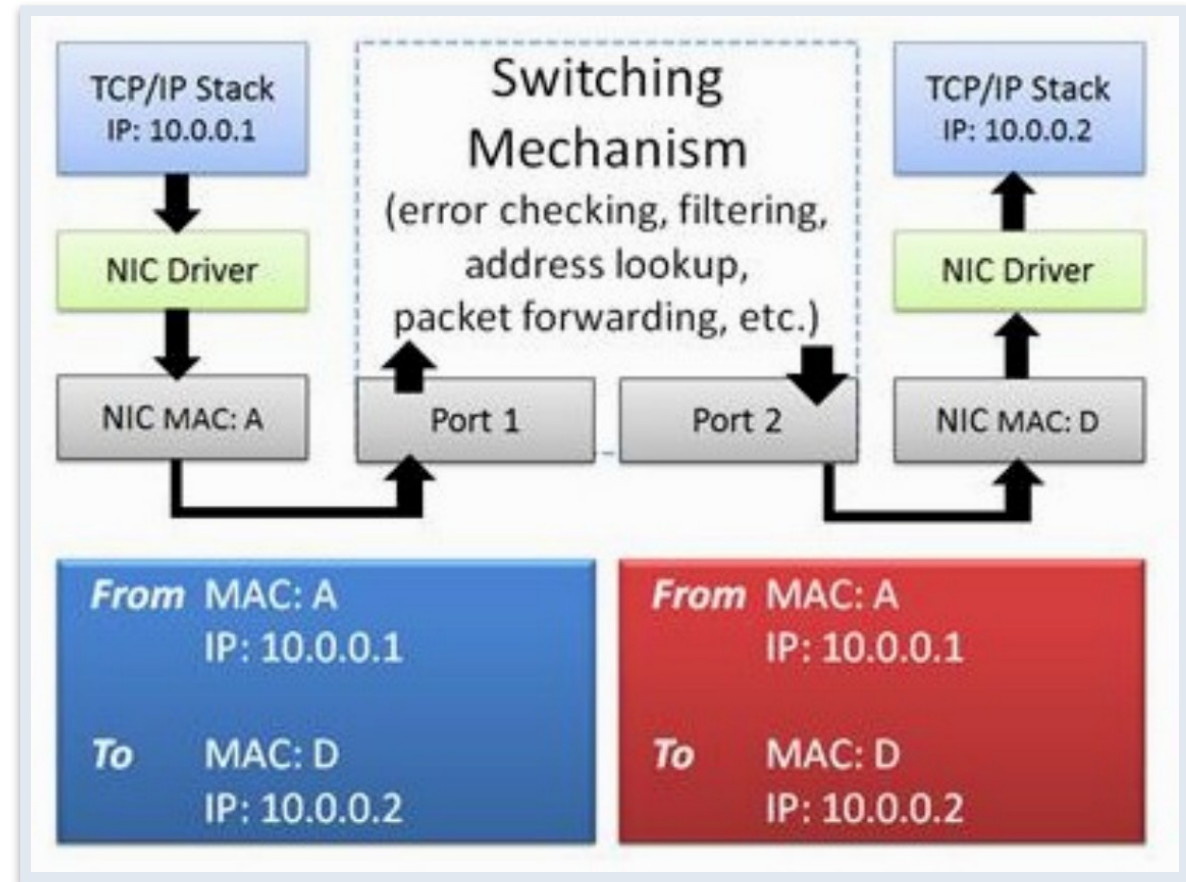
- Three-way handshake: SYN, SYN/ACK, ACK.
- GET /health, server ACK, 200 OK, client ACK.
- FIN/ACK closes the connection.
- Every arrow is a frame you can click in the lab.



TCP stream 0 from the lab capture

Switching Overview

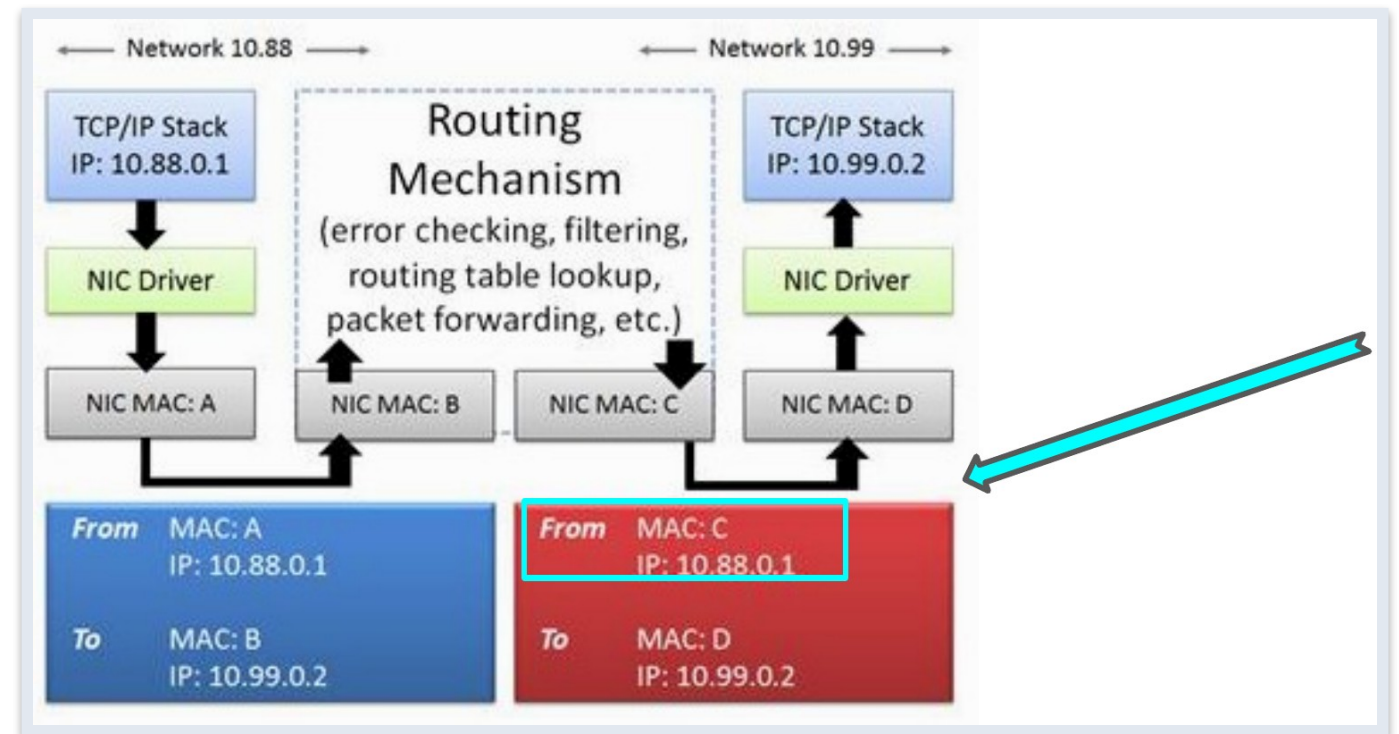
- Switches learn MAC addresses and forward frames within a broadcast domain.
- Source and destination MAC stay the same across a switch.
- ARP resolves the next-hop MAC before the first frame is sent.



A frame crossing a switch keeps its MAC addresses

Routing Overview

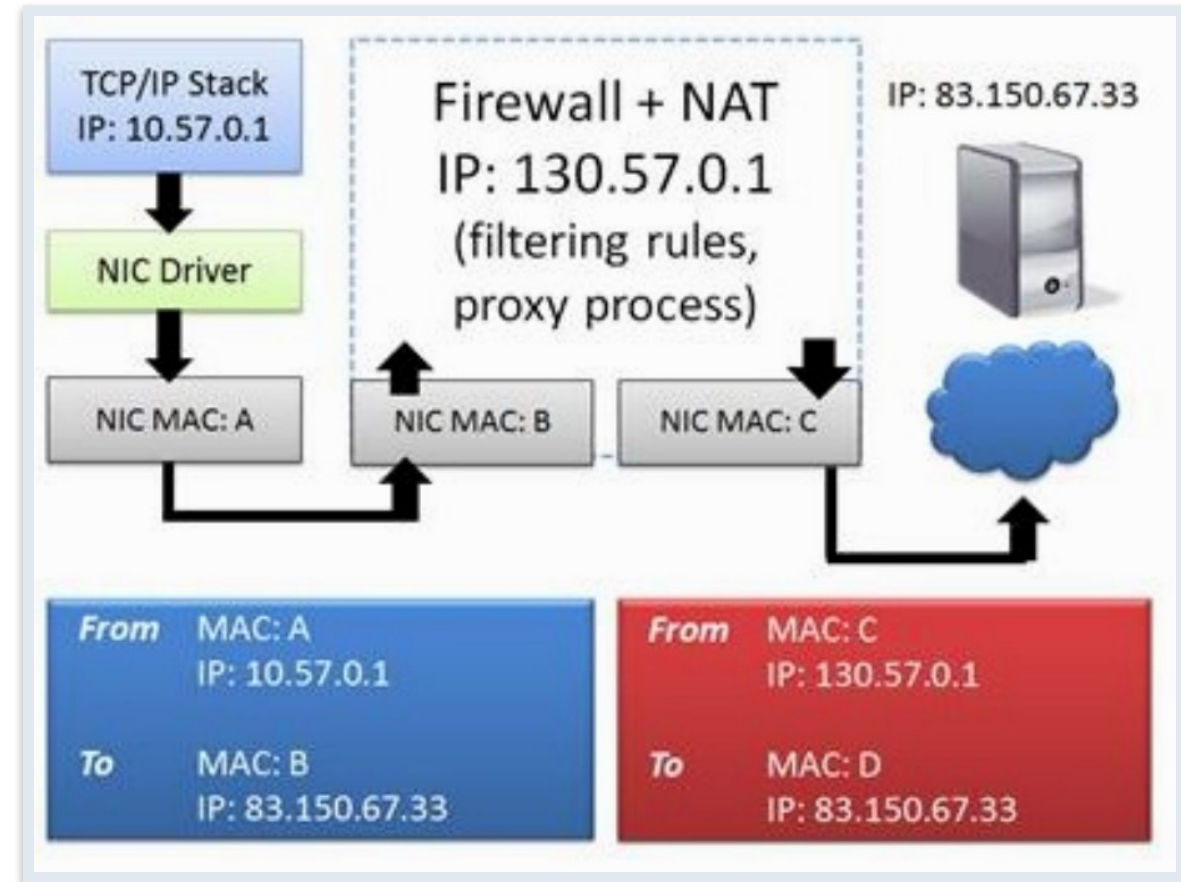
- Routers forward on destination IP and rewrite the link-layer addresses.
- TTL decreases by one at every router hop.
- The IP addresses are unchanged across routers (without NAT).



MAC addresses change hop-by-hop; IP addresses stay end-to-end

Firewalls and NAT

- NAT/PAT rewrites addresses and ports — capture on both sides to correlate.
- Firewalls may drop silently or reply with RST / ICMP unreachable.
- A proxy creates two separate TCP connections.



Address translation changes what each capture point sees

NAT Changes What Each Capture Point Sees



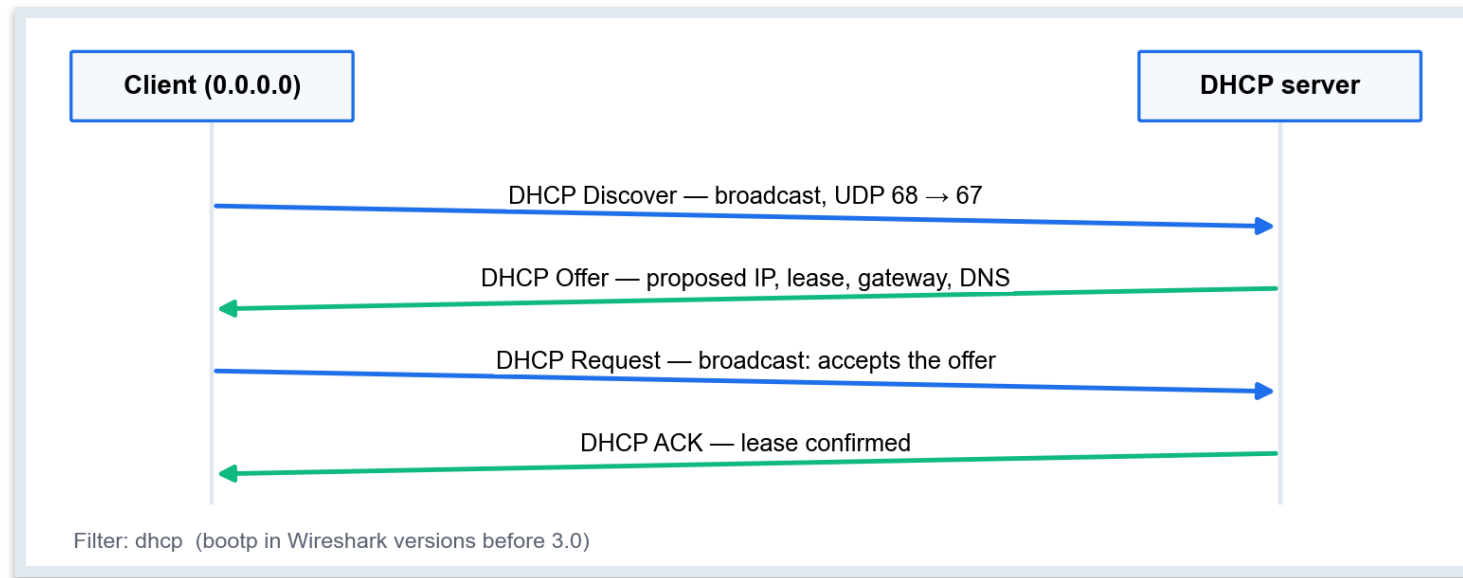
Translation table: 10.0.0.5:51001 ↔ 203.0.113.9:62001

Capture on BOTH sides and match on timing, sequence numbers or payload to correlate the flows

Inside and outside captures show different addresses and ports for the same flow

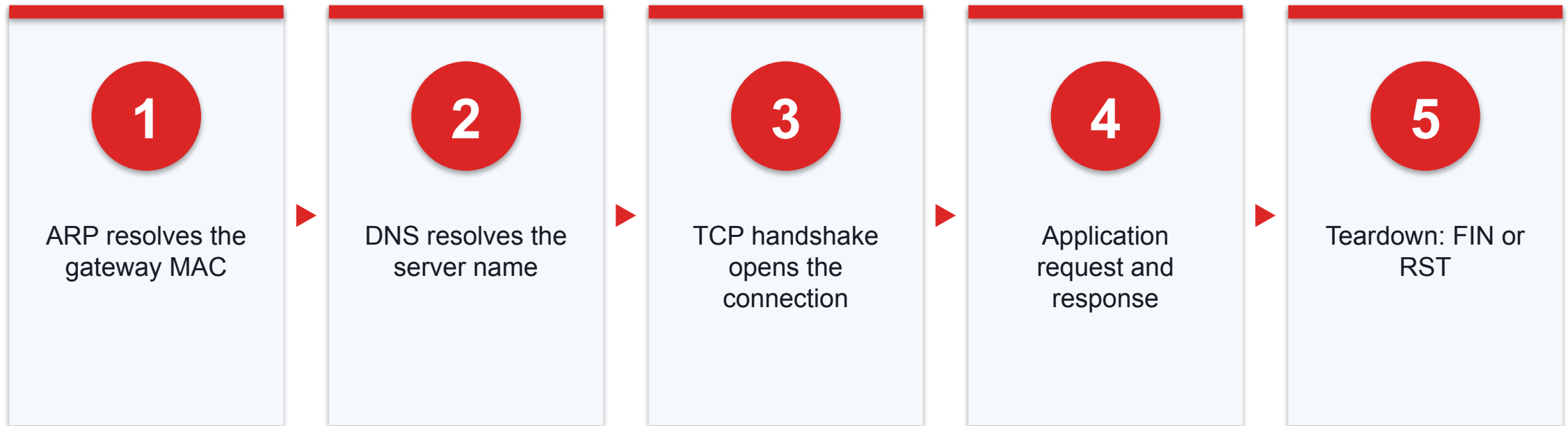
DHCP — How a Host Gets Its Address

- Discover and Request are broadcasts from 0.0.0.0 (UDP 68 → 67).
- Offer and ACK carry the address, lease, gateway and DNS server.
- No DHCP ACK means no address — check for Offers first.



Discover, Offer, Request, ACK

The Dependency Chain



Reconstruct an application dependency chain

LAB 8

A user says "the portal is down". Before blaming the web server, you rebuild every dependency the browser needed: address resolution, name resolution, connection and request.

You'll build

A completed dependency map with frame ranges

Tools: Wireshark, TShark, Python

Lab 8 — What You'll Do

1

Locate the ARP
exchange and DNS
lookup

2

Draw the TCP/HTTP
exchange in Flow Graph

3

Fill in the dependency
map

4

Contrast a local and a
routed next hop

Full step-by-step instructions: [Learner Guide · labs/lab-08-reconstruct-an-application-dependency-chain/LAB-08-Instructions.pdf](#)

Expected Evidence — Lab 8

dns.qry.name == "portal.example.test"					
No.	Time	Source	Destination	Protocol	Info
3	0.020000	192.0.2.10	192.0.2.53	DNS	Standard query 0x0065 A portal.example.test
4	0.040000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0065 A portal.example.t...
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Reconstruct an application dependency chain

Test it

The supplied `dns.qry.name == "portal.example.test"` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — TCP/IP Communications and Resolution

1 You can now: navigate, filter, summarize and time network exchanges.

2 ARP resolves a local next-hop MAC; DNS resolves a name to an address.

3 The destination IP stays end-to-end across routing while link-layer addresses change per hop.

4 Resolution, connection establishment and application exchange form a dependency chain.

Tea Break

15 minutes

TOPIC 09

DNS Traffic Analysis

Diagnose DNS failure and delay

Key Concepts — DNS Traffic Analysis

1

Transaction ID plus addresses and ports pair a query with its response.

2

NXDOMAIN reports that a name does not exist; it differs from silence or timeout.

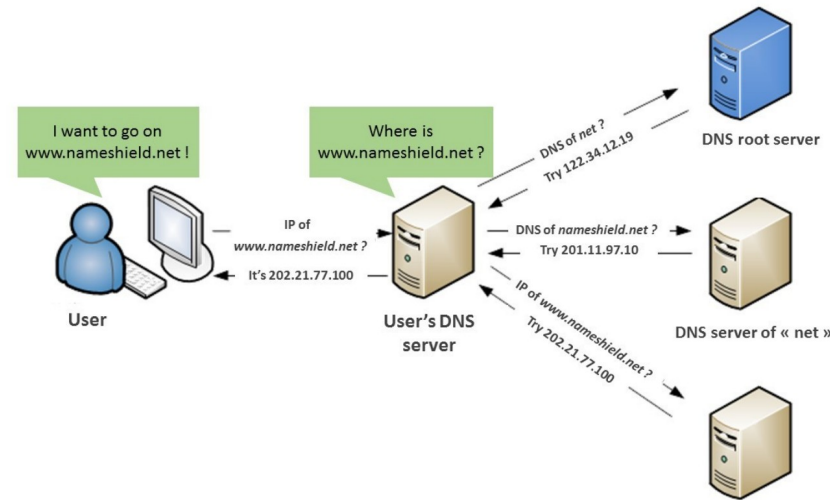
3

DNS response time is a measured exchange, influenced by resolver and path behaviour.

How DNS Works

- The client asks its resolver; the resolver walks the hierarchy or answers from cache.
- Queries normally use UDP 53; large answers and zone transfers use TCP.
- Transaction ID pairs each response with its query.

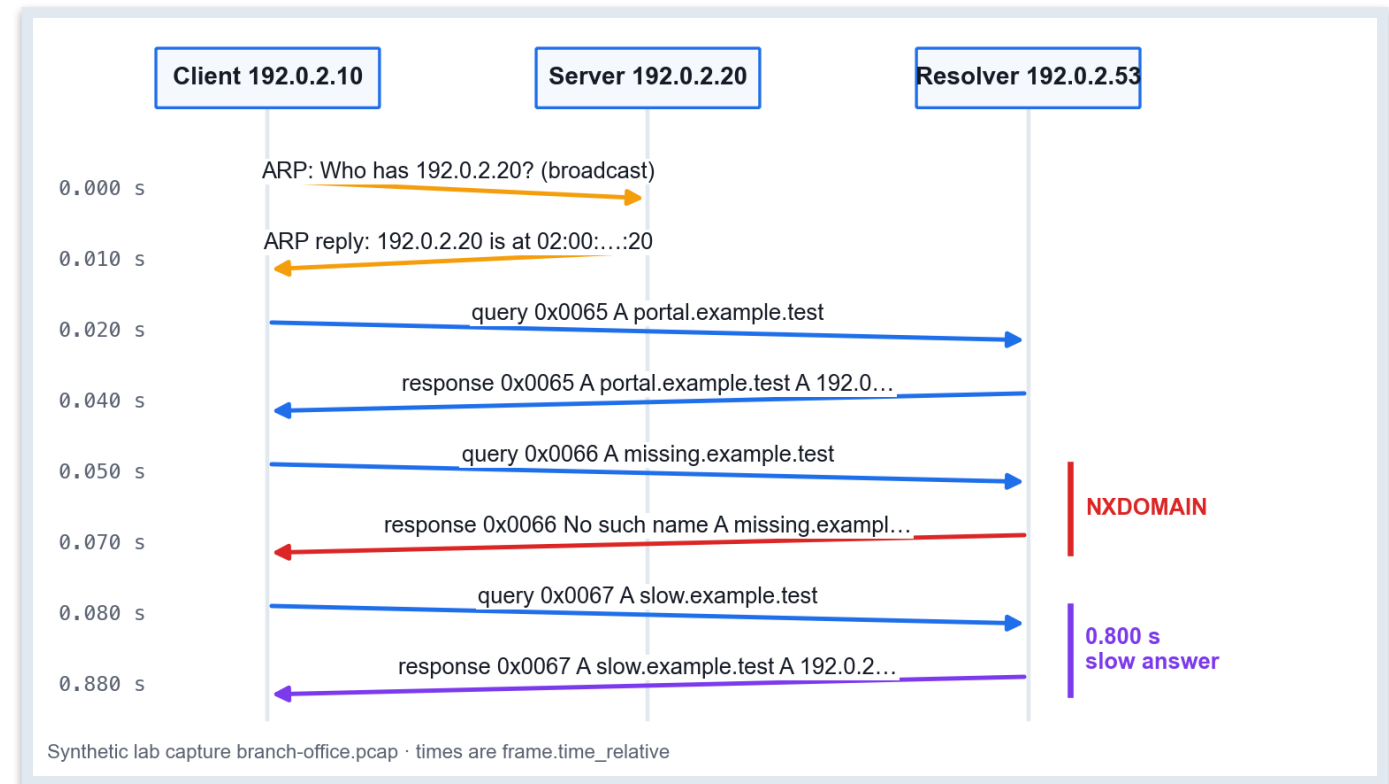
Domain Name System (DNS) provides host name-to-IP address resolution services.



Name resolution through a local resolver

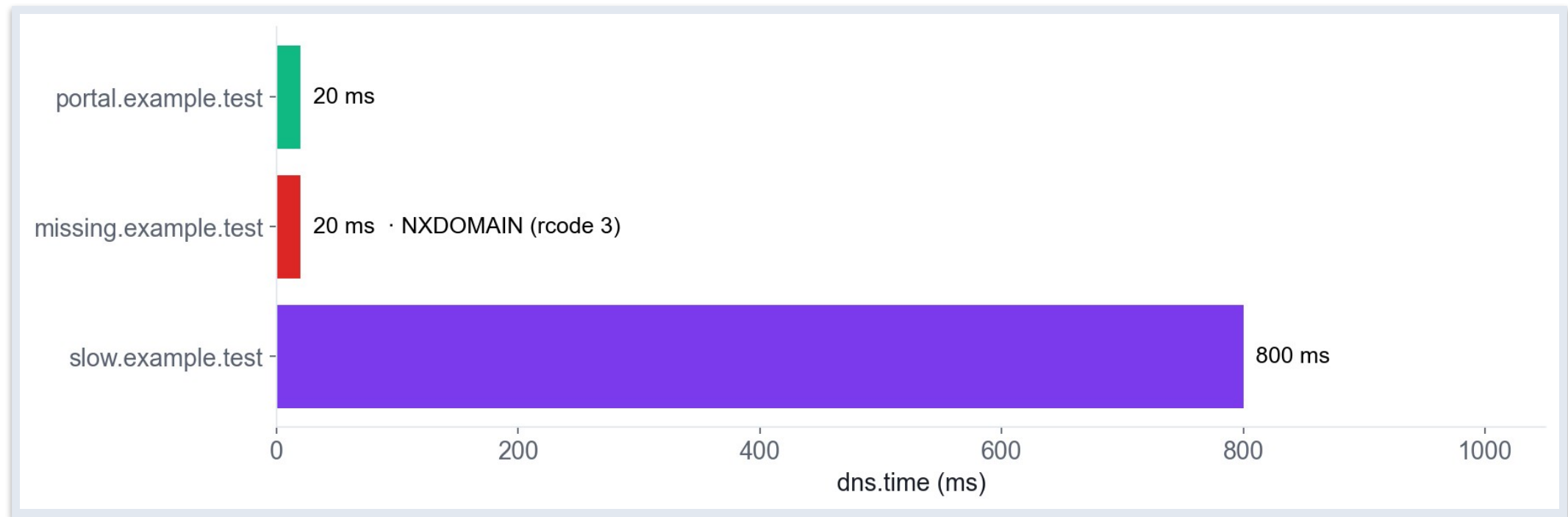
DNS in the Lab Capture

- ARP first, then three DNS transactions to the resolver.
- 0x0066 returns NXDOMAIN in 20 ms — a fast, definite “no”.
- 0x0067 takes 800 ms — slow, but successful.



Frames 1–8 of the lab capture

Measuring DNS Response Time



dns.time on each response: a failing name and a slow name are different problems

DNS Record Types You Will See

1

A / AAAA

IPv4 / IPv6 address

2

CNAME

Alias to another name

3

MX

Mail server

4

NS

Authoritative name server

5

PTR

Reverse lookup (address → name)

6

TXT

Text, e.g. SPF or verification

Reading DNS Evidence

Success

- rcode 0 (NoError)
- Answer records present
- Check TTL and address

Failure

- rcode 3 = NXDOMAIN
- rcode 2 = SERVFAIL
- Name or zone problem

Silence

- Query with no response
- Retries after timeout
- Path, firewall or resolver

Useful DNS Filters

1 `dns.flags.response == 0` — queries only.

2 `dns.flags.rcode == 3` — name does not exist.

3 `dns.time > 0.5` — responses slower than 500 ms.

4 `dns.qry.name contains "example"` — one domain family.

Diagnose DNS failure and delay

LAB 9

Users report that some names fail and others resolve slowly. You separate a missing name from a slow resolver using the DNS evidence.

You'll build

A DNS observations sheet with measured intervals

Tools: Wireshark, TShark, Python

Lab 9 — What You'll Do

1

Pair DNS queries and responses by ID

2

Explain the NXDOMAIN response

3

Measure the 0.8 s slow transaction

4

Record observations and next steps

Full step-by-step instructions: [Learner Guide · labs/lab-09-diagnose-dns-failure-and-delay/LAB-09-Instructions.pdf](#)

Expected Evidence — Lab 9

dns.flags.rcode == 3					
No.	Time	Source	Destination	Protocol	Info
6	0.070000	192.0.2.53	192.0.2.10	DNS	Standard query response 0x0066 No such name A mis...
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Diagnose DNS failure and delay

Test it

The supplied `dns.flags.rcode == 3` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — DNS Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

Transaction ID plus addresses and ports pair a query with its response.

3

NXDOMAIN reports that a name does not exist; it differs from silence or timeout.

4

DNS response time is a measured exchange, influenced by resolver and path behaviour.

End of Day 2

See you tomorrow

DAY 3

IP and transport diagnostics

Recap of the previous day, then today's topics and labs

TOPIC 10

ARP Traffic Analysis

Verify link-local resolution

Key Concepts — ARP Traffic Analysis

1

ARP requests are link-local broadcasts; replies usually return to the requester.

2

A repeated unresolved request suggests a local resolution problem, but a single trace may miss the reply.

3

MAC changes require corroboration before claiming duplicate IP or spoofing.

ARP in Practice

Request

- Broadcast to ff:ff:ff:ff:ff:ff
- "Who has 192.0.2.20? Tell 192.0.2.10"
- arp.opcode == 1

Reply

- Unicast back to the requester
- "192.0.2.20 is at 02:00:..."
- arp.opcode == 2

Gratuitous

- Sender announces its own IP
- Used after failover / IP change
- arp.isgratuitous

The Ethernet II Frame

Fields

- Destination MAC (6 B)
- Source MAC (6 B)
- EtherType (2 B)
- Payload, then FCS (4 B, rarely captured)

Values to recognise

- ff:ff:ff:ff:ff:ff — broadcast
- 01:00:5e:... — IPv4 multicast
- 0x0800 IPv4 · 0x0806 ARP
- 0x86DD IPv6 · 0x8100 VLAN tag

ARP Problems to Recognise

Symptoms

- Repeated requests with no reply
- Duplicate IP address detected warnings
- One IP mapped to two MAC addresses
- Excessive ARP broadcasts

Possible causes

- Host down or wrong subnet
- Two hosts configured with the same IP
- ARP spoofing — corroborate first
- Scanning or misconfigured devices

Spotting ARP Spoofing — Carefully

1 `arp.duplicate-address-detected` flags one IP seen with two MACs.

2 A flood of gratuitous replies (`arp.isgratuitous`) is suspicious.

3 Legitimate causes exist: failover, VRRP, a replaced NIC.

4 Corroborate with switch CAM tables before calling it an attack.

Verify link-local resolution

LAB 10

A technician suspects an ARP problem on the branch LAN. You check what the capture actually proves before anyone raises a spoofing alert.

You'll build

A verified ARP table entry and an evidence-based conclusion

Tools: Wireshark, TShark, Python

Lab 10 — What You'll Do

1

Record ARP request
and reply fields

2

Confirm the server IP-to-
MAC mapping

3

Verify the opcode in the
packet bytes

4

Test the hypotheses
against the evidence

Full step-by-step instructions: [Learner Guide · labs/lab-10-verify-link-local-resolution/LAB-10-Instructions.pdf](#)

Expected Evidence — Lab 10

arp.opcode == 2					
No.	Time	Source	Destination	Protocol	Info
2	0.010000	02:00:00:00:00:20	02:00:00:00:00:10	ARP	192.0.2.20 is at 02:00:00:00:00:20
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Verify link-local resolution

Test it

The supplied `arp.opcode == 2` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — ARP Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

ARP requests are link-local broadcasts; replies usually return to the requester.

3

A repeated unresolved request suggests a local resolution problem, but a single trace may miss the reply.

4

MAC changes require corroboration before claiming duplicate IP or spoofing.

Tea Break

15 minutes

TOPIC 11

IPv4 Traffic Analysis

Classify IPv4 scope and headers

Key Concepts — IPv4 Traffic Analysis

1

TTL limits forwarding hops; it is not a latency measurement.

2

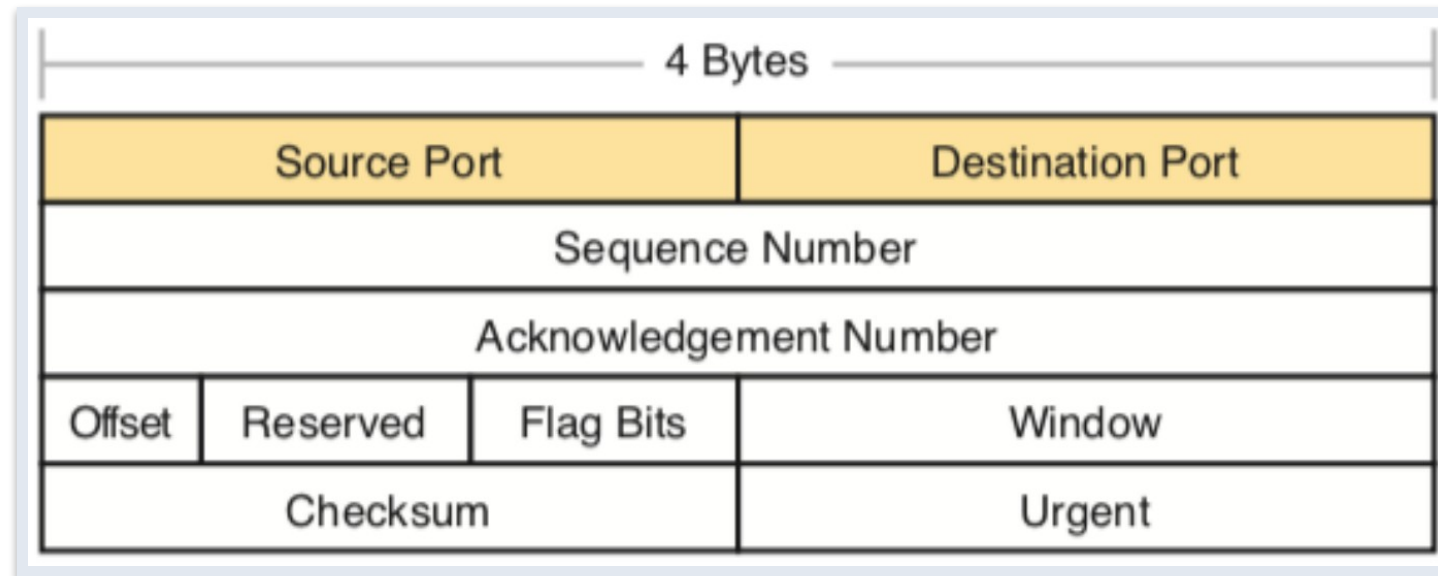
Broadcast and multicast use different addressing scopes and delivery rules.

3

Fragmentation fields describe packet handling; missing fragments can reflect capture limitations.

IPv4 Header Essentials

- TTL — hops remaining; not a measure of time.
- Protocol — 1 ICMP, 6 TCP, 17 UDP.
- Flags/Fragment offset — fragmentation handling.
- DSCP — quality-of-service marking.



Header layout (TCP header shown; IPv4 uses the same 32-bit rows)

Address Scope

Unicast

- One sender to one receiver
- Most application traffic
- `ip.dst == host`

Broadcast

- All hosts on the subnet
- `255.255.255.255` or directed
- Not forwarded by routers

Multicast

- `224.0.0.0/4` group addresses
- `224.0.0.1` = all hosts
- Joined with IGMP

IPv4 Filters and Checks

1 `ip.ttl < 10` — packets close to expiring.

2 `ip.flags.mf == 1 || ip.frag_offset > 0` — fragments.

3 `ip.dst == 224.0.0.0/4` — all multicast.

4 Missing fragments may be a capture limitation — check drops first.

IPv4 vs IPv6 at a Glance

IPv4

- 20-byte header (with options: more)
- TTL limits hops
- ARP resolves MAC addresses
- Routers may fragment
- Filter: ip, icmp

IPv6

- Fixed 40-byte header + extension headers
- Hop Limit replaces TTL
- Neighbor Discovery (ICMPv6 135/136)
- Only the source fragments
- Filter: ipv6, icmpv6

Classify IPv4 scope and headers

LAB 11

The security team asks whether any unusual IPv4 traffic — multicast, broadcast or fragments — appears in the branch capture.

You'll build

A completed IPv4 header and scope sheet

Tools: Wireshark, TShark, Python

Lab 11 — What You'll Do

1

Record IPv4 header
fields of an echo

2

Compare multicast MAC
and IP destination

3

Check for fragmented
packets

4

Classify scope in the IP
header sheet

Full step-by-step instructions: [Learner Guide · labs/lab-11-classify-ipv4-scope-and-headers/LAB-11-Instructions.pdf](#)

Expected Evidence — Lab 11

ip.dst == 224.0.0.1					
No.	Time	Source	Destination	Protocol	Info
17	1.030000	192.0.2.10	224.0.0.1	UDP	55001 → 5000 Len=13
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Classify IPv4 scope and headers

Test it

The supplied `ip.dst == 224.0.0.1` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — IPv4 Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

TTL limits forwarding hops; it is not a latency measurement.

3

Broadcast and multicast use different addressing scopes and delivery rules.

4

Fragmentation fields describe packet handling; missing fragments can reflect capture limitations.

Lunch Break

1 hour

TOPIC 12

ICMP Traffic Analysis

Interpret echo and unreachable messages

Key Concepts — ICMP Traffic Analysis

1

Echo requests/replies show an IP exchange; they do not guarantee an application works.

2

Destination unreachable messages quote the offending datagram.

3

ICMP may be filtered or rate-limited, so absence of a reply needs cautious interpretation.

ICMP Messages to Know

Echo

- Type 8 request, Type 0 reply
- ping / reachability
- Match on identifier + sequence

Unreachable

- Type 3
- Code 3 = port unreachable
- Code 1 = host unreachable

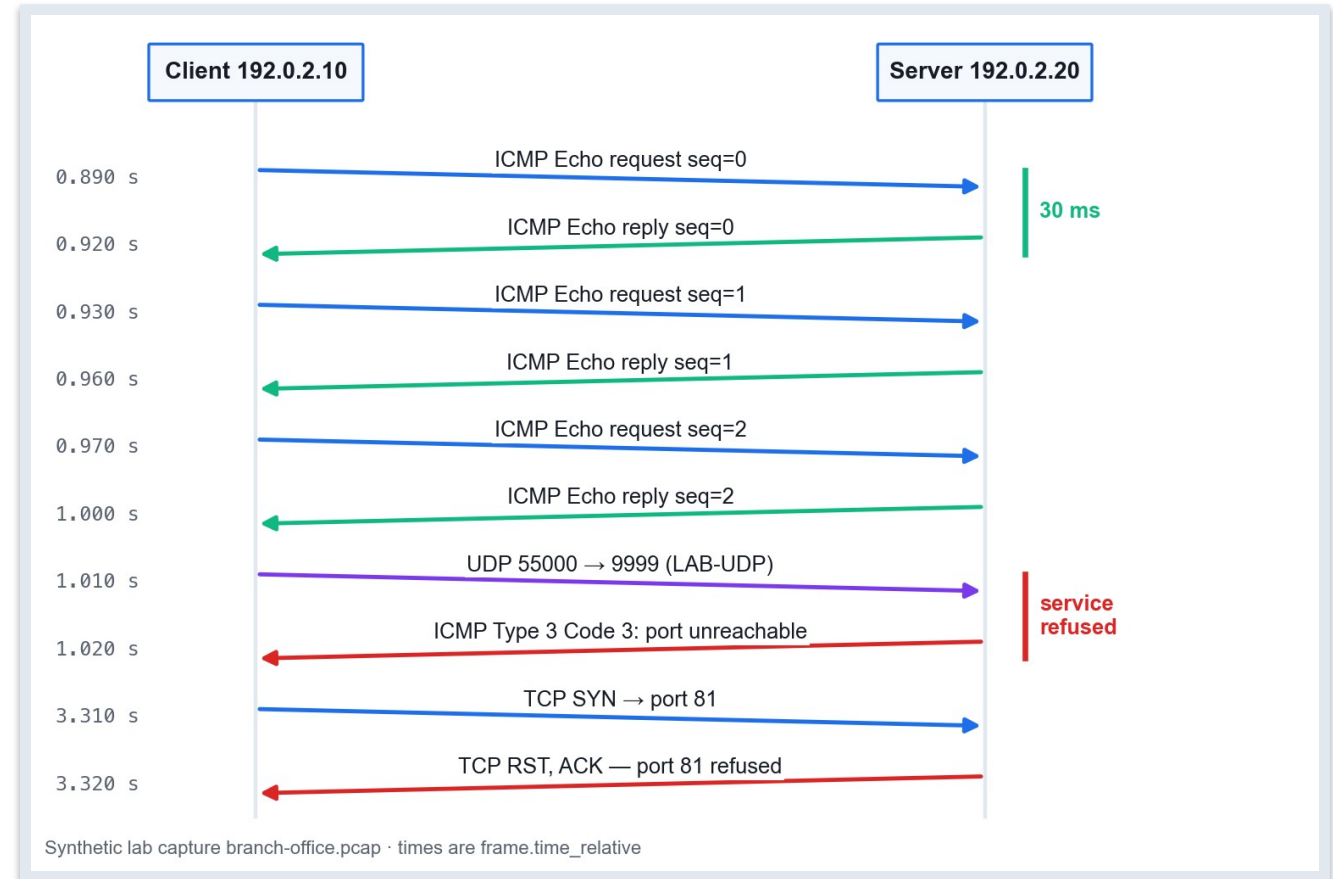
Time exceeded

- Type 11
- TTL expired in transit
- How traceroute works

ICMP EVIDENCE

Echo, Refusal and Reset in the Lab Capture

- Three echo pairs, 30 ms each — the host is reachable.
- UDP to port 9999 → ICMP Type 3 Code 3: nothing listening.
- TCP SYN to port 81 → RST: the TCP equivalent of “closed”.



Frames 9–16 and 55–56

Traceroute and TTL Exceeded

1 Traceroute sends probes with TTL 1, 2, 3 ...

2 Each router that drops a probe returns ICMP Type 11 (Time Exceeded).

3 Filter `icmp.type == 11` to list the hops.

4 Windows uses ICMP probes; Linux/macOS use UDP by default.

Interpreting ICMP Evidence

1

An unreachable message quotes the original IP header — read it to find the failed flow.

2

No echo reply can mean filtering, not a down host.

3

Rate-limited ICMP can hide real loss.

4

`icmp.type == 3 && icmp.code == 3` finds closed UDP ports.

Interpret echo and unreachable messages

LAB 12

Ping to the server works, yet a diagnostic tool on UDP port 9999 fails. You use ICMP evidence to explain the difference.

You'll build

An ICMP event log explaining the service refusal

Tools: Wireshark, TShark, Python

Lab 12 — What You'll Do

1

Pair echo requests and
replies

2

Read the quoted
headers in the
unreachable

3

Find the original UDP
datagram

4

Record the events and
your conclusion

Full step-by-step instructions: [Learner Guide · labs/lab-12-interpret-echo-and-unreachable-messages/LAB-12-Instructions.pdf](#)

Expected Evidence — Lab 12

icmp.type == 3 && icmp.code == 3					
No.	Time	Source	Destination	Protocol	Info
16	1.020000	192.0.2.20	192.0.2.10	ICMP	Destination unreachable (Port unreachable)
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Interpret echo and unreachable messages

Test it

The supplied `icmp.type == 3 && icmp.code == 3` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — ICMP Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

Echo requests/replies show an IP exchange; they do not guarantee an application works.

3

Destination unreachable messages quote the offending datagram.

4

ICMP may be filtered or rate-limited, so absence of a reply needs cautious interpretation.

TOPIC 13

UDP Traffic Analysis

Follow datagrams and service refusal

Key Concepts — UDP Traffic Analysis

1

UDP has no transport handshake or retransmission; the application may provide reliability.

2

A port number is a decoding hint; payload and context identify the application.

3

Following a UDP stream collects datagrams and does not invent missing data.

TCP vs UDP

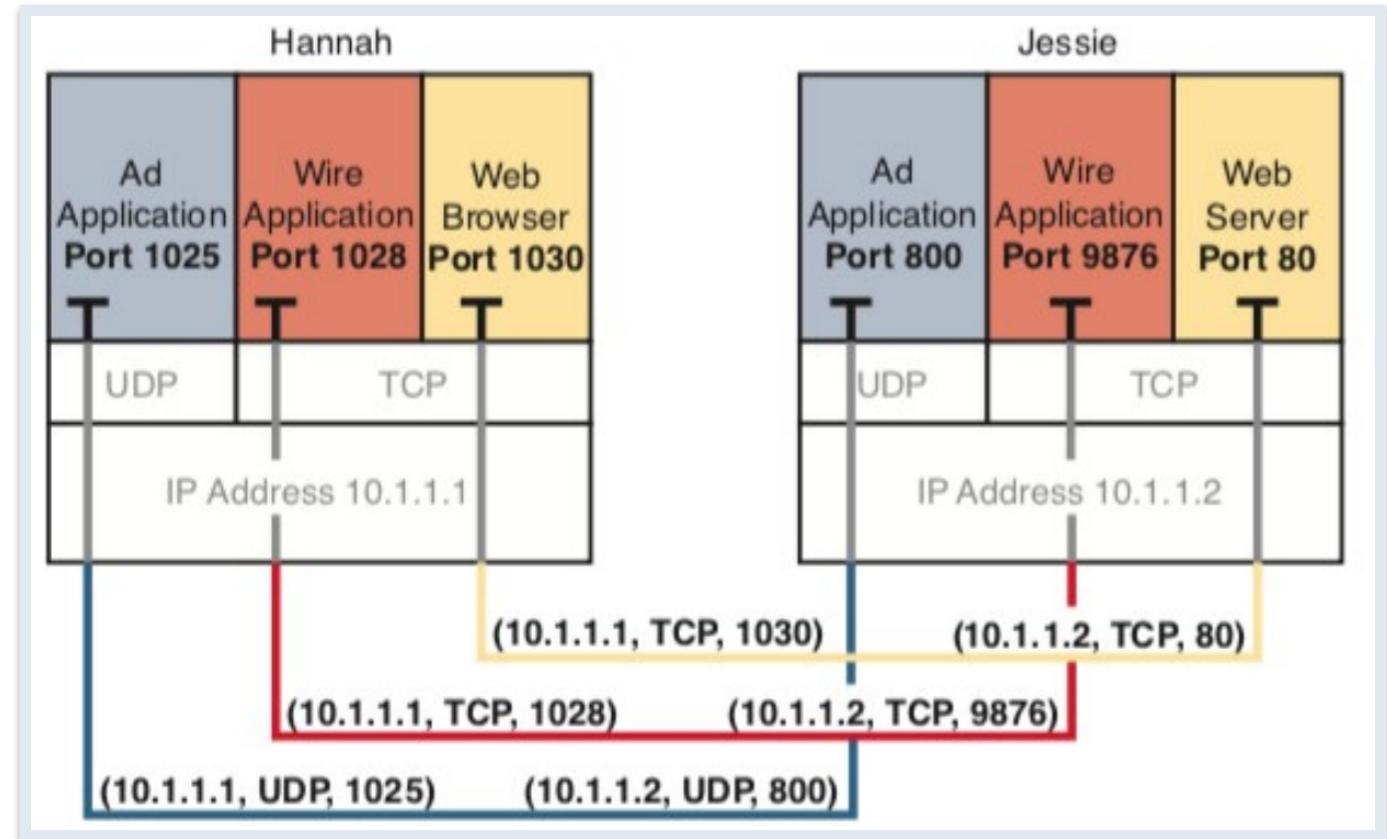
- UDP: connectionless, 8-byte header, no retransmission or ordering.
- TCP: connection-oriented, reliable, ordered, flow-controlled.
- Choose the analysis by the transport: UDP problems show up at the application.

	TCP	UDP																								
Acronym	Transmission Control Protocol	User Datagram Protocol																								
Reliability	High	Best Effort																								
Header Size	4 Bytes <table><tr><td colspan="2">Source Port</td><td colspan="2">Destination Port</td></tr><tr><td colspan="4">Sequence Number</td></tr><tr><td colspan="4">Acknowledgement Number</td></tr><tr><td>Offset</td><td>Reserved</td><td>Flag Bits</td><td>Window</td></tr><tr><td colspan="2">Checksum</td><td colspan="2">Urgent</td></tr></table> Figure 5-1 TCP Header Fields 4x5=20 Byte	Source Port		Destination Port		Sequence Number				Acknowledgement Number				Offset	Reserved	Flag Bits	Window	Checksum		Urgent		4 Bytes <table><tr><td>Source Port</td><td>Destination Port</td></tr><tr><td>Length</td><td>Checksum</td></tr></table> Figure 5-10 UDP Header 4x2=8 Byte	Source Port	Destination Port	Length	Checksum
Source Port		Destination Port																								
Sequence Number																										
Acknowledgement Number																										
Offset	Reserved	Flag Bits	Window																							
Checksum		Urgent																								
Source Port	Destination Port																									
Length	Checksum																									

Header and feature comparison

Ports and Sockets

- A socket is IP address + port.
- A conversation is the pair of sockets — the 5-tuple with the protocol.
- Ephemeral client ports change per connection.



Socket pairs identify each conversation

Analysing UDP

1

Follow > UDP Stream reassembles a datagram conversation.

2

A UDP request answered by ICMP port unreachable = closed port.

3

Missing responses need application-level timing to interpret.

4

Decode As lets you dissect traffic on non-standard ports.

UDP Services You Will Meet

1

DNS · 53

Name resolution

2

DHCP · 67/68

Address assignment

3

NTP · 123

Time synchronisation

4

SNMP · 161/162

Device monitoring

5

Syslog · 514

Log forwarding

6

QUIC · 443

HTTP/3 transport

Follow datagrams and service refusal

LAB 13

The diagnostic application sends UDP and never hears back. You follow the datagrams to show what was sent and why there is no transport-level acknowledgement.

You'll build

A saved UDP stream and an explanation of the refusal

Tools: Wireshark, TShark, Python

Lab 13 — What You'll Do

1

Tell the datagram from
its ICMP quote

2

Follow the UDP stream
(LAB-UDP)

3

Decode the RTP stream
on port 4002

4

Explain the refusal
without UDP ACKs

Full step-by-step instructions: [Learner Guide · labs/lab-13-follow-datagrams-and-service-refusal/LAB-13-Instructions.pdf](#)

Expected Evidence — Lab 13

udp.dstport == 9999					
No.	Time	Source	Destination	Protocol	Info
15	1.010000	192.0.2.10	192.0.2.20	UDP	55000 → 9999 Len=7
16	1.020000	192.0.2.20	192.0.2.10	ICMP	Destination unreachable (Port unreachable)
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Follow datagrams and service refusal

Test it

The supplied `udp.dstport == 9999` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — UDP Traffic Analysis

1

You can now: interpret DNS, ARP, IPv4, ICMP and UDP evidence.

2

UDP has no transport handshake or retransmission; the application may provide reliability.

3

A port number is a decoding hint; payload and context identify the application.

4

Following a UDP stream collects datagrams and does not invent missing data.

Tea Break

15 minutes

TOPIC 14

TCP Protocol Analysis

Investigate retransmission and zero window

Key Concepts — TCP Protocol Analysis

1

Sequence numbers count bytes; ACK numbers indicate the next expected byte.

2

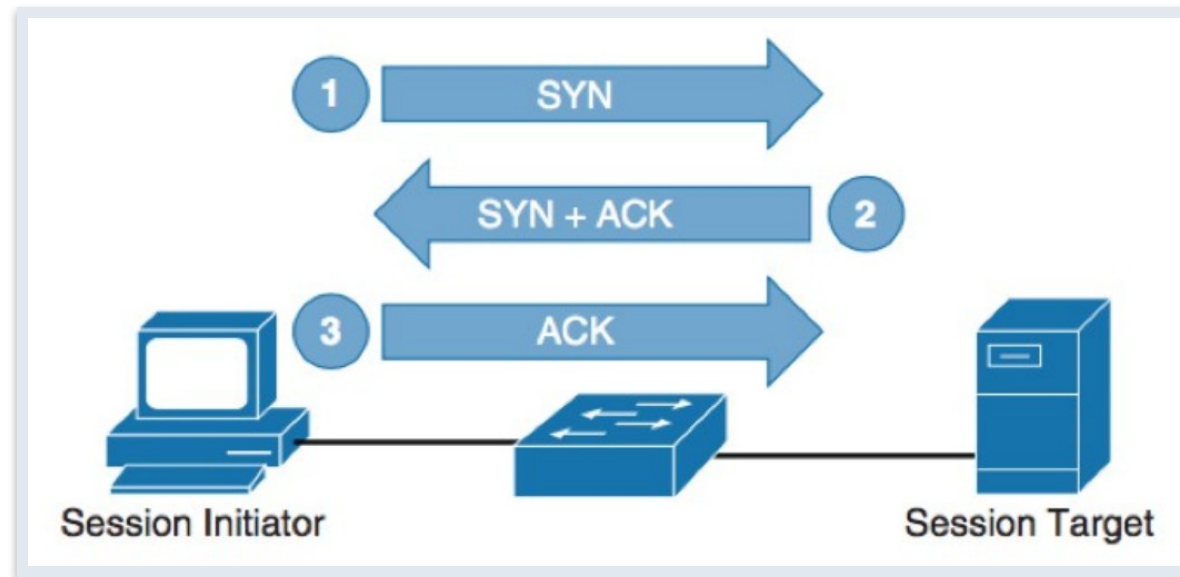
Retransmission and duplicate ACK labels are heuristics sensitive to capture placement and completeness.

3

Zero window indicates receive-side flow control; it is distinct from network congestion.

The Three-Way Handshake

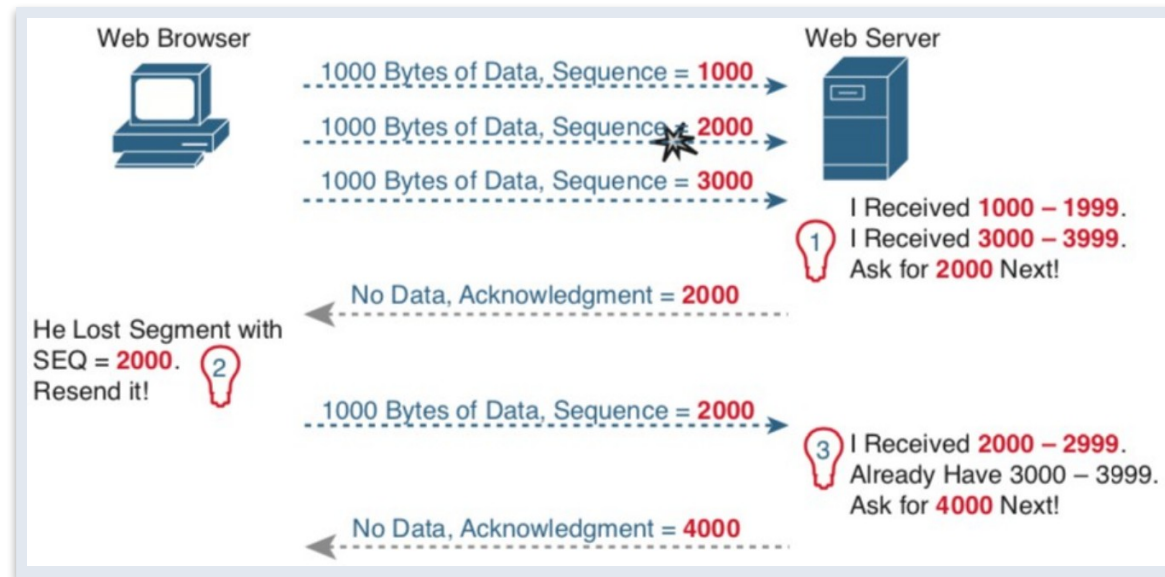
- SYN → SYN/ACK → ACK establishes the connection.
- Options are negotiated here: MSS, window scale, SACK permitted.
- The SYN → SYN/ACK gap is the initial RTT seen by the client.



Connection establishment

Sequence Numbers and Error Recovery

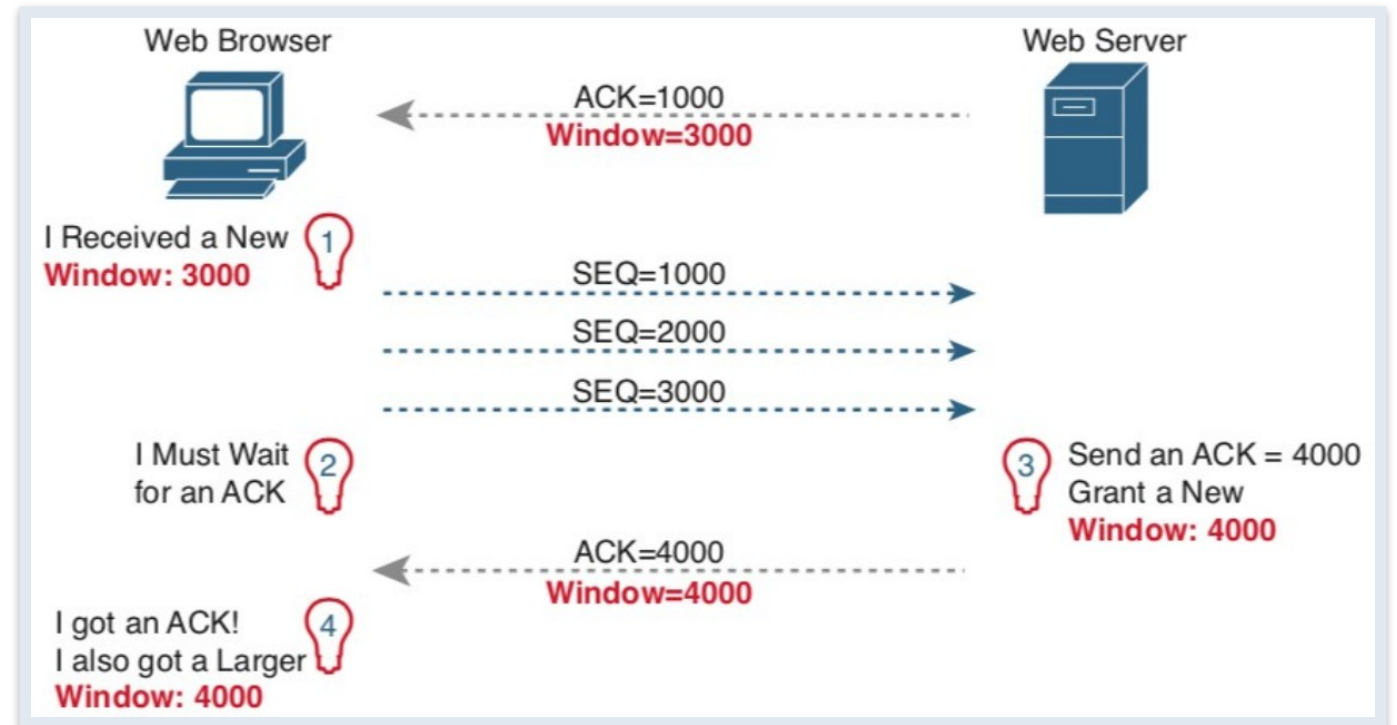
- Sequence numbers count bytes sent; ACK = next byte expected.
- A gap triggers duplicate ACKs and retransmission.
- Wireshark shows relative sequence numbers by default.



Lost segment recovered after the receiver asks again

TCP Windowing

- The receive window advertises buffer space available.
- Window scaling multiplies it for high-bandwidth paths.
- Window = 0 (Zero Window) stops the sender until a window update.

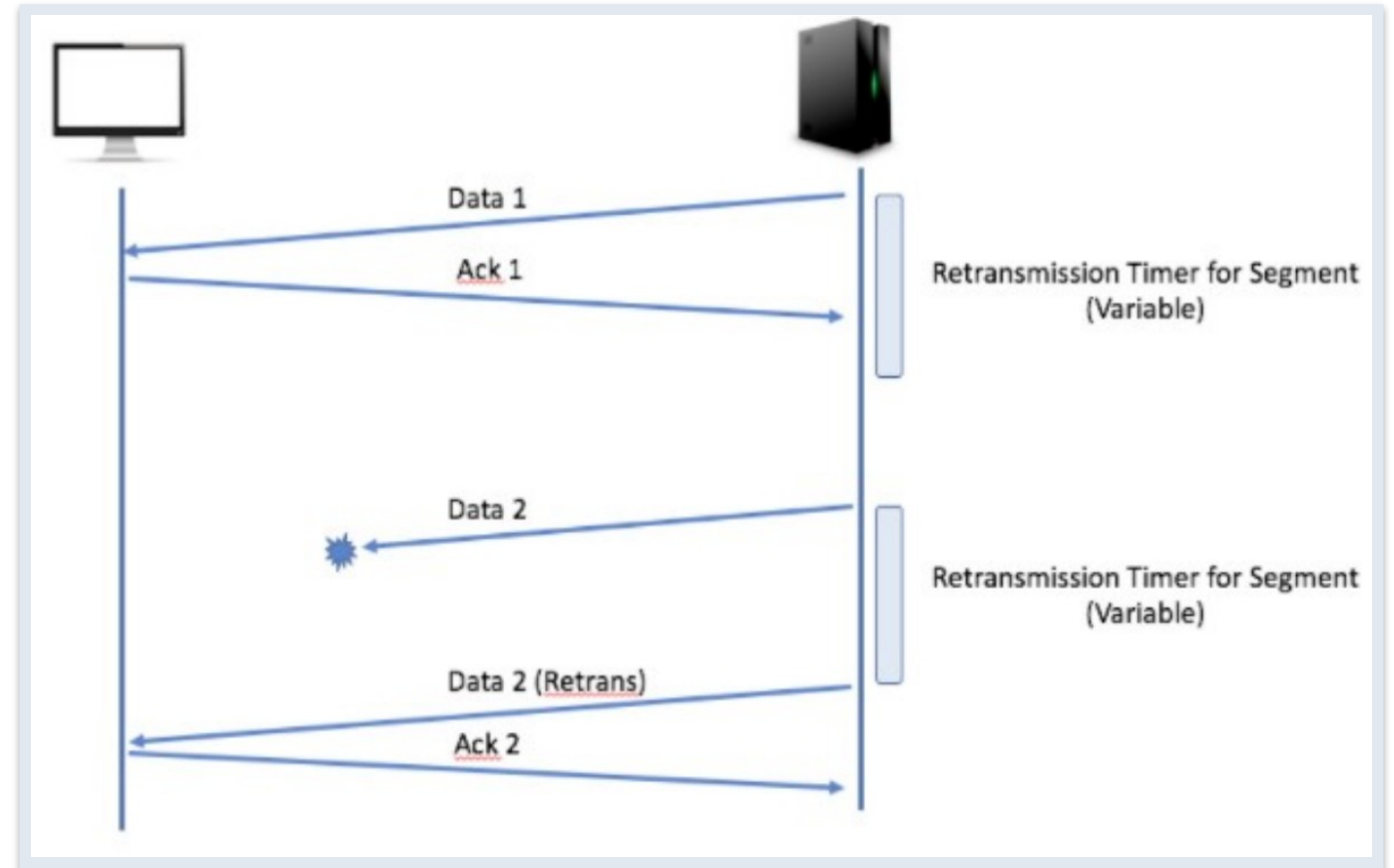


Flow control with window advertisements

LOSS

Retransmissions

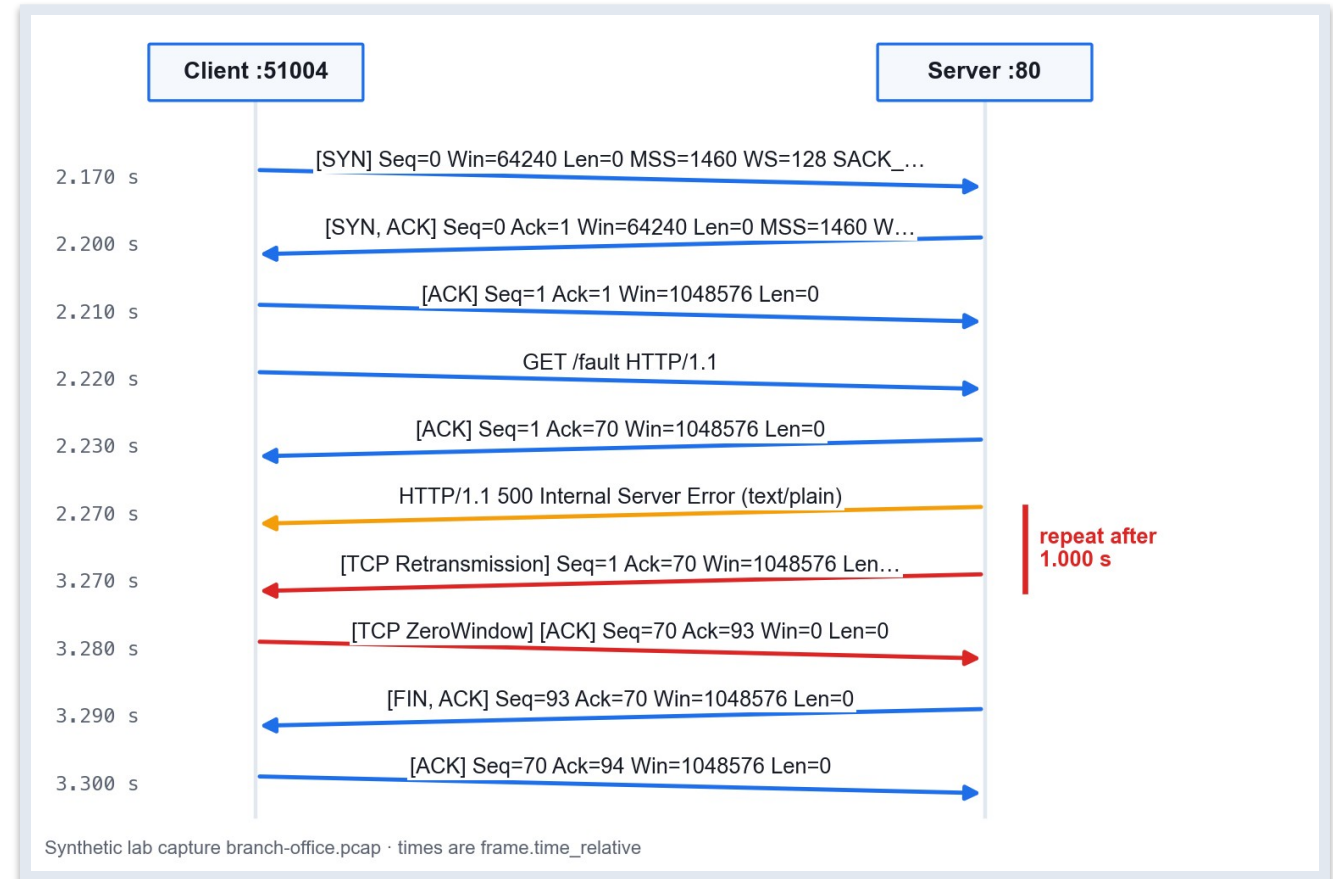
- Timeout retransmission (RTO): no ACK in time — backs off exponentially.
- Fast retransmission: three duplicate ACKs trigger an early resend.
- Spurious retransmissions repeat data that was already received.



Retransmission timers on a lossy path

Retransmission and Zero Window in Stream 3

- The server sends the HTTP 500 at 2.270 s.
- No ACK arrives — the same bytes are resent 1.000 s later.
- The client then advertises Win=0: its buffer is full.
- Finally FIN/ACK closes the stream.



TCP stream 3 from the lab capture

Closing a Connection

1 FIN/ACK from each side is an orderly close (four-way, often three packets).

2 RST aborts immediately — refused port, crash or firewall.

3 A RST straight after a SYN means nothing is listening on that port.

4 Long gaps before FIN usually mean idle timeouts.

TCP Expert Flags

1

Retransmission

Data sent again

2

Duplicate ACK

Receiver still waiting for a gap

3

Previous segment not captured

Gap — loss or capture drop

4

Zero Window

Receiver buffer full

5

Window Full

Sender filled the advertised window

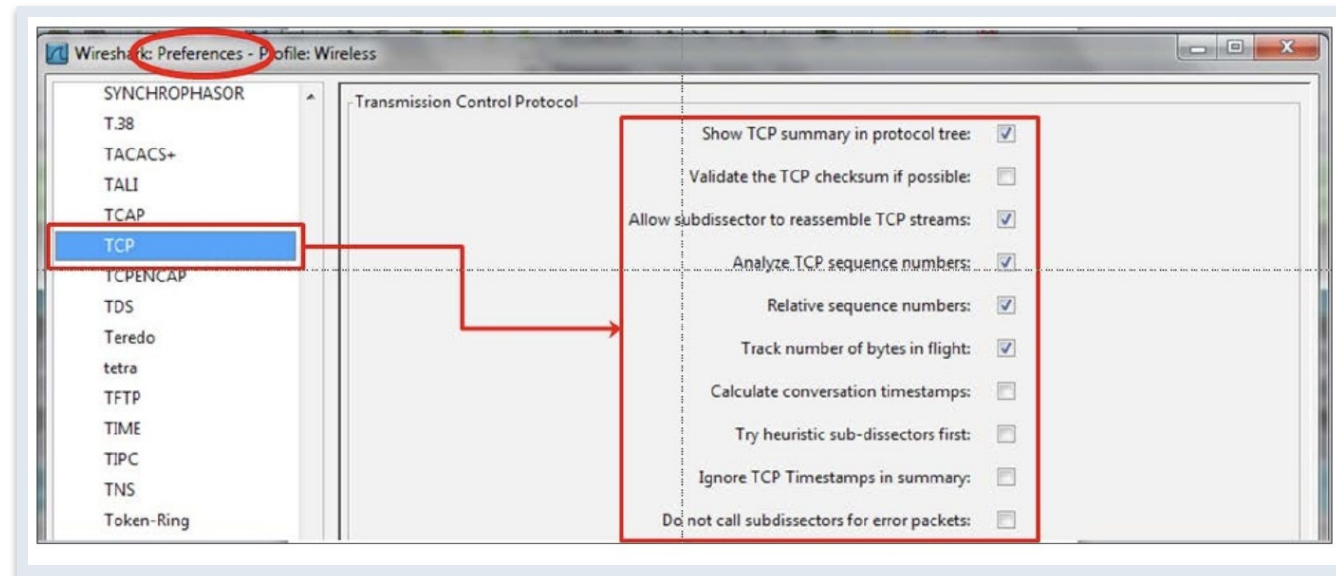
6

RST

Abrupt connection reset

TCP Preferences

- Analyze TCP sequence numbers — enables tcp.analysis.* flags.
- Calculate conversation timestamps — adds tcp.time_delta.
- Relative sequence numbers make streams readable.



Edit | Preferences | Protocols | TCP

Investigate retransmission and zero window

LAB 14

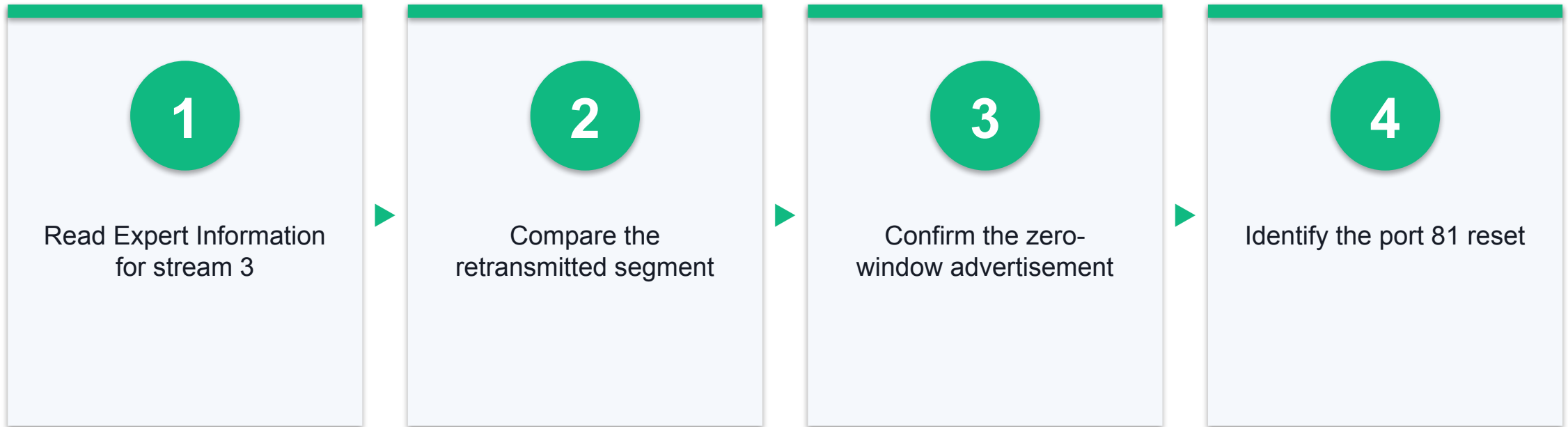
A file download stalls part-way through. You must show whether the sender, the receiver or the path held things up.

You'll build

A TCP evidence sheet for retransmission, zero window and reset

Tools: Wireshark, TShark, Python

Lab 14 — What You'll Do



Full step-by-step instructions: [Learner Guide · labs/lab-14-investigate-retransmission-and-zero-window/LAB-14-Instructions.pdf](#)

Expected Evidence — Lab 14

tcp.analysis.retransmission

No.	Time	Source	Destination	Protocol	Info
51	3.270000	192.0.2.20	192.0.2.10	TCP	[TCP Retransmission] 80 → 51004 [PSH, ACK] Seq=1 ...
branch-office.pcap Displayed: 1					

tcp.analysis.zero_window

No.	Time	Source	Destination	Protocol	Info
52	3.280000	192.0.2.10	192.0.2.20	TCP	[TCP ZeroWindow] 51004 → 80 [ACK] Seq=70 Ack=93 W...
branch-office.pcap Displayed: 1					

tcp.flags.reset == 1

No.	Time	Source	Destination	Protocol	Info
56	3.320000	192.0.2.20	192.0.2.10	TCP	81 → 52000 [RST, ACK] Seq=1 Ack=1 Win=8192 Len=0
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Investigate retransmission and zero window

Test it

The supplied `tcp.analysis.retransmission` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — TCP Protocol Analysis

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

Sequence numbers count bytes; ACK numbers indicate the next expected byte.

3

Retransmission and duplicate ACK labels are heuristics sensitive to capture placement and completeness.

4

Zero window indicates receive-side flow control; it is distinct from network congestion.

End of Day 3

See you tomorrow

DAY 4

Graphs, applications and reporting

Recap of the previous day, then today's topics and labs

TOPIC 15

Traffic Graphs

Graph bytes, RTT and recovery

Key Concepts — Traffic Graphs

1

An I/O graph bins observed events; units and interval determine what it means.

2

TCP time-sequence graphs show byte progress, stalls and repeated sequence ranges.

3

RTT samples, application response time and throughput answer different questions.

I/O Graphs

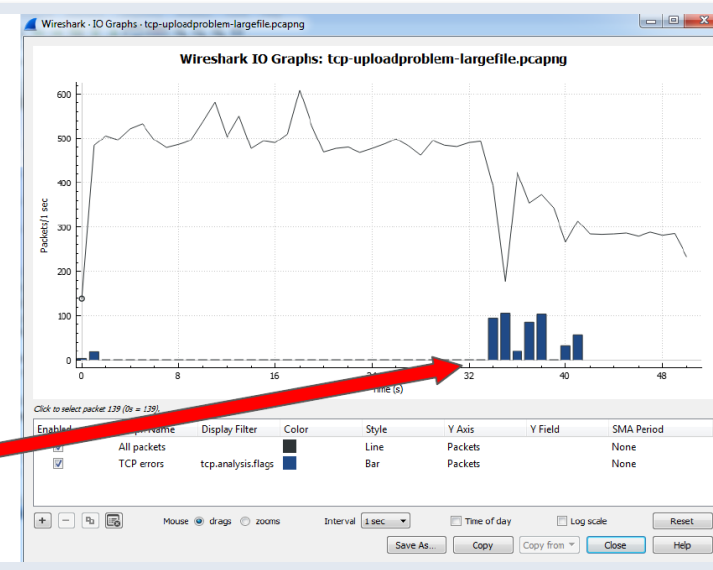
- Statistics > I/O Graphs plots packets, bytes or bits per interval.
- Add graphs with filters — e.g. total vs tcp.analysis.flags.
- Use log scale and moving averages to compare very different values.

SMA → Smooth Moving Average

Log Scale

Use when there is a relatively large difference between the two numerical values and the relationship can be difficult to see.

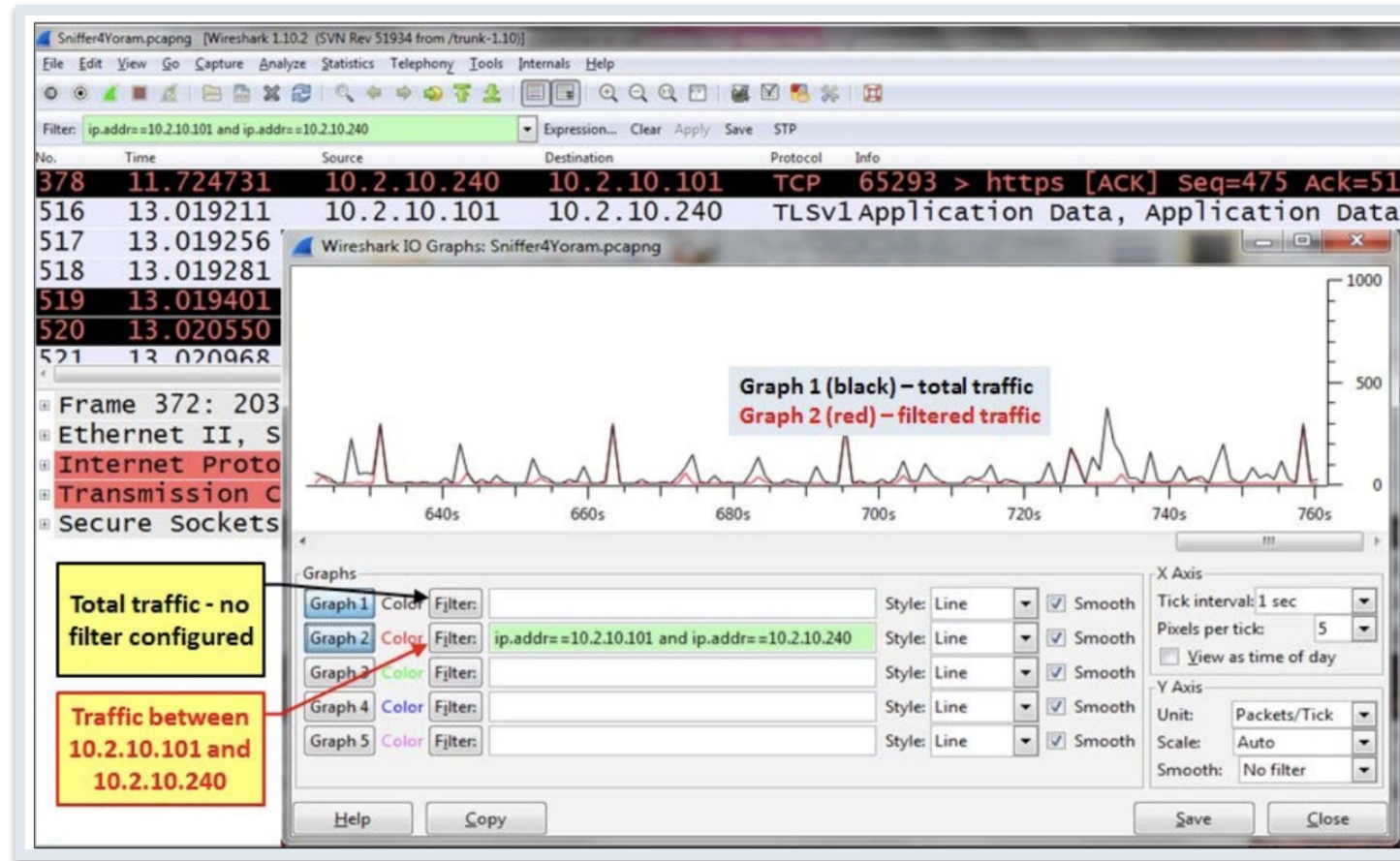
WindowZero, DupAck, etc.



Multiple graphs with different filters on one chart

GRAPHS

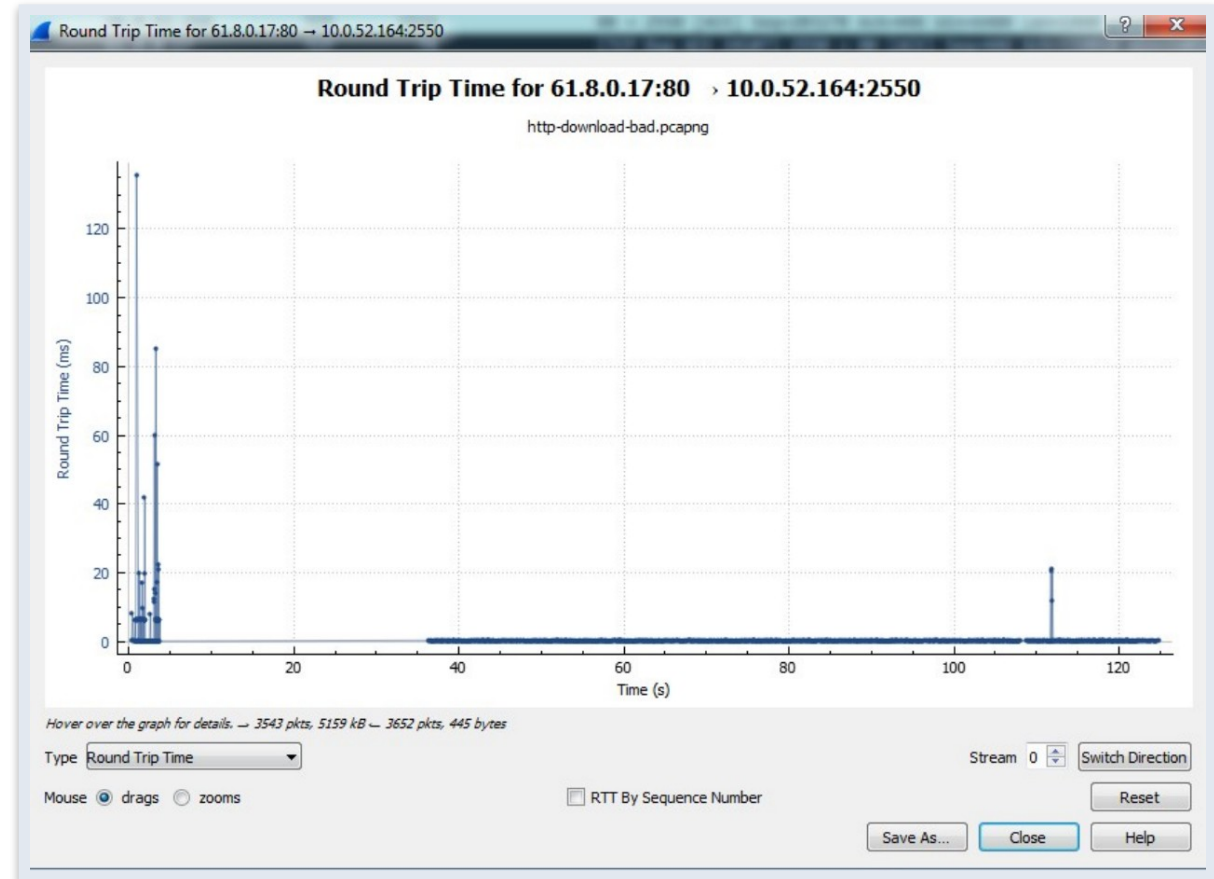
Reading an I/O Graph



Drops in throughput that line up with TCP expert events point to the cause

Round Trip Time Graph

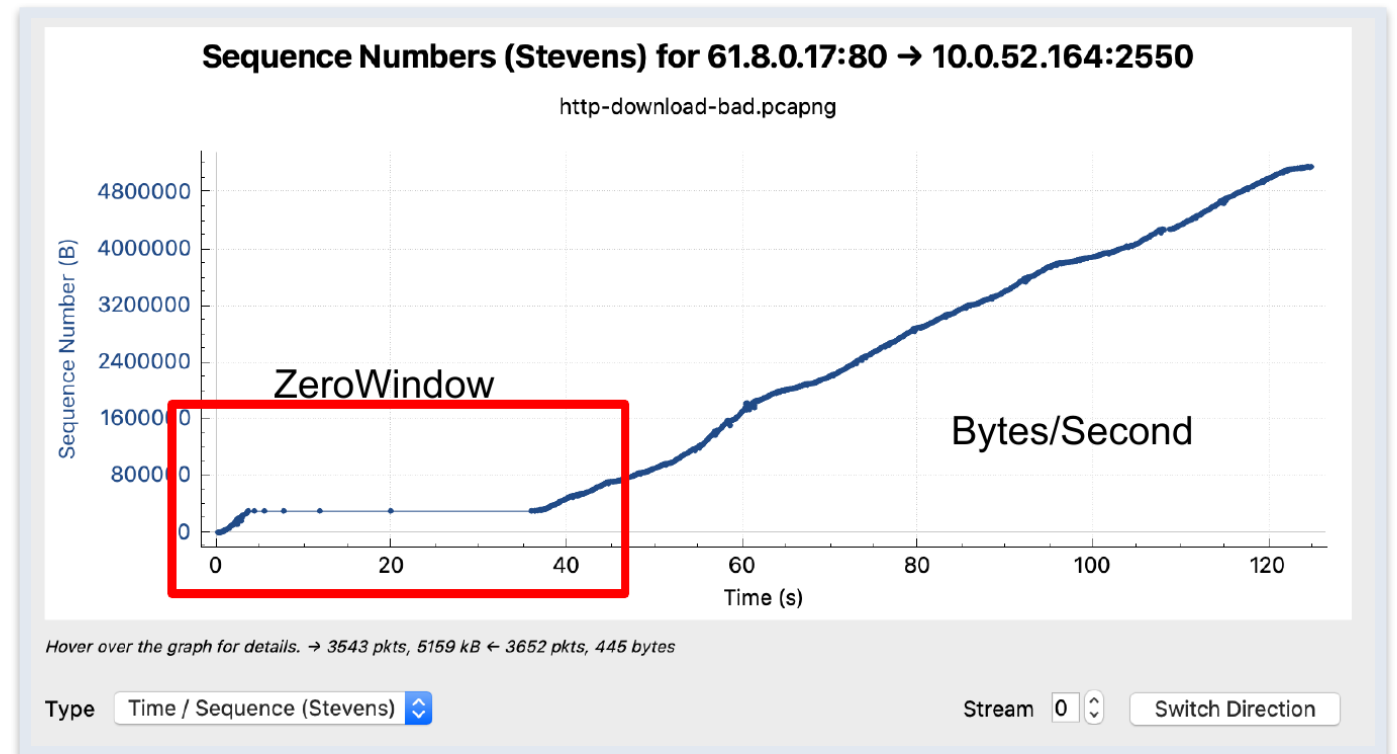
- Statistics > TCP Stream Graphs > Round Trip Time.
- Graphs are unidirectional — pick the data-sending direction.
- Spikes show where the path or receiver was slow to acknowledge.



RTT per acknowledged segment

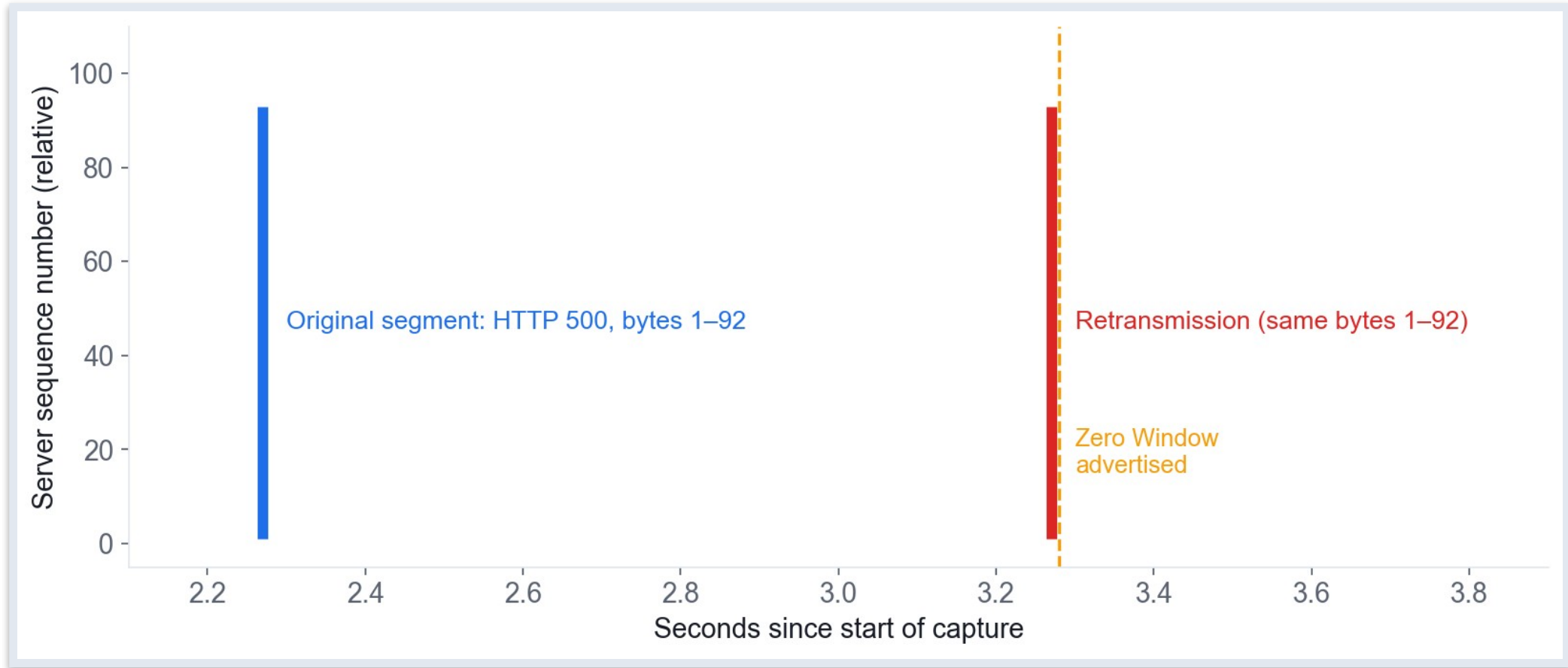
Time/Sequence (Stevens) Graph

- Sequence number vs time — the slope is throughput.
- Flat steps reveal stalls such as zero window.
- Vertical repeats indicate retransmissions.



A flat region marks a zero-window stall

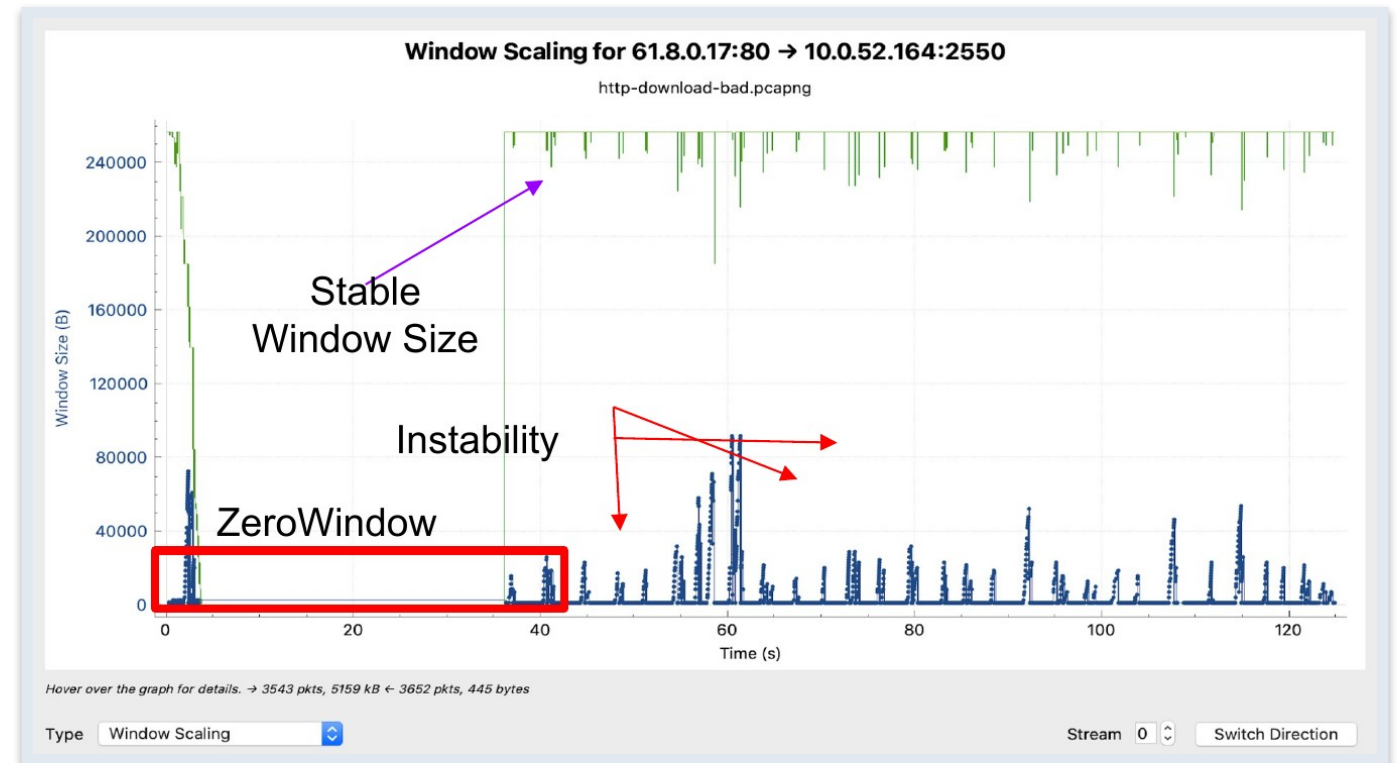
Stevens Graph of the Stalled Transfer



Drawn from stream 3 of the lab capture: the same bytes appear twice, then the zero window follows

Window Scaling Graph

- Plots the receiver's advertised window alongside bytes in flight.
- When bytes in flight meet the window, the receiver is the bottleneck.
- A collapsing window points at the receiving application.



Stable window, instability and zero window

Graph bytes, RTT and recovery

LAB 15

Management wants a picture, not a packet list. You graph the stalled transfer so the retransmission and zero-window events are obvious.

You'll build

Graph notes with interval, unit, stream and limitation

Tools: Wireshark, TShark, Python

Lab 15 — What You'll Do

1

Open the Stevens
time/sequence graph

2

Graph retransmission
and zero-window events

3

Compare RTT samples
with the handshake

4

Write graph notes with
units and limits

Full step-by-step instructions: [Learner Guide · labs/lab-15-graph-bytes-rtt-and-recovery/LAB-15-Instructions.pdf](#)

Expected Evidence — Lab 15

tcp.analysis.retransmission tcp.analysis.zero_window					
No.	Time	Source	Destination	Protocol	Info
51	3.270000	192.0.2.20	192.0.2.10	TCP	[TCP Retransmission] 80 → 51004 [PSH, ACK] Seq=1 ...
52	3.280000	192.0.2.10	192.0.2.20	TCP	[TCP ZeroWindow] 51004 → 80 [ACK] Seq=70 Ack=93 W...
branch-office.pcap		Displayed: 2			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Graph bytes, RTT and recovery

Test it

The supplied `tcp.analysis.retransmission || tcp.analysis.zero_window` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Traffic Graphs

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

An I/O graph bins observed events; units and interval determine what it means.

3

TCP time-sequence graphs show byte progress, stalls and repeated sequence ranges.

4

RTT samples, application response time and throughput answer different questions.

Tea Break

15 minutes

TOPIC 16

HTTP and HTTP/2 Analysis

Inspect HTTP outcomes and HTTP/2 frames

Key Concepts — HTTP and HTTP/2 Analysis

1

HTTP response codes describe application outcomes after transport delivery.

2

A response time contains more than network latency; pair the correct request and response.

3

HTTP/2 multiplexes frames; encrypted HTTP/2 needs authorised TLS secrets to inspect payload.

HTTP Response Codes

2xx / 3xx

- 200 OK
- 301/302 redirect
- 304 not modified

4xx client

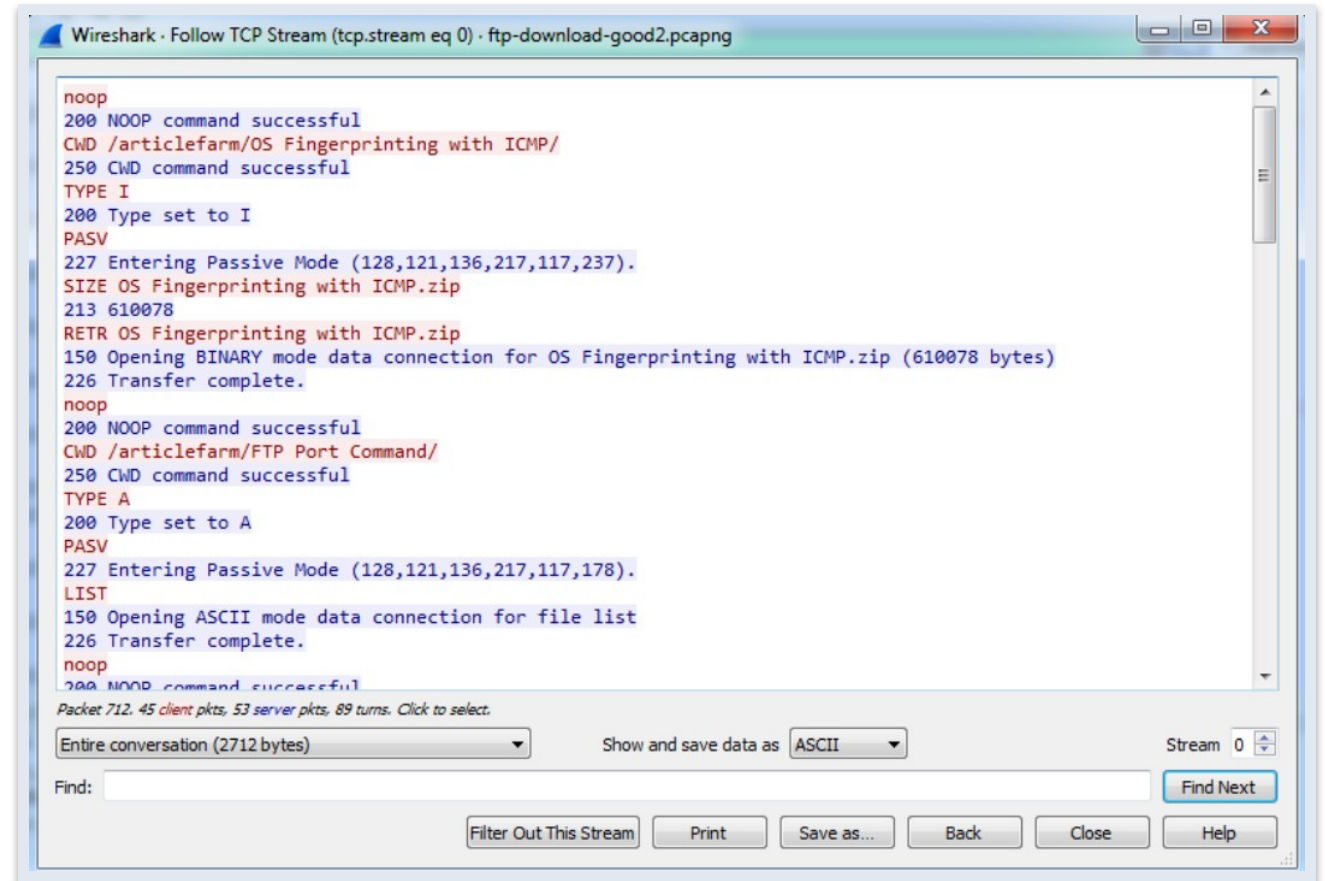
- 400 bad request
- 403 forbidden
- 404 not found

5xx server

- 500 internal error
- 502 bad gateway
- 503 unavailable

Follow the Stream

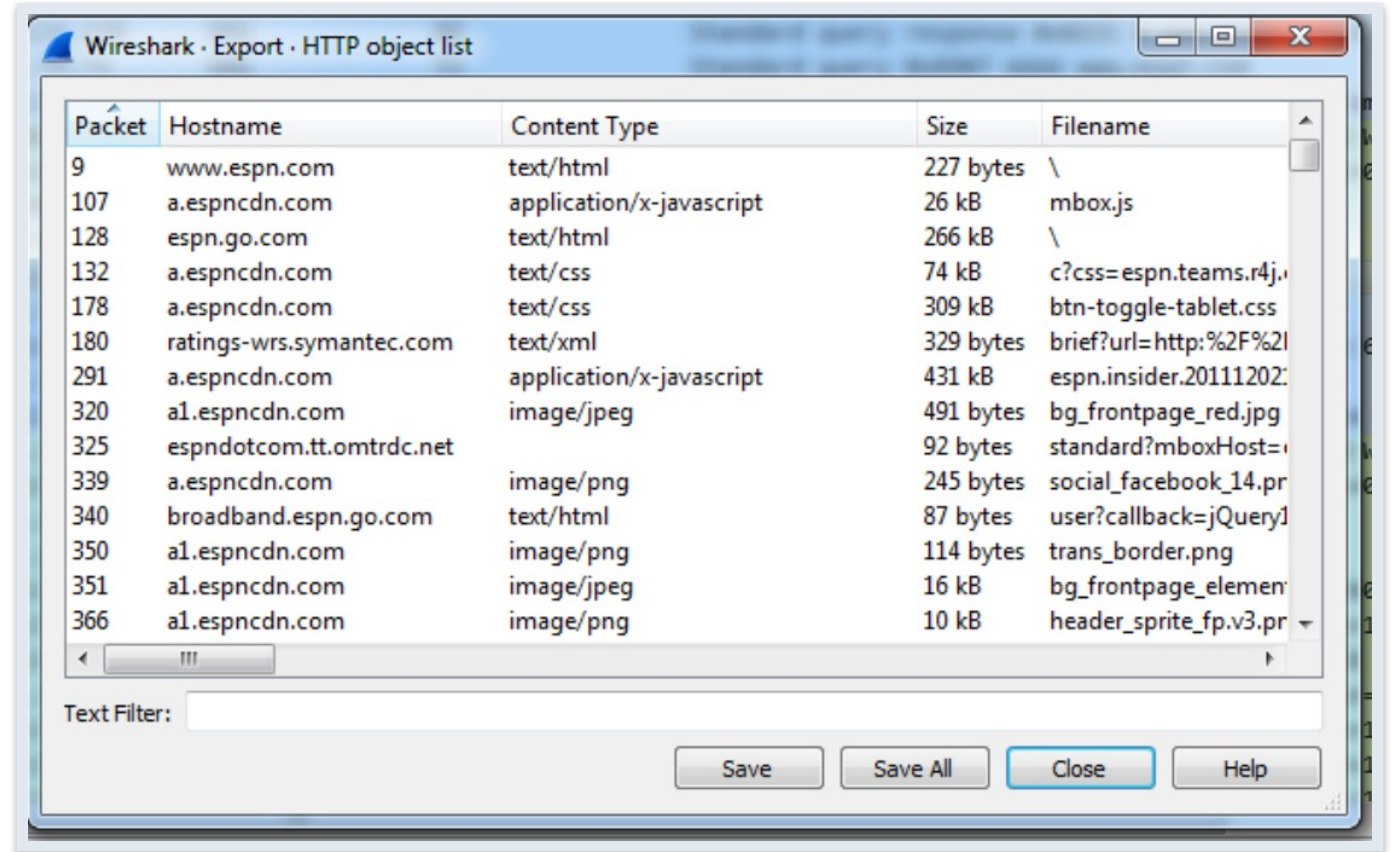
- Right-click > Follow > TCP Stream (or HTTP Stream) reassembles the conversation.
- Client data in red, server data in blue.
- The filter changes to tcp.stream == N — clear it to return.



Follow Stream reassembles the transferred data

Export HTTP Objects

- File > Export Objects > HTTP lists every transferred object.
- Save files for inspection — handle unknown content safely.
- http.time measures each response time.



File | Export Objects | HTTP

HTTP/1.1 vs HTTP/2

HTTP/1.1

- Text headers, one request at a time per connection
- Readable directly in the packet list
- `http.request.method`, `http.response.code`

HTTP/2

- Binary frames multiplexed into streams
- Usually inside TLS (ALPN h2)
- `http2.type`, `http2.streamid`; Decode As for clear-text test ports

Export Objects — and Prove What You Exported

1 File > Export Objects > HTTP saves every transferred object.

2 Hash each file (`shasum -a 256 <file>`) and record the hash in your report.

3 Never open or run exported files on your workstation.

4 TShark: `tshark -r file -q --export-objects http,outputs/objects`

HTTP/3 and QUIC

What changes

- HTTP/3 runs over QUIC on UDP 443
- QUIC encrypts almost all transport headers
- Connection IDs replace the 4-tuple

What you can still see

- Filter: quic
- Initial packets and SNI (with care)
- Full decryption needs the key log, as with TLS

Inspect HTTP outcomes and HTTP/2 frames

LAB 16

The web team wants to know which pages fail and whether the new HTTP/2 test service is negotiating correctly.

You'll build

An HTTP outcome summary and an exported object

Tools: Wireshark, TShark, Python

Lab 16 — What You'll Do

1

Pair four URLs with their status codes

2

Follow the stream and export an object

3

Decode port 8080 as HTTP/2

4

Summarise outcomes and next steps

Full step-by-step instructions: [Learner Guide · labs/lab-16-inspect-http-outcomes-and-http-2-frames/LAB-16-Instructions.pdf](#)

Expected Evidence — Lab 16

http.response.code >= 400					
No.	Time	Source	Destination	Protocol	Info
41	2.130000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 404 Not Found (text/plain)
50	2.270000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 500 Internal Server Error (text/plain)
branch-office.pcap Displayed: 2					
http2.type == 4 [Decode As: tcp.port==8080,http2]					
No.	Time	Source	Destination	Protocol	Info
66	3.460000	192.0.2.10	192.0.2.20	HTTP2	Magic, SETTINGS[0]
branch-office.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Inspect HTTP outcomes and HTTP/2 frames

Test it

The supplied `http.response.code >= 400` expression matches 2 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — HTTP and HTTP/2 Analysis

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

HTTP response codes describe application outcomes after transport delivery.

3

A response time contains more than network latency; pair the correct request and response.

4

HTTP/2 multiplexes frames; encrypted HTTP/2 needs authorised TLS secrets to inspect payload.

Lunch Break

1 hour

TOPIC 17

TLS-Encrypted Traffic Analysis

Compare encrypted and decrypted views

Key Concepts — TLS-Encrypted Traffic Analysis

1

Without session secrets, encrypted application records do not disclose HTTP payload.

2

A controlled key log enables authorised inspection of this synthetic TLS exchange.

3

A server RSA private key alone cannot decrypt modern ECDHE or TLS 1.3 sessions.

TLS Handshake Evidence

Client Hello

- SNI = requested server name
- Offered versions and ciphers
- `tls.handshake.type == 1`

Server Hello

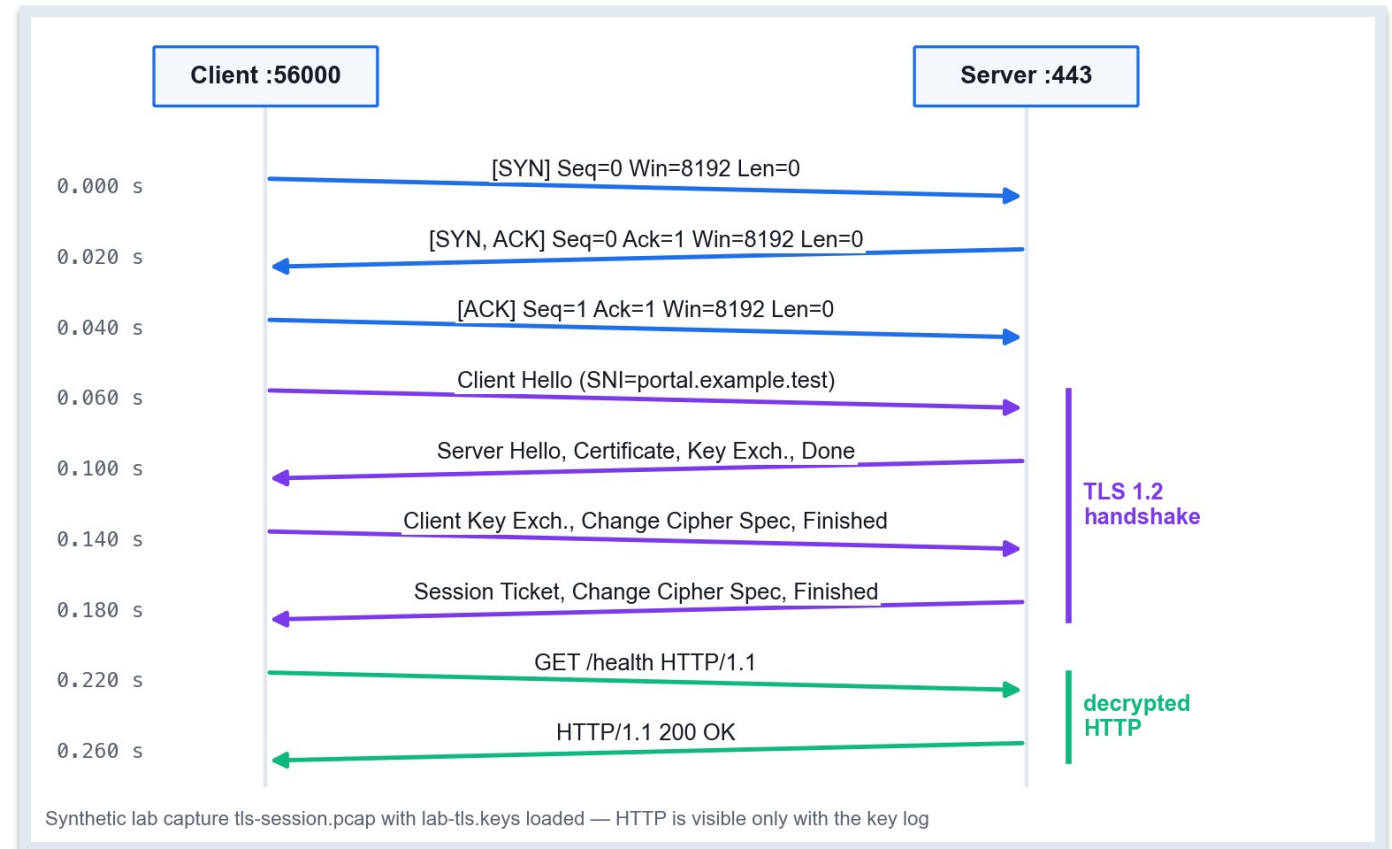
- Chosen version and cipher
- Certificate (TLS 1.2)
- `tls.handshake.type == 2`

Application Data

- Encrypted records
- Sizes and timing still visible
- `tls.record.content_type == 23`

TLS 1.2 Handshake in the Lab Capture

- TCP handshake, then Client Hello with SNI portal.example.test.
- Server Hello, Certificate and key exchange; both sides send Finished.
- With lab-tls.keys loaded, GET /health and 200 OK appear as HTTP.



tls-session.pcap, decrypted view

Decrypting TLS — Authorised Only

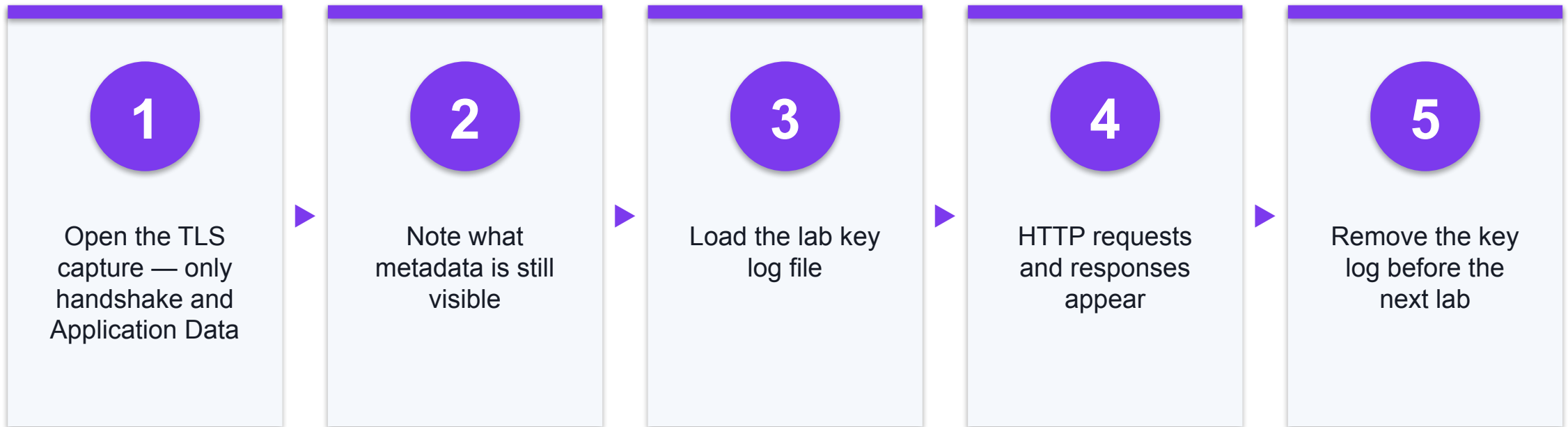
Works

- Key log file from the client (SSLKEYLOGFILE)
- Preferences > Protocols > TLS > (Pre)-Master-Secret log filename
- Works for TLS 1.2 and TLS 1.3

Does not work

- RSA private key with (EC)DHE or TLS 1.3 sessions
- Captures without the matching session secrets
- Anything you are not authorised to inspect

Encrypted vs Decrypted View



Compare encrypted and decrypted views

LAB 17

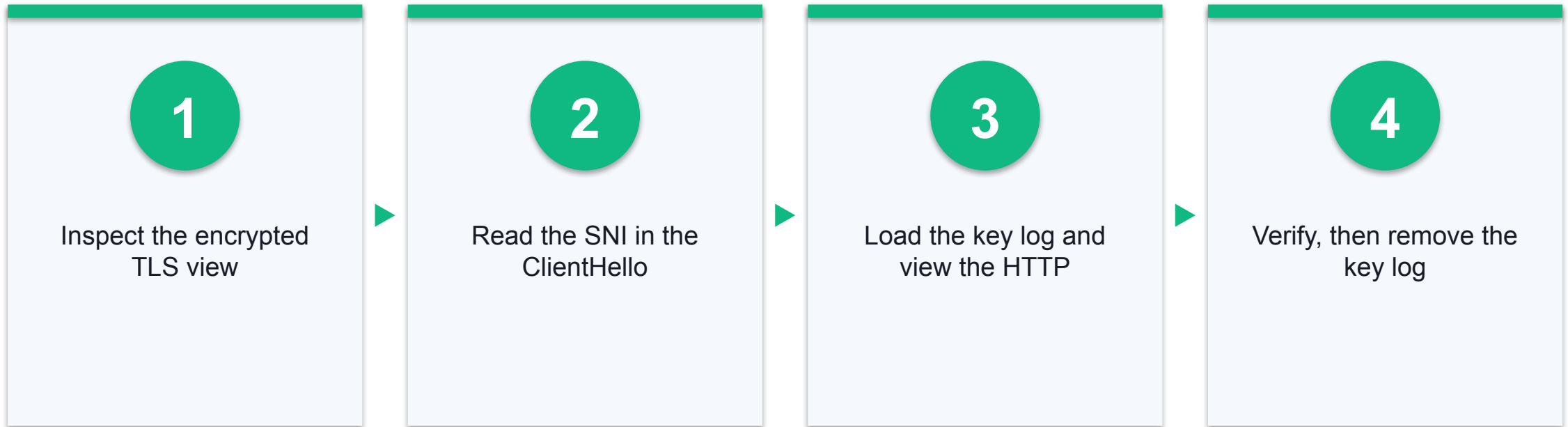
Under an authorised test, the portal's HTTPS traffic must be inspected. You compare what an analyst sees with and without the session secrets.

You'll build

A comparison of the encrypted and decrypted views

Tools: Wireshark, TShark, Python

Lab 17 — What You'll Do



Full step-by-step instructions: [Learner Guide · labs/lab-17-compare-encrypted-and-decrypted-views/LAB-17-Instructions.pdf](#)

Expected Evidence — Lab 17

tls.handshake.type == 1					
No.	Time	Source	Destination	Protocol	Info
4	0.060000	192.0.2.10	192.0.2.20	TLSv1	Client Hello (SNI=portal.example.test)
tls-session.pcap Displayed: 1					
http.request					
No.	Time	Source	Destination	Protocol	Info
(no packets match – expected for this view)					
tls-session.pcap Displayed: 0					
http.request [TLS key log loaded]					
No.	Time	Source	Destination	Protocol	Info
12	0.220000	192.0.2.10	192.0.2.20	HTTP	GET /health HTTP/1.1
tls-session.pcap Displayed: 1					
http.response.code == 200 [TLS key log loaded]					
No.	Time	Source	Destination	Protocol	Info
14	0.260000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK
tls-session.pcap Displayed: 1					

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Compare encrypted and decrypted views

Test it

The supplied `tls.handshake.type == 1` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — TLS-Encrypted Traffic Analysis

1

You can now: diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

2

Without session secrets, encrypted application records do not disclose HTTP payload.

3

A controlled key log enables authorised inspection of this synthetic TLS exchange.

4

A server RSA private key alone cannot decrypt modern ECDHE or TLS 1.3 sessions.

Tea Break

15 minutes

TOPIC 18

Ten Troubleshooting Steps and Reporting

Produce an evidence-led incident report

KEY CONCEPTS

Key Concepts — Ten Troubleshooting Steps and Reporting

1

Begin with scope and a baseline, then focus with endpoints, filters, time and graphs.

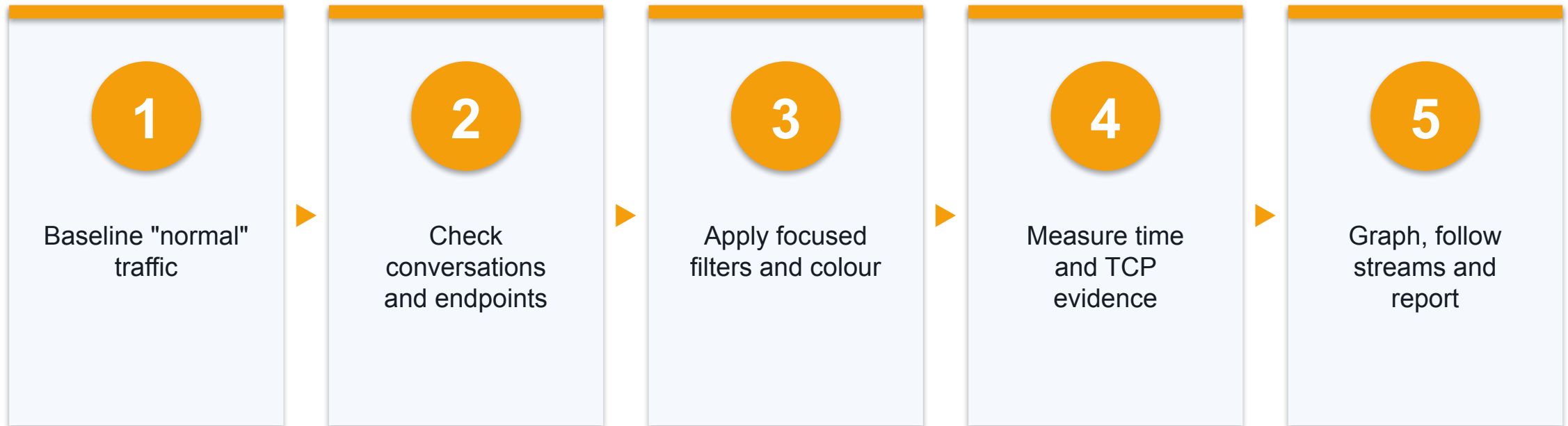
2

Separate observed facts from hypotheses and specify what new evidence would test them.

3

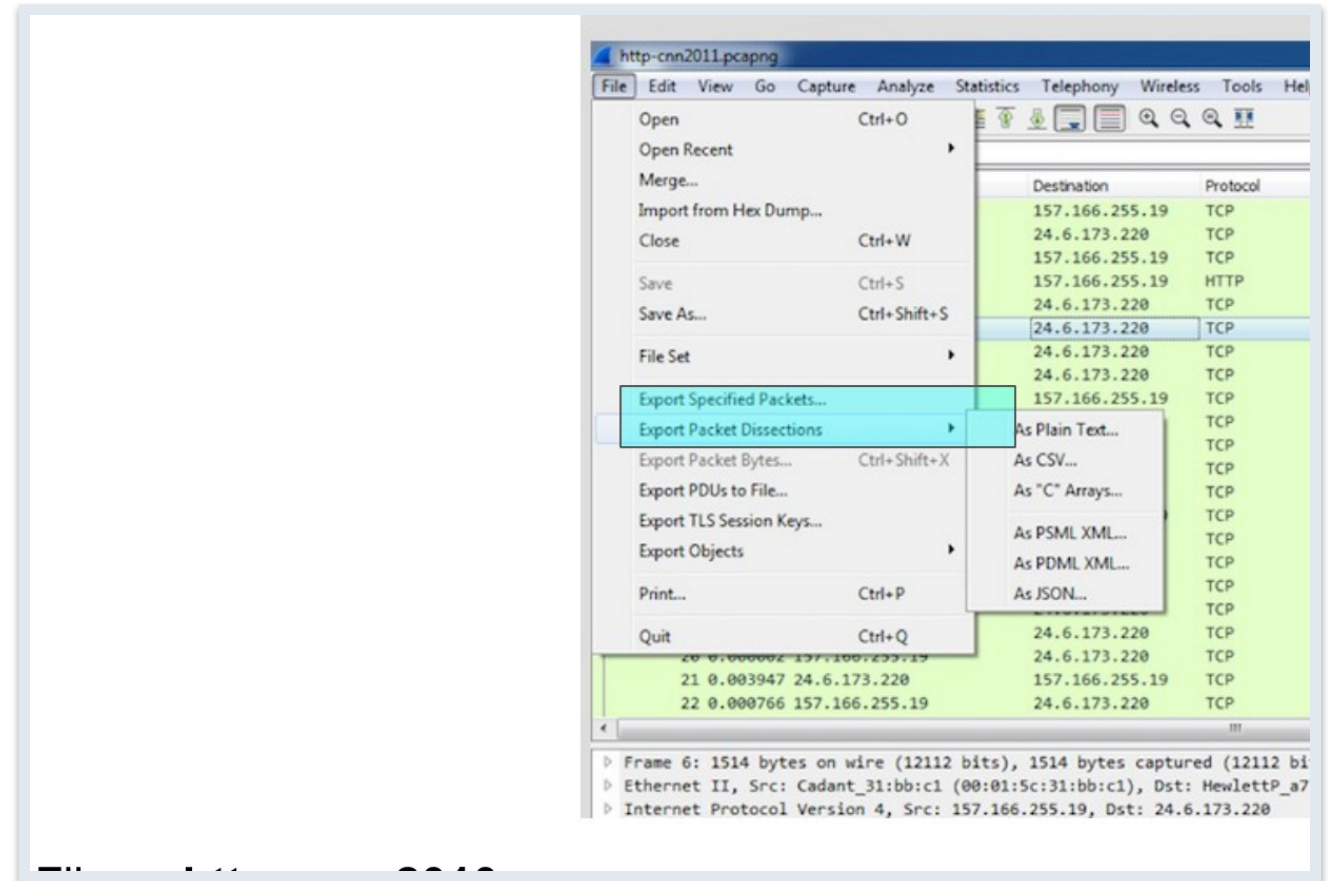
A useful report states affected flow, measurement, impact, recommended action and uncertainty.

Ten-Step Troubleshooting Method



Export and Preserve Evidence

- File > Export Specified Packets saves only the relevant frames.
- Export Packet Dissections produces text/CSV for reports.
- Keep the original capture unchanged and record its hash.



File | Export Packet Dissections

Writing the Incident Report

Observed facts

- Frame numbers
- Filters used
- Measured times and counts

Interpretation

- What the evidence suggests
- Alternative explanations
- Confidence level

Next actions

- Further captures needed
- Owner and system to check
- Limitations of this trace

Security Patterns to Recognise

1

Port scan

Many SYNs to different ports, answered by RST

2

ARP spoofing

One IP, two MACs; gratuitous floods

3

DNS tunnelling

Very long or random query names

4

Cleartext credentials

http.authorization, ftp PASS, telnet

5

Beaconing

Regular small connections to one host

6

Unusual ports

Services where they should not be

Build and Use a Baseline

1

Capture “normal” at known-good times: login, file copy, a web page.

2

Note protocols, top talkers, typical response times and packet sizes.

3

Compare a problem trace against the baseline, not against guesses.

4

Keep baselines with dates — networks change.

Keep Learning

1

Wireshark User's Guide

wireshark.org/docs

2

Sample captures

wiki.wireshark.org/SampleCaptures

3

Kurose & Ross Wireshark labs

gaia.cs.umass.edu/kurose_ross/wireshark.php

4

Ask Wireshark

ask.wireshark.org

5

SharkFest talks

youtube.com/@WireSharkFest

6

Wireshark Certified Analyst

wireshark.org/certifications

Produce an evidence-led incident report

LAB 18

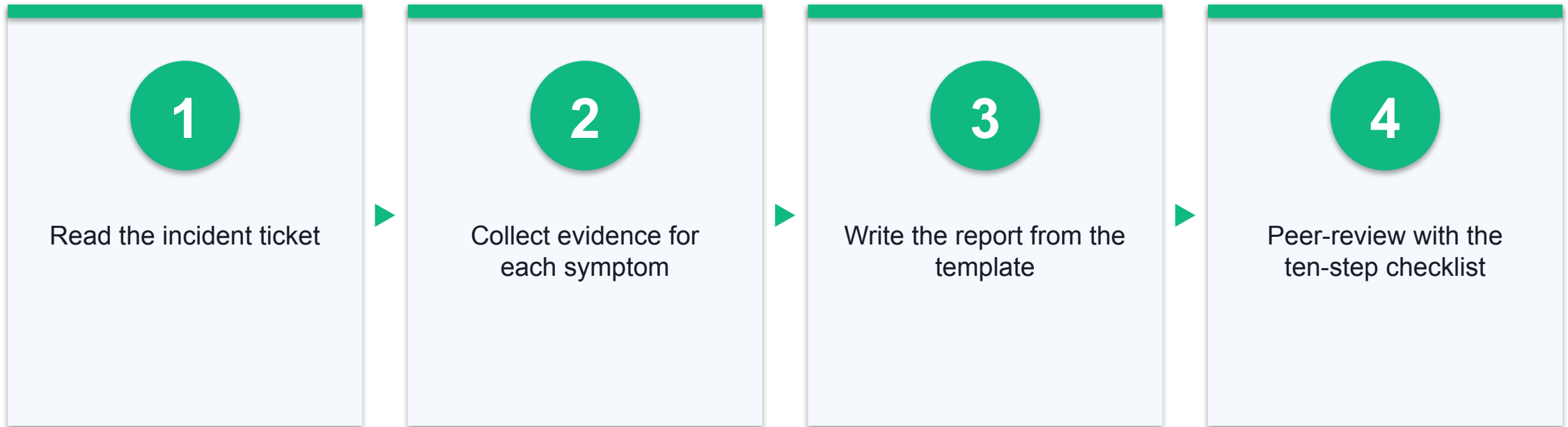
Ticket BR-104 combines several user complaints. You produce a reproducible incident report that separates observed facts from hypotheses.

You'll build

An evidence-led incident report

Tools: Wireshark, TShark, Python

Lab 18 — What You'll Do



Full step-by-step instructions: [Learner Guide · labs/lab-18-produce-an-evidence-led-incident-report/LAB-18-Instructions.pdf](#)

Expected Evidence — Lab 18

http.response.code == 500					
No.	Time	Source	Destination	Protocol	Info
50	2.270000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 500 Internal Server Error (text/plain)
branch-office.pcap		Displayed: 1			

Produced by TShark 4.6 from this lab's own synthetic capture with the lab filter(s) applied — your packet list should match.

Produce an evidence-led incident report

Test it

The supplied `http.response.code == 500` expression matches 1 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Recap — Ten Troubleshooting Steps and Reporting

1

You can now: report observed facts, hypotheses and next actions with defensible evidence.

2

Begin with scope and a baseline, then focus with endpoints, filters, time and graphs.

3

Separate observed facts from hypotheses and specify what new evidence would test them.

4

A useful report states affected flow, measurement, impact, recommended action and uncertainty.



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Capture & profile

Plan scoped captures and configure a reproducible analyst profile.

2

Navigate & time

Navigate, filter, summarize and time network exchanges.

3

Core protocols

Interpret DNS, ARP, IPv4, ICMP and UDP evidence.

4

TCP & applications

Diagnose TCP and application behaviour, including HTTP and authorised TLS inspection.

5

Evidence reporting

Report observed facts, hypotheses and next actions with defensible evidence.

Continue Your Learning

1

All course material stays on the LMS at <https://lms-tms.tertiaryinfotech.com>.

2

The 18 labs and their captures are in the course GitHub repository — re-run them any time.

3

Apply the ten-step troubleshooting checklist to an authorised capture at work.

4

Explore networking and cyber-security courses at www.tertiarycourses.com.sg.

Keep Practising After the Course

1

Re-run the labs on your own machine — repetition is what makes the skills stick.

2

Apply the techniques to a real process or project in your own organisation.

3

Your course material stays available at <https://lms-tms.tertiaryinfotech.com/>.

4

Explore the follow-on courses at www.tertiarycourses.com.sg.

THANK YOU FOR ATTENDING

Thank You!

Keep the evidence, explain the uncertainty — and keep practising with the 18 labs on your own machine.